
py-mathx-lab Documentation

Release 0.1.0.dev25

Walter Weinmann

Dec 28, 2025

CONTENTS

1	Start here	3
2	Run one experiment	5
3	Latest	7
3.1	Mathematical experimentation	7
3.2	Valid Tags	9
3.3	Experiments Gallery	10
3.4	Background	104
3.5	Getting started	114
3.6	Development	116
3.7	References	119
3.8	PDF download	119
	Bibliography	121

Small, reproducible math experiments implemented in Python.

- **Audience:** curious engineers, students, and researchers
- **Idea:** each experiment is a self-contained runnable module with a short write-up
- **Goal:** a growing, searchable “lab notebook” of experiments

START HERE

- *Mathematical experimentation* - what “experiments” in mathematics mean and how to read this repo
- *Experiments Gallery* - experiment gallery (IDs, tags, how to run)
- *Valid Tags* - central directory of valid tags for experiments
- *Background* - mathematical background for experiments
- *Getting started* - install, setup, and run your first experiment
- *Development* - Makefile workflow, CI, coding conventions
- *References* - bibliography and reading list
- *PDF download* - download the PDF version of these docs

RUN ONE EXPERIMENT

```
make uv-check  
make venv  
make install-dev  
make run EXP=e001 ARGS="--out out/e001 --seed 1"
```


- – E011 - E011: *Heuristic rarity of Mersenne primes*
-

3.1 Mathematical experimentation

Mathematical experimentation is the practice of using examples, computation, and visualization to discover structure, to generate conjectures, find counterexamples, estimate quantities, and build intuition that later supports (or refutes) formal arguments.

It is *not* “proof by computer output”. Instead, experiments are a disciplined way to ask better questions and to stress-test ideas-especially now that modern computers make it easy to explore large search spaces, high-precision numerics, and rich visualizations.

This repository, **py-mathx-lab**, is a small “lab notebook” of such experiments: compact, reproducible, and readable.

3.1.1 What counts as an “experiment” in mathematics?

An experiment is a finite procedure that produces evidence about a mathematical claim or object. Typical outcomes:

- **Conjecture generation:** patterns suggest statements that might be true (or false).
- **Counterexample search:** systematic exploration tries to break a hypothesis early.
- **Quantitative exploration:** estimate constants, rates, limits, or distributions.
- **Model checking:** validate (or invalidate) approximations and heuristics.
- **Visualization:** reveal structure that is hard to see symbolically.

The modern viewpoint-where computation is a genuine part of mathematical discovery-is widely discussed under the name *experimental mathematics* ([BB05, BBC04, Bor05]). Some authors emphasize “plausible reasoning” supported by computation, paired with careful verification and eventual proof ([Bor09, BB08]). Other texts take a more problem-driven, exploratory style aimed at students and engineers ([LCD+03]), or present computation as a tool to formulate concrete research problems ([Arn15]).

3.1.2 Why computers change the game

Computers do not replace mathematical thinking-but they expand what is feasible to *inspect*:

- **Scale:** enumerate millions of cases (to find the first failure or build confidence).
- **Precision:** compute with high-precision floats or exact rationals/integers to avoid roundoff illusions.
- **Multiple lenses:** combine numerics, exact arithmetic, symbolic manipulation, and plotting.
- **Search:** automate discovery (parameter sweeps, optimization, random sampling, heuristics).
- **Reproducibility:** re-run the same pipeline with fixed seeds and pinned dependencies.

Used well, computation turns “I wonder if...” into “here is evidence, here are edge cases, and here is what we should prove next”.

3.1.3 Experiments vs. proofs

A proof is the end of the story; an experiment is often the beginning.

Experiments are excellent for:

- **disproving** statements (one counterexample ends it),
- identifying **what is actually true** (after a naive conjecture fails),
- suggesting **lemmas** and **invariants** that make a proof possible.

But experiments can also mislead. Common failure modes:

- floating point error and catastrophic cancellation,
- plotting artifacts,
- “pattern matching” based on too few samples,
- unintentional selection bias (“I tried the cases that worked”).

The goal is to use experiments to *reduce uncertainty*, not to hide it.

3.1.4 Targets for py-mathx-lab

py-mathx-lab aims to be a practical, long-lived collection of experiments with shared conventions:

1. **Reproducible runs**

Each experiment is runnable as a module, writes results to a single output directory, and (when relevant) uses a fixed seed.

2. **Readable code**

The code should be short, well-typed, and structured so readers can modify it.

3. **Useful artifacts**

Each experiment should generate at least one of:

- a figure
- a table / summary statistics
- a counterexample / witness object
- a short narrative explaining what was learned

4. **Clear boundaries**

An experiment write-up should state:

- what is being tested,
- what counts as “success” or “failure”,
- what might invalidate the result (precision limits, domain constraints, runtime limits).

5. **Traceability**

Each page includes references to books/papers that motivated the work or explain the background.

3.1.5 Repository conventions for experiments

An experiment page should usually include:

- **Goal** (one paragraph)
- **How to run** (a command that works from the repo root)
- **Parameters** (including defaults and ranges, if swept)

- **Results** (figures/tables and a short interpretation)
- **Notes / pitfalls** (numerical caveats, surprising behavior)
- **References** (bibliography keys)

In code, prefer:

- deterministic outputs (fixed seeds, stable sorting),
- explicit configuration objects / CLI arguments,
- sanity checks (dimension checks, bounds checks, invariants),
- cross-checks (e.g., float vs. exact, two independent formulas).

3.1.6 Examples of good “experiment themes”

The experiment format supports many domains, for example:

- **Numerical analysis:** error landscapes, stability regions, conditioning, Monte Carlo integration.
- **Number theory:** continued fractions and convergents, integer sequences, modular patterns, primality heuristics.
- **Geometry/topology:** random point clouds, curvature approximations, combinatorial invariants.
- **Optimization/probability:** stochastic search behavior, concentration phenomena, empirical distributions.

A concrete example of a rich, experiment-friendly topic is continued fractions, where it is natural to compute and plot convergents, partial quotient statistics, and periodicity phenomena ([BvdPSZ14]).

Next steps: start with *Getting started*, browse the *Valid Tags* directory, and then browse the *Experiments Gallery* gallery.

3.2 Valid Tags

This page defines the allowed tags for experiments in **py-mathx-lab**. Tags are used in the *Experiments Gallery* and individual experiment pages to categorize content.

3.2.1 Primary Tags (Domains)

These represent the broad mathematical area of the experiment.

Tag	Description
<code>analysis</code>	Calculus, real/complex analysis, limits, and approximation.
<code>conjecture-generation</code>	Patterns suggest statements that might be true (or false).
<code>counterexample-search</code>	Systematic exploration tries to break a hypothesis early.
<code>model-checking</code>	Validate (or invalidate) approximations and heuristics.
<code>number-theory</code>	Properties of integers, divisibility, and prime numbers.
<code>quantitative-exploration</code>	Estimate constants, rates, limits, or distributions.
<code>visualization</code>	Reveal structure that is hard to see symbolically.

3.2.2 Secondary Tags

- `carmichael` — Carmichael numbers (absolute Fermat pseudoprimes).
- `factorization` — Integer factorization methods and hardness.
- `fermat` — Fermat numbers and Fermat primes.
- `primorial` — Primorials, Euclid numbers, primorial primes.

- `pseudoprime` — Pseudoprimes, primality-test failures.
- `semiprime` — Semiprimes, RSA-type composites.
- `wieferich` — Wieferich primes and related congruences.

These provide more specific detail about the techniques or sub-topics involved.

Tag	Description
<code>classification</code>	Grouping objects into classes based on shared properties.
<code>exploration</code>	Open-ended search for patterns or properties.
<code>numerics</code>	Heavy use of floating-point or high-precision computation.
<code>open-problems</code>	Related to famous unproven conjectures.
<code>optimization</code>	Finding maxima, minima, or best-fit parameters.
<code>perfect</code>	Related specifically to perfect, abundant, or deficient numbers.
<code>search</code>	Systematic search through a large state space.
<code>sigma</code>	Related to the sum-of-divisors function $\sigma(n)$.
<code>taylor</code>	Related to Taylor series and their approximations.

3.2.3 Usage

When adding a new experiment:

1. Choose at least one **Primary Tag** (Domain or Type).
2. Choose one or more **Secondary Tags** (Topics & Methods).
3. Add them to the `**Tags:**` line in your `.md` file.
4. Update the *Experiments Gallery* using the corresponding CSS classes (`tag-primary` for primary tags, `tag-secondary` for secondary tags).

3.3 Experiments Gallery

A compact, image-first overview of the experiments in **py-mathx-lab**.

3.3.1 E001 — Taylor Error Landscapes

Tags: analysis, quantitative-exploration, visualization, numerics, taylor (see *Valid Tags*)

Goal

Build intuition for Taylor truncation error by visualizing the absolute error landscape $|\sin(x) - T_n(x; x_0)|$ over a domain while varying:

- the polynomial degree n ,
- the expansion center x_0 .

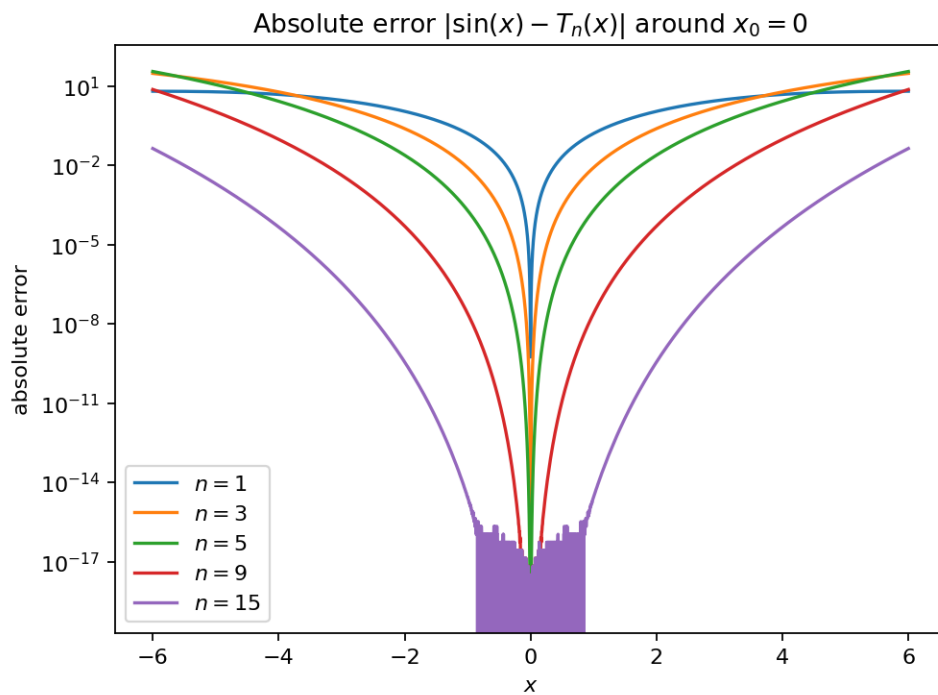
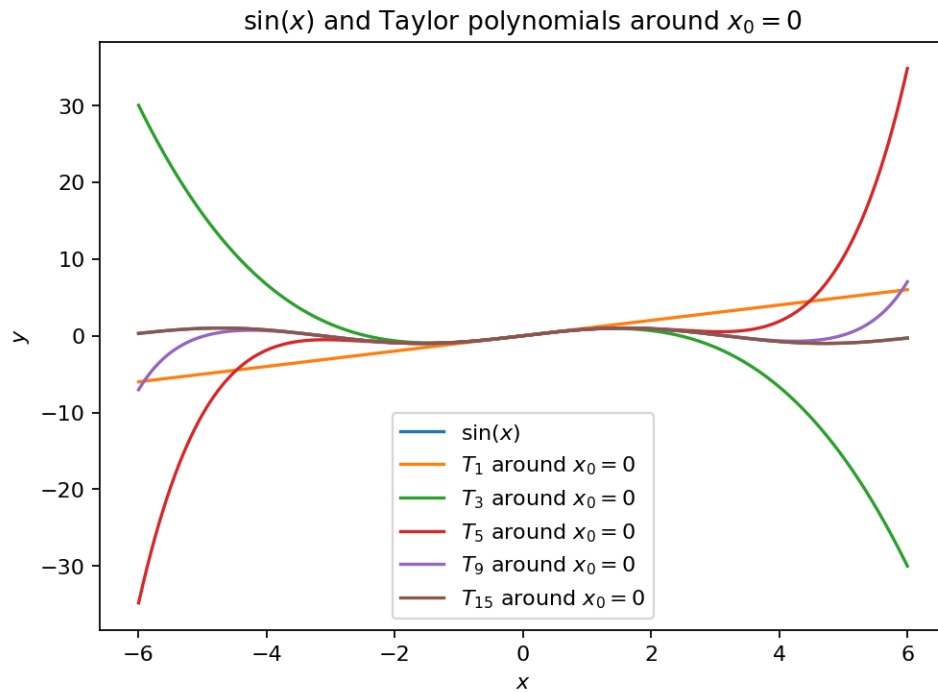
Background (quick refresher)

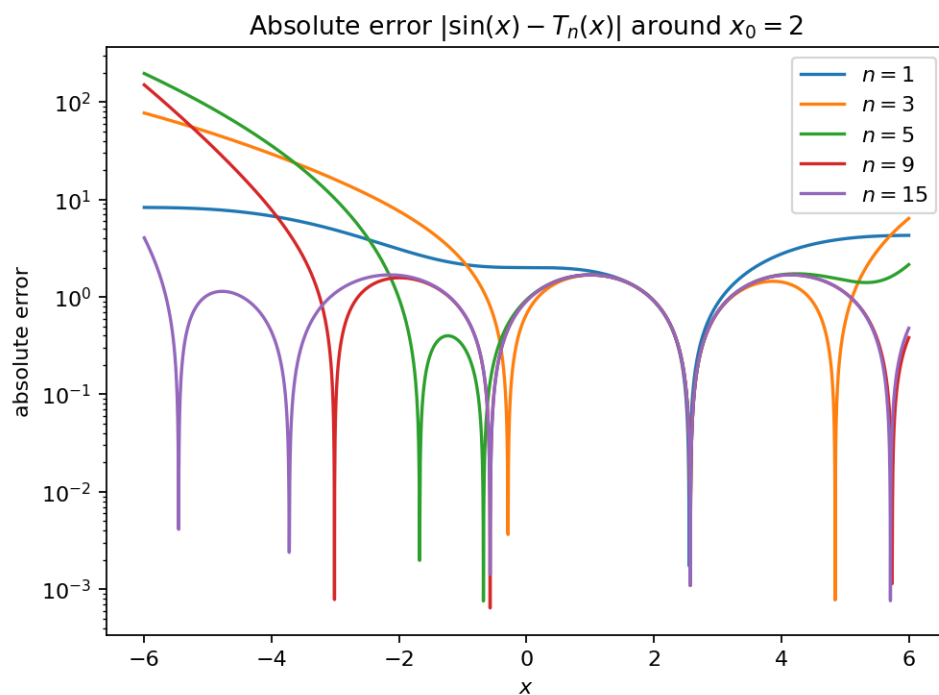
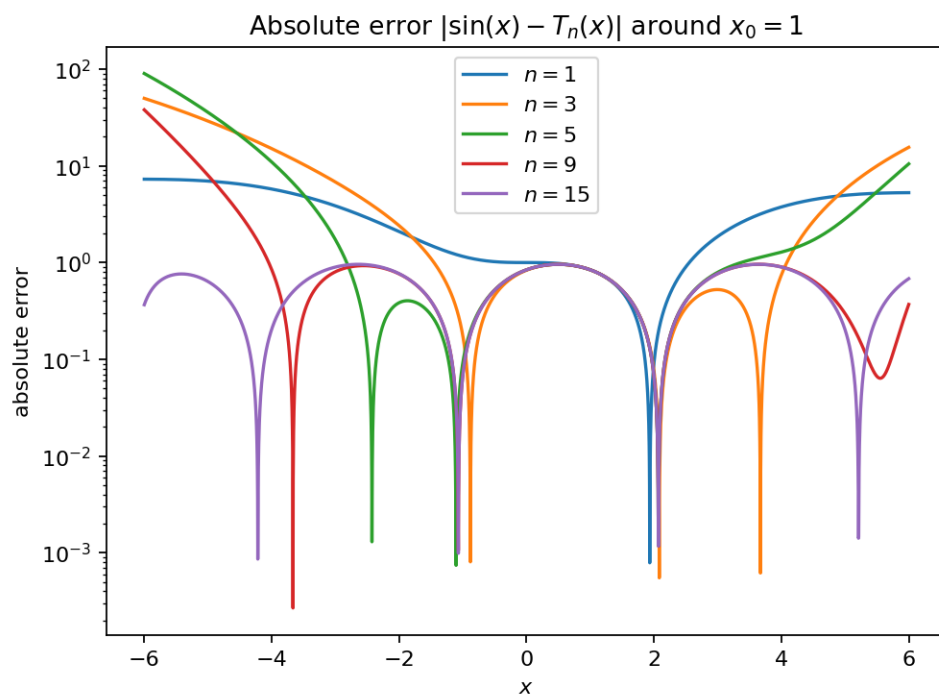
If you want a short mathematical recap first, read: *Taylor series refresher*.

Research question

How does the approximation error of Taylor polynomials for $\sin(x)$ depend on the polynomial degree n and the expansion center x_0 , over a fixed domain of x ?

Concretely: for a chosen grid of (n, x_0) , what does the error landscape $E_{n, x_0}(x) = |\sin(x) - T_n(x; x_0)|$ look like, and where do numerical artifacts start to dominate the truncation error?





Why this qualifies as a mathematical experiment

This page is not just a worked example or a derivation — it is an *experiment* in the sense of **experimental mathematics**: a finite, reproducible procedure that produces **evidence** about how a mathematical object behaves.

For E001, the object is the family of Taylor polynomials $T_n(x; x_0)$ for $\sin(x)$, and the observable is the error function $E_{n, x_0}(x) = |\sin(x) - T_n(x; x_0)|$. The experiment qualifies because it:

- **Explores a parameter space:** it varies degree n and center x_0 and inspects how the entire error landscape changes.
- **Generates testable conjectures:** e.g. “there is a widening low-error region around x_0 as n increases” and “improvement is not uniform across a fixed interval”.
- **Searches for failure modes / edge cases:** large $|x - x_0|$, high degrees, and wide domains can expose numerical artifacts that *look* like truncation error but are actually floating-point limitations.
- **Produces artifacts that can be checked independently:** plots and parameter snapshots make it easy to compare runs, reproduce the same conditions, and verify that an observed pattern is not accidental.

The goal is to turn “Taylor series should be good near x_0 ” into *structured evidence* about **where** and **how** the approximation is good (or bad), which then informs what one would try to prove or bound formally.

- How does the truncation error $|\sin(x) - T_n(x; x_0)|$ behave as we move away from the center x_0 ?
- How many terms are needed to achieve a specific precision (e.g., 10^{-6}) over a fixed interval?
- Does increasing the degree n always improve the result everywhere in the domain?

Experiment design

- **Target function:** $\sin(x)$
- **Evaluation:** $x \in [-2\pi, 2\pi]$ (default)
- **Parameters:**
 - degrees: $\{1, 3, 5, 7, 9\}$
 - centers: $\{0, \pi/2, \pi\}$
- **Outputs:**
 - Plot of $f(x)$ vs $T_n(x)$
 - Semi-log plot of $|f(x) - T_n(x)|$

How to run

```
make run EXP=e001 ARGS="--seed 1"
```

Artifacts are written under `out/e001/` (figures, parameters, and a short `report.md`).

What to expect

Qualitatively (and this is what the plots should confirm):

- near x_0 , the higher degree reduces the error quickly,
- away from x_0 , the approximation can degrade even for higher degrees,
- changing x_0 shifts the “low-error region”.

Results

After running the experiment, include (or regenerate) the figures in the documentation. The canonical output location is `out/e001/`. For publishing, copy one representative “hero” image into `docs/_static/experiments/` (see “Gallery images” below).

Notes / pitfalls

- Use **log-scale** for absolute error plots to see the full dynamic range.
- Be careful interpreting relative error near zeros of $\sin(x)$.
- Huge domains and high degrees can expose floating-point artifacts that are not truncation error.

Extensions

- **Alternative functions:** Repeat the experiment for $\exp(x)$ or $1/(1-x)$.
- **Relative error:** Plot $|(f - T_n)/f|$ instead of absolute error.
- **Automatic degree selection:** Find the minimal n such that error $< \epsilon$ on a given interval.

Gallery images (recommended)

To keep the experiment gallery attractive and stable:

1. run the experiment locally,
2. pick one representative output figure,
3. copy it into the repo under:

```
docs/_static/experiments/e001_hero.png
```

This allows the docs to show thumbnails without depending on generated `out/` artifacts.

References

See *References*. [BB05, Bor05, BB08]

3.3.2 E002: Even Perfect Numbers — Generator and Growth

Tags: number-theory, quantitative-exploration, visualization, numerics (see *Valid Tags*)

Highlights

- Generate even perfect numbers from known Mersenne exponents p .
- Plot digits and bit-length growth vs. p .
- Test the approximation $\log_{10}(N(p)) \approx 2p \log_{10}(2)$.

Goal

Generate **even perfect numbers** from known Mersenne prime exponents p and visualize how fast the numbers grow. Measure growth via **digit count**, **bit length**, and simple logarithmic approximations.

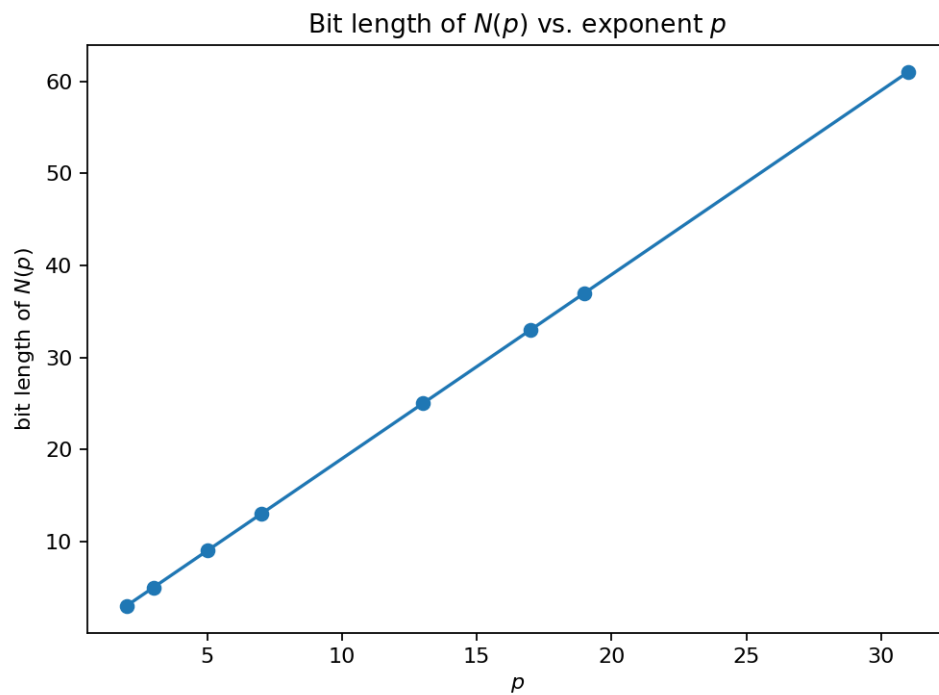
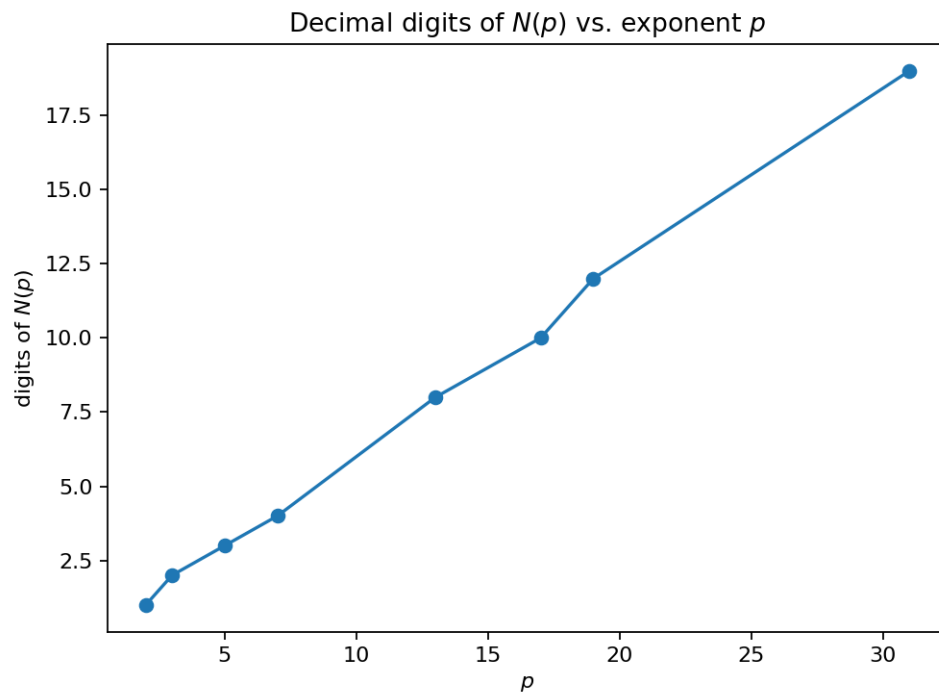
Research question

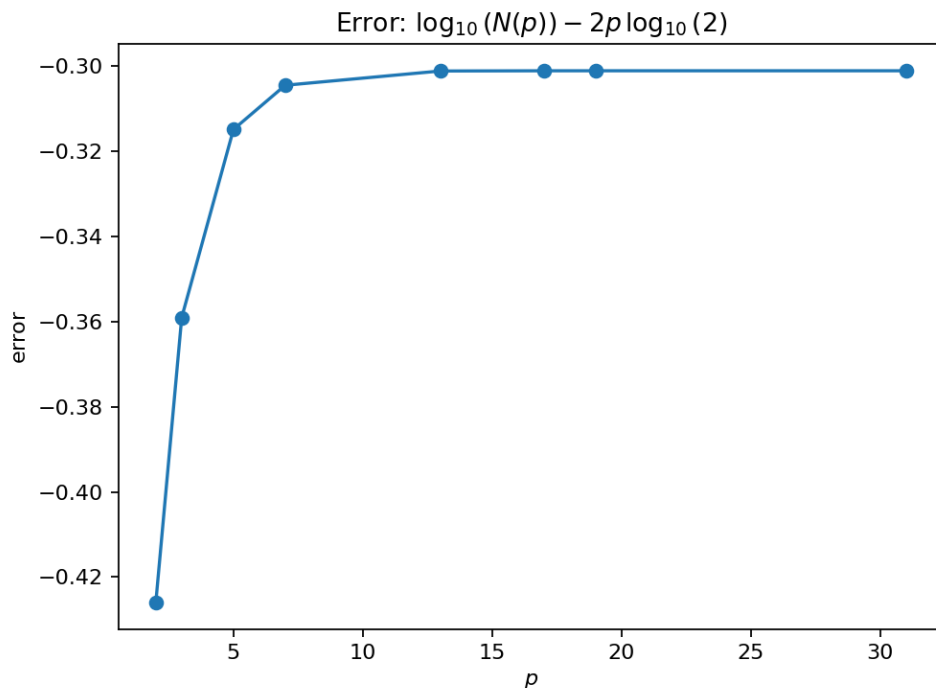
For

$$N(p) = 2^{p-1}(2^p - 1),$$

how do:

- `digits(N)`,
- `bit_length(N)`,





- $\log_{10}(N)$

scale with the exponent p ?

How accurate is the approximation

$$\log_{10}(N(p)) \approx 2p \log_{10}(2)?$$

Why this qualifies as a mathematical experiment

The Euclid–Euler theorem tells us exactly what even perfect numbers look like, but it does not directly convey how quickly the objects become astronomically large. This experiment uses computation and visualization to build quantitative intuition and test simple asymptotic approximations.

Experiment design

Inputs

- A curated list of known Mersenne prime exponents, e.g. $p \in \{2, 3, 5, 7, 13, 17, 19, 31, \dots\}$.

Observables

For each exponent p :

- $N(p)$ as a Python integer
- $\text{digits}(N(p))$
- $\text{bit_length}(N(p))$
- approximation error:

$$\Delta(p) = \log_{10}(N(p)) - 2p \log_{10}(2)$$

Plots

- p vs. digits
- p vs. bit length
- p vs. $\Delta(p)$

How to run

From the repo root:

```
make run EXP=e002
```

or:

```
uv run python -m mathxlab.experiments.e002
```

Notes / pitfalls

- Avoid converting huge integers to decimal strings repeatedly in tight loops; compute digits using logs where possible.
- Use `int.bit_length()` for stable bit length (fast and exact).
- Keep the exponent list modest for fast docs builds; this is a growth experiment, not a search for new Mersenne primes.

Extensions

- Compare growth to 2^{2^p} and quantify the relative gap.
- Add a “human scale” axis: compare $\text{digits}(N(p))$ to common benchmarks (atoms in the observable universe, etc.).
- Pull the exponent list from a small data file so the experiment can be updated without code changes.

References

See *References*.

[Cald.a, Voi98, OEISFInc25a]

3.3.3 E003: Abundancy Index Landscape

Tags: number-theory, quantitative-exploration, visualization, numerics (see *Valid Tags*)

Highlights

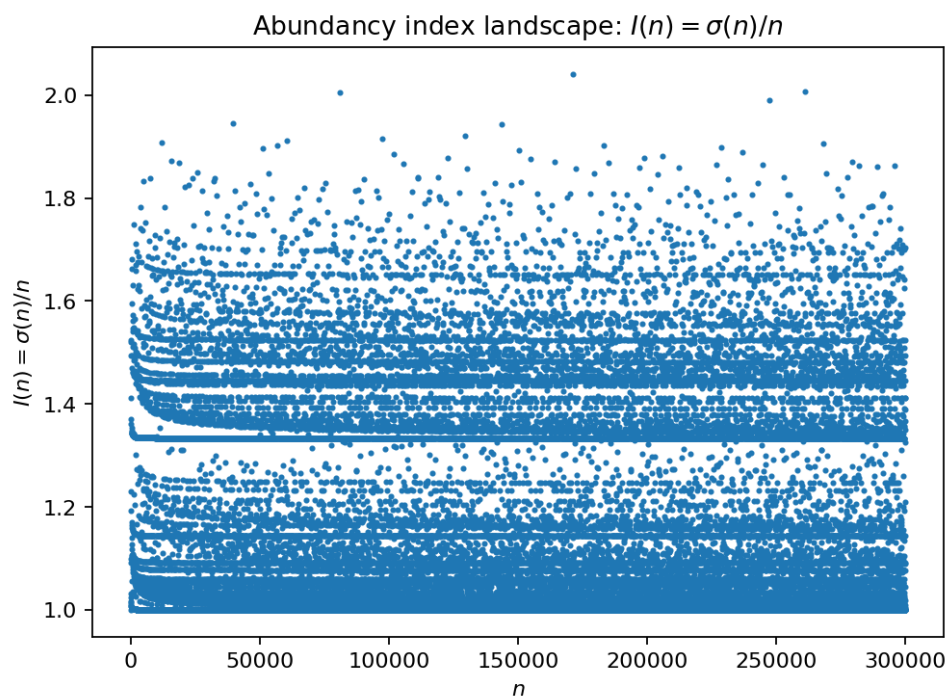
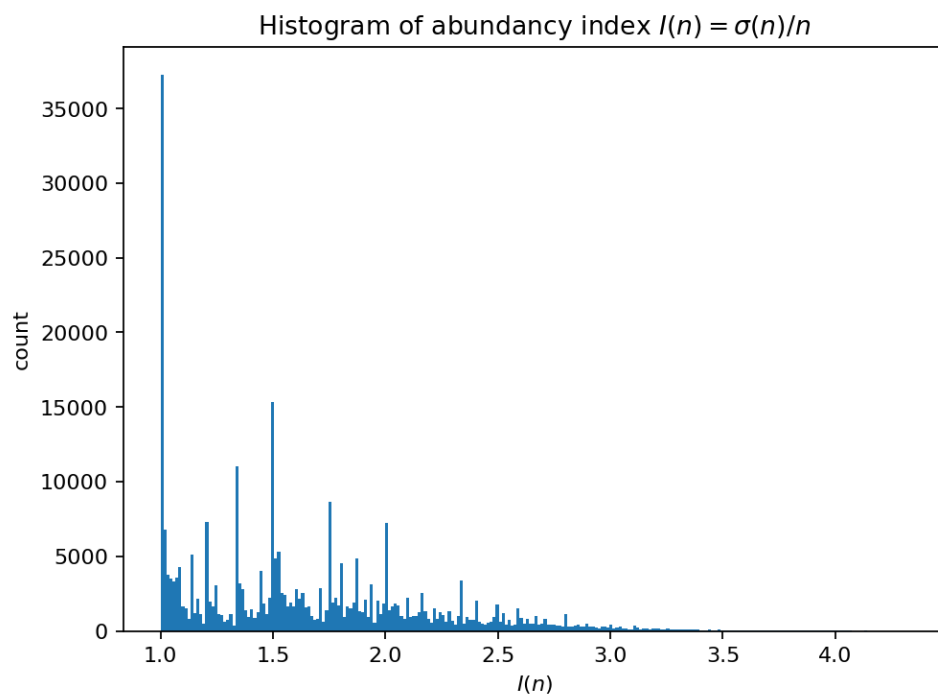
- Compute $\sigma(1..N)$ via a divisor-sum sieve.
- Visualize the distribution of $I(n) = \sigma(n)/n$.
- Highlight perfect numbers as the razor-thin level set $I(n) = 2$.

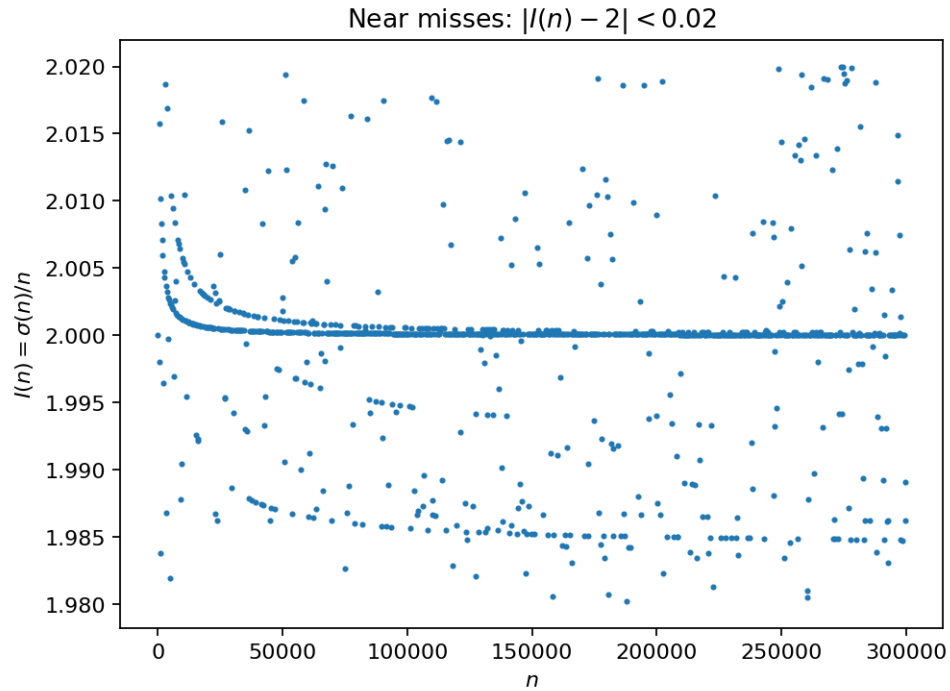
Goal

Visualize how **rare** perfect numbers are by plotting the distribution of the **abundancy index**

$$I(n) = \frac{\sigma(n)}{n}$$

for integers $n \leq N$. Perfect numbers satisfy $I(n) = 2$.





Research question

For a given bound N :

- What does the empirical distribution of $I(n)$ look like?
- How frequently do we see values near 2?
- Where do the perfect numbers appear in the landscape?

Why this qualifies as a mathematical experiment

The divisor-sum function $\sigma(n)$ is highly structured but “spiky” and hard to intuit symbolically. Computational sweeps reveal qualitative structure (clusters, gaps, tails) and put perfect numbers into context.

Experiment design

Method: compute (1), ..., (N) via a divisor-sum sieve

Use the identity “each divisor contributes to its multiples”:

- initialize an array `sigma[0..N]` with zeros
- for each $d = 1..N$:
 - add d to `sigma[k]` for all multiples $k = d, 2d, 3d, \dots$

This runs in about $O(N \log N)$ time and avoids per-number factorization.

Observables

- $I(n) = \sigma(n)/n$
- distance to perfection: $|I(n) - 2|$
- classification:
 - deficient: $I(n) < 2$
 - perfect: $I(n) = 2$

- abundant: $I(n) > 2$

Plots

- histogram of $I(n)$ (or of $I(n) - 1$)
- scatter plot of n vs. $I(n)$ (optionally with log-scale on n)
- highlight perfect numbers

How to run

```
make run EXP=e003
```

or:

```
uv run python -m mathxlab.experiments.e003
```

Notes / pitfalls

- $I(n)$ is rational. For classification, compare integers using $\sigma(n)$ and $2n$ rather than floats.
- For plots, floats are fine, but compute the “perfect” condition as `sigma[n] == 2*n`.
- Start with $N \leq 1\,000\,000$ to keep runtime and memory reasonable.

Extensions

- Repeat for different ranges and overlay histograms.
- Plot the top- k values of $I(n)$ and compare to known extremal families.
- Explore the “near misses” set (feeds directly into E006).

References

See *References*.

[Voi98, Wei03, OEISFInc25a]

3.3.4 E004: Computing (n) at Scale — Sieve vs. Factorization

Tags: number-theory, quantitative-exploration, numerics, optimization (see *Valid Tags*)

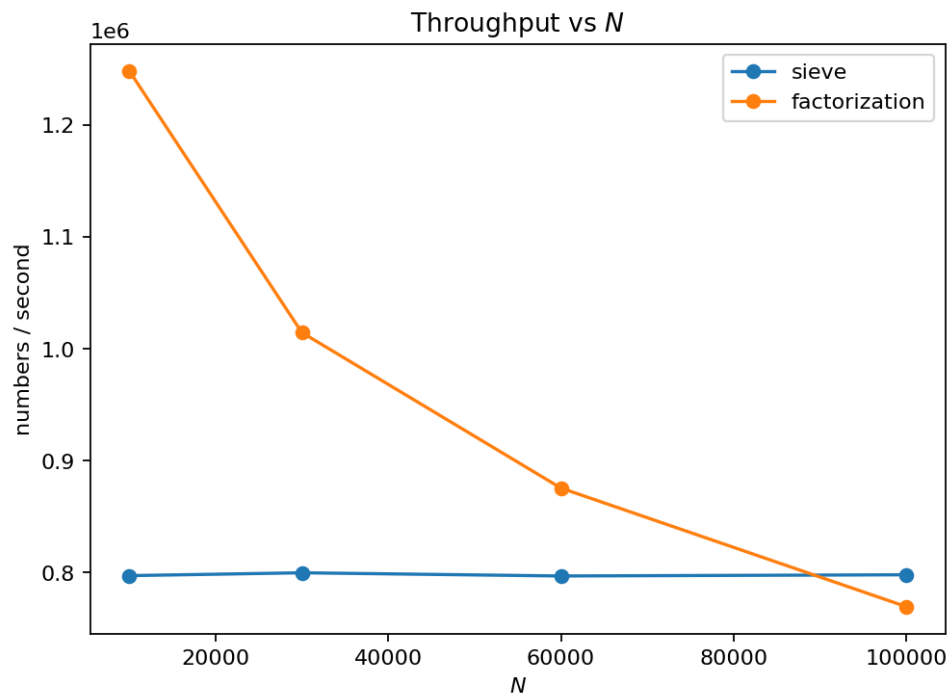
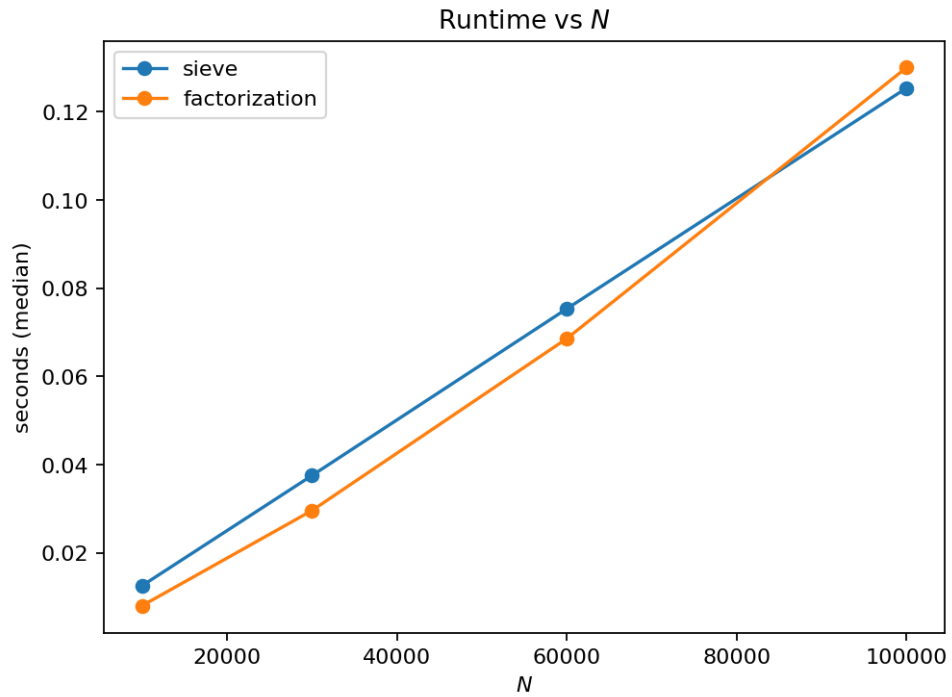
Highlights

- Benchmark bulk sieve vs. per-number factorization.
- Find runtime crossover points as N grows.
- Validate correctness on sampled inputs.

Goal

Benchmark two practical ways to compute the sum-of-divisors function:

1. **Sieve method:** compute all $\sigma(1..N)$ in one pass.
2. **Factorization method:** compute $\sigma(n)$ per-number from the prime factorization.



Research question

For a range of bounds N :

- which approach is faster?
- where is the crossover point?
- what are the memory tradeoffs?

Why this qualifies as a mathematical experiment

Both methods are mathematically equivalent, but performance depends on constants, caching, and implementation details. This is a quantitative exploration of algorithmic behavior grounded in number-theory structure.

Experiment design

Method A: divisor-sum sieve (bulk computation)

Compute `sigma[1..N]` by adding each divisor to its multiples (as in E003).

Method B: per-number factorization

Factor each n (e.g. using a precomputed prime list up to $\sqrt{N}$) and compute:

If

$$n = \prod_{i=1}^k p_i^{a_i},$$

then

$$\sigma(n) = \prod_{i=1}^k \frac{p_i^{a_i+1} - 1}{p_i - 1}.$$

Measurements

- wall-clock runtime vs. N (multiple trials, median)
- peak memory (rough estimate acceptable for v1)
- correctness cross-check: random sample where both methods agree

Outputs

- plot: runtime vs. N for both methods
- short table: N , runtime(A), runtime(B), speedup

How to run

```
make run EXP=e004
```

or:

```
uv run python -m mathxlab.experiments.e004
```

Notes / pitfalls

- Factorization becomes expensive quickly; keep the factorization method limited to moderate N .
- Use integer arithmetic for correctness checks (`sigma[n] == 2*n` etc.).
- Report both runtime and *effective throughput* (numbers processed per second).

Extensions

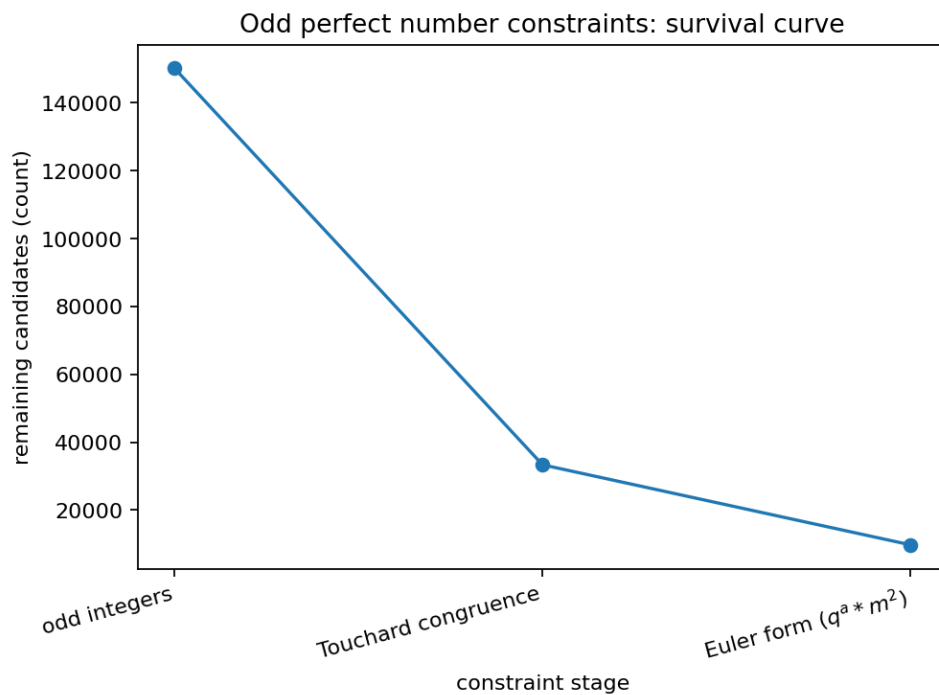
- Add a third method using smallest-prime-factor (SPF) sieve for fast factorization.
- Compare pure Python vs. `numpy` arrays for Method A.
- Turn the benchmark into a reusable utility for later experiments.

References

See *References*.

[Voi98, Wei03]

3.3.5 E005: Odd Perfect Numbers — Constraint Filter Pipeline



Tags: number-theory, counterexample-search, visualization, search (see *Valid Tags*)

Highlights

- Apply necessary constraints as staged filters on odd candidates.
- Plot survival curves (remaining candidates per constraint stage).
- Make the “open problem” constraints tangible at finite scales.

Goal

Demonstrate *why brute-force search for odd perfect numbers is unrealistic* by applying known **necessary conditions** as a step-by-step filter pipeline and visualizing how the candidate pool collapses.

Research question

Given odd integers up to a bound N :

- how many survive each constraint stage?
- which constraints are most “eliminating” in practice at small scales?
- what does a survival curve suggest about scalability?

Why this qualifies as a mathematical experiment

Odd perfect numbers are an open problem. Theorems provide constraints rather than a classification. Computation makes these constraints tangible by measuring their elimination power on finite candidate sets.

Experiment design

Candidate set

Start with odd integers $1 < n \leq N$.

Constraint stages (v1)

Use a conservative, well-known set of *necessary* conditions that can be checked mechanically:

1. **Euler form (shape constraint):** any odd perfect number must be of the form

$$n = q^\alpha m^2$$

where q is prime, $\gcd(q, m) = 1$, and

$$q \equiv \alpha \equiv 1 \pmod{4}.$$

2. **Congruence filter (Touchard-type):** odd perfect numbers satisfy strong congruence restrictions (implemented as one or two simple congruence checks supported by the references).
3. **Small-prime structure filters:** apply a few lightweight necessary conditions (e.g., reject candidates divisible by some specific small patterns if justified by the reference set).

For each stage, record “remaining candidates”.

Output

- table: stage name, remaining count, elimination percentage
- plot: survival curve (stage index vs. remaining candidates)

How to run

```
make run EXP=e005
```

or:

```
uv run python -m mathxlab.experiments.e005
```

Notes / pitfalls

- Be explicit about what is *proved* vs. what is a heuristic. Only implement conditions that are truly necessary.
- At small N , some deep constraints won’t show their full strength; the goal is the *pipeline idea*, not a record bound.
- Euler-form testing requires factorization; keep N small enough for trial division (v1).

Extensions

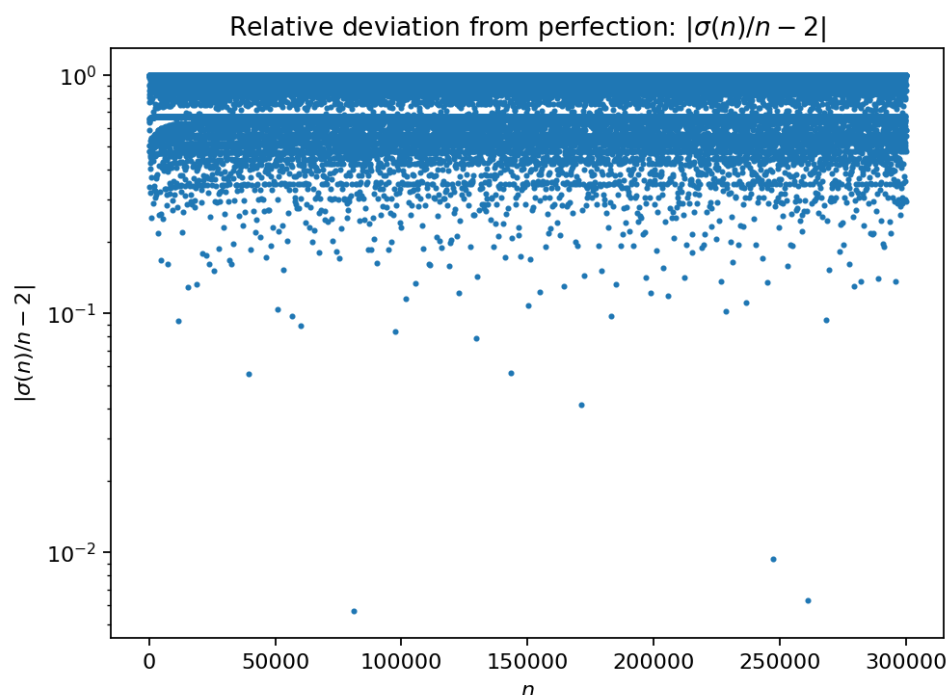
- Add stronger bounds/constraints from the modern literature and compare elimination power.
- Replace trial division with an SPF sieve to scale the Euler-form test.
- Report not just counts but also the distribution of remaining prime-factor patterns.

References

See *References*.

[Guy04, OR14, Sto24, Voi98]

3.3.6 E006: Near Misses to Perfection



Tags: number-theory, conjecture-generation, visualization, numerics (see *Valid Tags*)

Highlights

- Search for n with $\sigma(n)$ unusually close to $2n$.
- Build leaderboards for absolute and relative deviation.
- Visualize how “near perfection” clusters by structure.

Goal

Find and visualize integers whose divisor sum is **unusually close** to the perfect condition $\sigma(n) = 2n$, without being perfect.

Research question

For integers $n \leq N$, which numbers minimize:

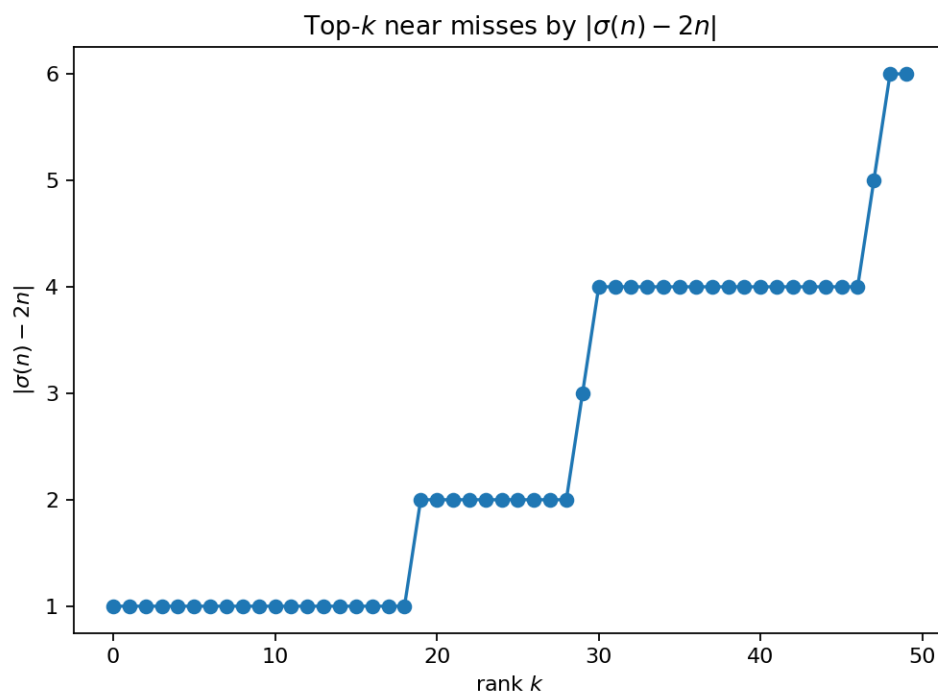
- absolute deviation:

$$D_1(n) = |\sigma(n) - 2n|$$

- relative deviation:

$$D_2(n) = \left| \frac{\sigma(n)}{n} - 2 \right|?$$

Do “near misses” cluster in recognizable families (highly composite, abundant, etc.)?



Why this qualifies as a mathematical experiment

The perfect condition is a sharp equality. Studying the closest failures often reveals structure and suggests new questions (e.g., which multiplicative patterns drive $\sigma(n)$ toward $2n$).

Experiment design

Computation

- Compute $\sigma(1..N)$ via the divisor-sum sieve (as in E003).
- For each n , compute $D_1(n)$ and $D_2(n)$.
- Keep the top- k smallest deviations (excluding actual perfect numbers).

Outputs

- table: top- k near misses (with n , $\sigma(n)$, D_1 , D_2)
- plot: n vs. $D_2(n)$ (log-scale on D_2 often helps)
- mark perfect numbers for reference

How to run

```
make run EXP=e006
```

or:

```
uv run python -m mathxlab.experiments.e006
```

Notes / pitfalls

- Use integer comparisons to identify perfect numbers (`sigma[n] == 2*n`).
- For D_2 , floats are fine for plotting, but store exact rational values for ranking when possible (e.g., compare $|\sigma(n) - 2n|$ first, then normalize for reporting).

- Choose k small (e.g. 50 or 200) so the report stays readable.

Extensions

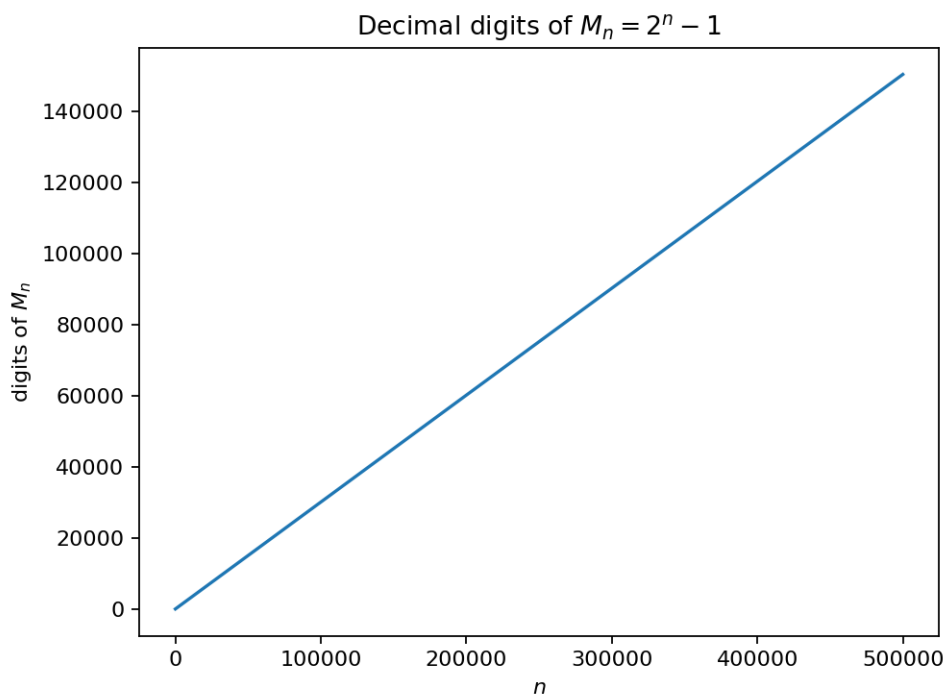
- Repeat for different N and compare stability of the “near miss” leaderboard.
- Add a second leaderboard restricted to odd n only.
- Compare near misses to known abundant/deficient classifications and prime factorizations.

References

See [References](#).

[Voi98, Wei03, OEISFInc25a]

3.3.7 E007: Mersenne growth (bits and digits)



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

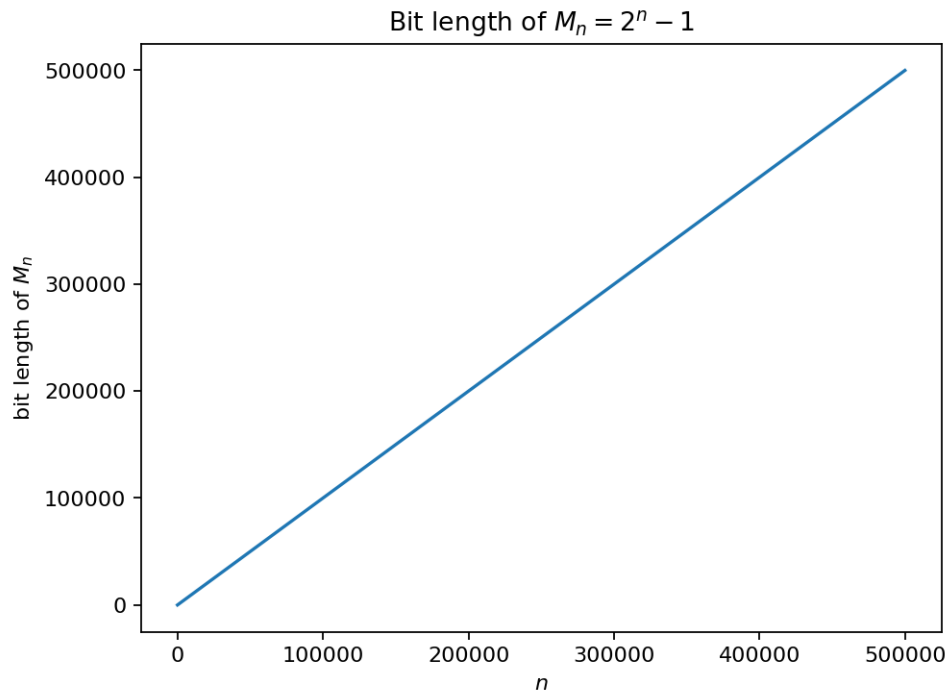
Highlights

- Visualize how quickly $M_n = 2^n - 1$ grows in **bits** and **decimal digits**.
- Compute size metrics *without* constructing huge integers.
- Provide simple rules-of-thumb for choosing feasible bounds in later experiments.

Goal

Quantify and visualize the growth of Mersenne numbers $M_n = 2^n - 1$ as n increases.

This experiment focuses on **size** (bit-length and decimal digits), because it determines what later algorithms (Lucas–Lehmer, sieving) can realistically handle on a laptop.



Background (quick refresher)

- *Mersenne numbers and primes refresher*

Research question

How fast does the size of M_n grow with n , and what simple formulas approximate:

- the number of bits of M_n
- the number of decimal digits of M_n

Why this qualifies as a mathematical experiment

- **Finite procedure:** evaluate size formulas on a range $n = 1..N$.
- **Observable(s):** bit-length and digit count as functions of n .
- **Parameter space:** vary N (and optional sampling density).
- **Outcome:** visual evidence (curves) and tables usable as planning guidance.
- **Reproducibility:** parameters written to `params.json`; figures written to `figures/`.

Experiment design

Computation

Key facts:

- $\text{bits}(2^n - 1) = n$ for $n \geq 1$.
- $\text{digits}(2^n - 1) = \lfloor n \log_{10}(2) \rfloor + 1$ for $n \geq 1$.

Compute both for a grid of n values.

Outputs

- plot: n vs. bits
- plot: n vs. digits
- table: selected n with digits (e.g., 10, 100, 1k, 10k, ...)

How to run

```
make run EXP=e007
```

or:

```
uv run python -m mathxlab.experiments.e007
```

Notes / pitfalls

- Avoid converting huge M_n to decimal just to count digits. The logarithm formula is exact for the digit count.
- For very large n , use stable floating-point evaluation for $n \log_{10}(2)$ (double precision is fine up to very large n).
- This experiment is intentionally lightweight; it should run quickly.

Extensions

- Overlay “feasibility bands” (e.g., digits thresholds) for what your other experiments can handle.
- Compare M_n to other fast-growing families (factorials, primorials) on a log scale.

References

See *References*.

[con25b]

3.3.8 E008: Lucas–Lehmer scan (prime exponents)

Tags: number-theory, quantitative-exploration, visualization (see *Valid Tags*)

Highlights

- Run the Lucas–Lehmer test for many prime exponents p .
- Record timing and residue behavior for composite vs. prime outcomes.
- Produce a reproducible “scan report” (which p were tested, which passed).

Goal

Empirically explore Mersenne primality testing by scanning prime exponents p and applying the Lucas–Lehmer test to $M_p = 2^p - 1$.

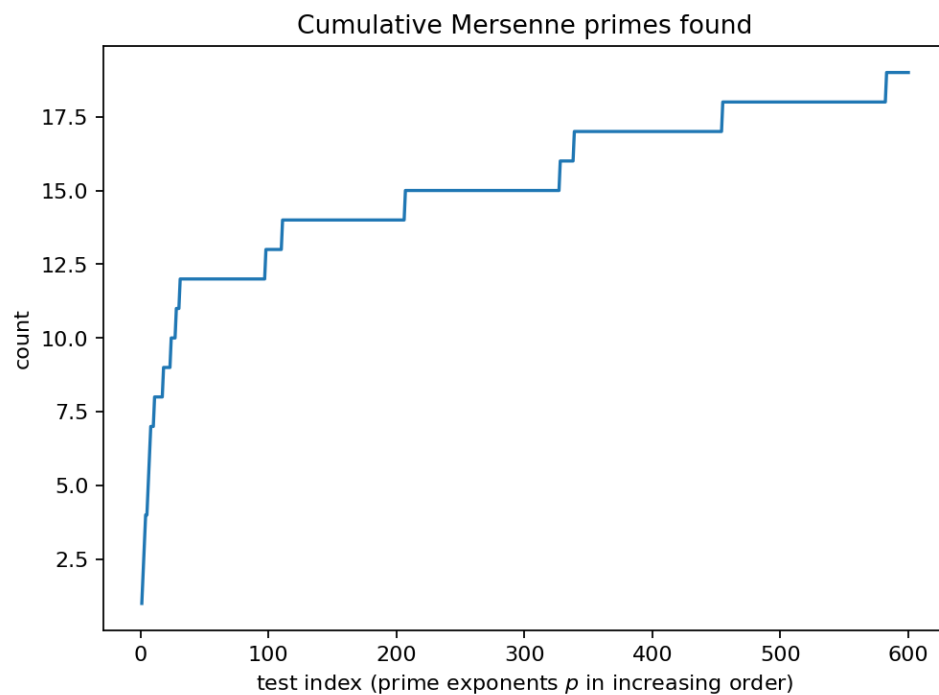
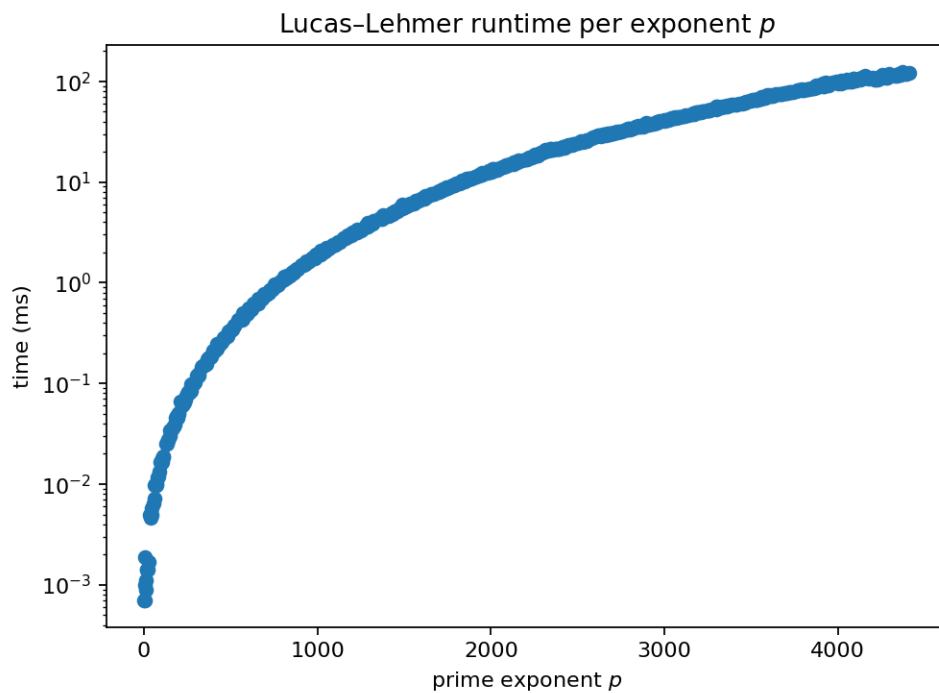
Background (quick refresher)

- *Mersenne numbers and primes refresher*

Research question

For prime exponents $p \leq P$, how does:

- runtime of the Lucas–Lehmer test scale with p ?
- the distribution of final residues (mod M_p) differ between prime and composite outcomes?



Why this qualifies as a mathematical experiment

- **Finite procedure:** enumerate prime exponents up to a bound and test them.
- **Observable(s):** pass/fail, final residue, and runtime.
- **Parameter space:** vary P and optional “max tests” cap.
- **Outcome:** a dataset (tested p values) and plots/tables that suggest scaling behavior.
- **Failure modes:** naive implementations can be slow; parameters bound the search to keep runs feasible.

Experiment design

Computation

- Enumerate prime exponents p up to a bound P .
- For each p , compute $M_p = 2^p - 1$ and run Lucas–Lehmer:
 - $s_0 = 4$
 - $s_{k+1} = s_k^2 - 2 \pmod{M_p}$
 - M_p is prime iff $s_{p-2} \equiv 0 \pmod{M_p}$.
- Record (at minimum): p , pass/fail, final residue, and wall time.

Outputs

- table: scanned exponents with result and time
- plot: p vs runtime (often log-scale is helpful)
- plot: residue magnitude / patterns (optional)

How to run

```
make run EXP=e008
```

or:

```
uv run python -m mathxlab.experiments.e008
```

Notes / pitfalls

- Only run Lucas–Lehmer for **prime** p ; if p is composite then M_p is composite.
- The test is defined for odd prime p ; treat $p = 2$ separately ($M_2 = 3$ is prime).
- Keep bounds modest at first; the cost grows quickly with p .

Extensions

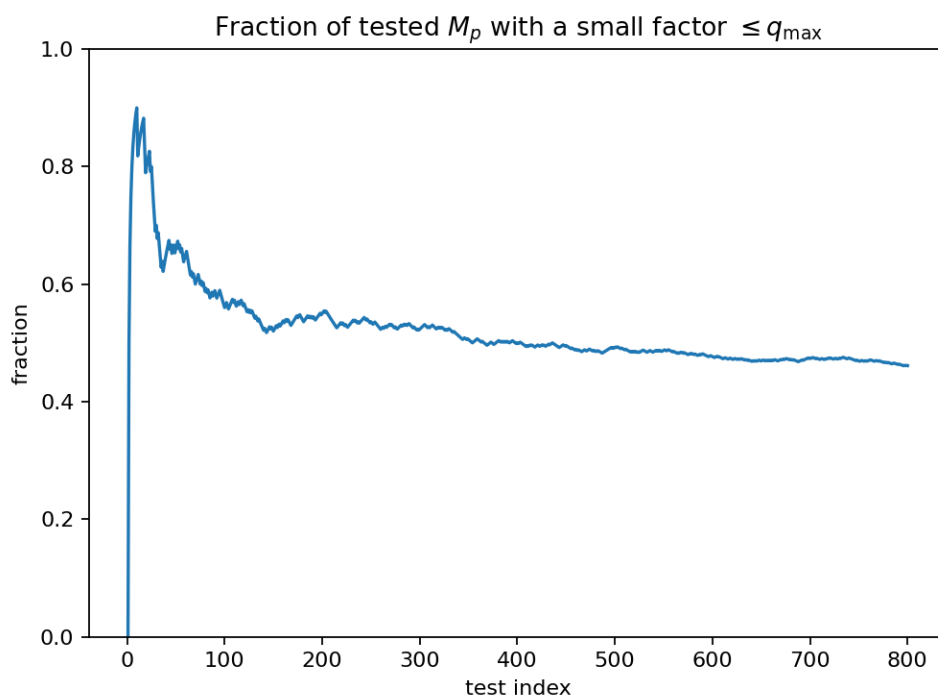
- Add a “pre-sieve” that checks for small factors before running Lucas–Lehmer.
- Compare your timings to published implementations (as a sanity check, not a competition).

References

See *References*.

[Cal21, con25a, MR24]

3.3.9 E009: Small-factor scan for Mersenne numbers



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- Quickly eliminate many M_p candidates by finding small prime factors.
- Use the structure of possible factors of M_p to reduce wasted work.
- Show how often “trivial” factors appear in a range of exponents.

Goal

Explore how often Mersenne numbers $M_p = 2^p - 1$ (with prime p) have **small factors**, and how a lightweight sieve can filter candidates before running Lucas–Lehmer.

Background (quick refresher)

- *Mersenne numbers and primes refresher*

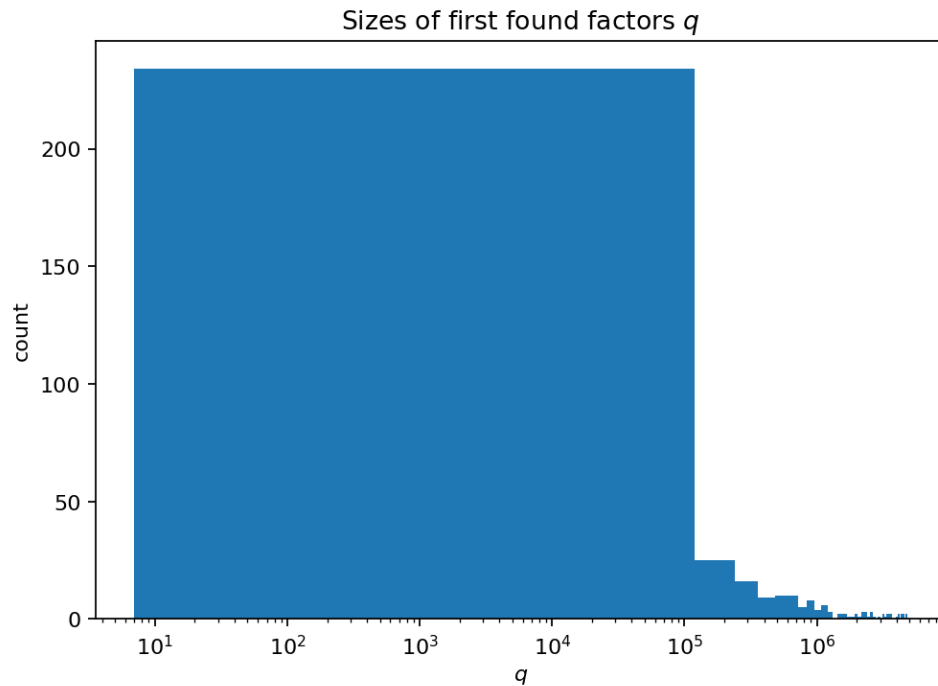
Research question

For prime exponents $p \leq P$ and a factor bound $q \leq Q$:

- how many M_p have a factor below Q ?
- how much compute can be saved by sieving before Lucas–Lehmer?

Why this qualifies as a mathematical experiment

- **Finite procedure:** scan a finite range of p and candidate factors.
- **Observable(s):** first small factor found (if any), counts by p and by factor size.
- **Parameter space:** vary P and Q .
- **Outcome:** tables/plots that show factor frequency and sieve effectiveness.
- **Failure modes:** incomplete sieve bounds can mislead; clearly report Q as a limitation.



Experiment design

Computation

For fixed prime exponent p , any prime factor q of M_p satisfies:

- $q \equiv 1 \pmod{2p}$ (a common necessary condition used to generate candidates)

A practical sieve approach:

- Generate candidate primes q of the form $q = 2pk + 1$ up to Q .
- For each candidate prime q , check:
 - $2^p \equiv 1 \pmod{q}$

If true, then q divides M_p .

Outputs

- table: per p , smallest factor found (or “none up to Q ”)
- plot: p vs. smallest factor (when present)
- summary: fraction of p eliminated by the sieve

How to run

```
make run EXP=e009
```

or:

```
uv run python -m mathxlab.experiments.e009
```

Notes / pitfalls

- This is a **bounded** sieve: “no factor found” only means “none found up to Q ”.
- Prime testing for q should be fast; for small Q , a simple deterministic method is fine.
- Report both P and Q prominently in the report.

Extensions

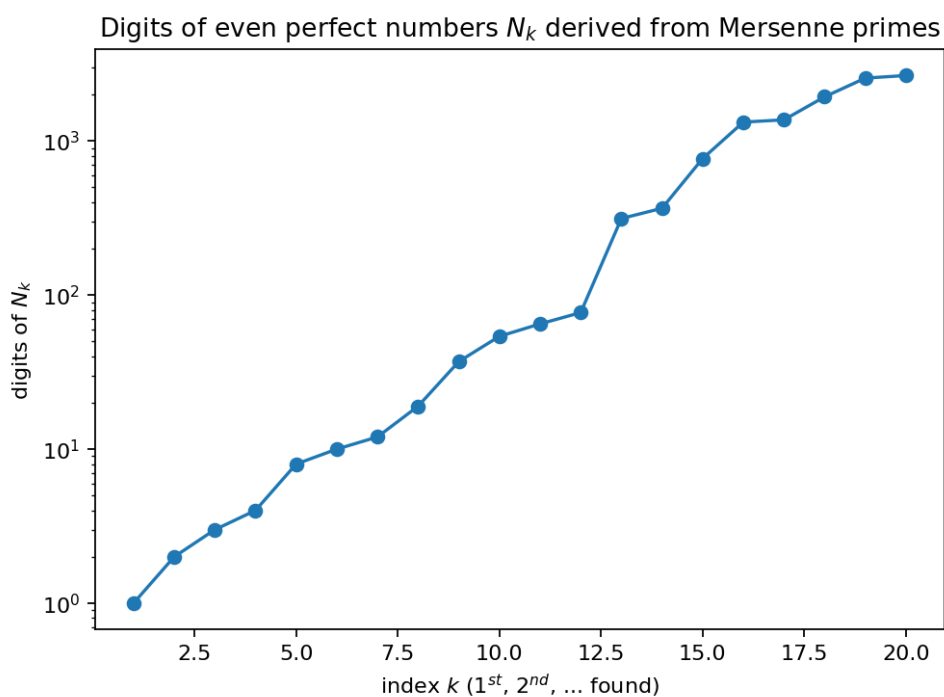
- Combine with E008: run Lucas–Lehmer only on exponents not eliminated by the sieve.
- Track how much wall time is saved by the combined pipeline.

References

See [References](#).

[Cal21, con25b, Inc25b]

3.3.10 E010: Even perfect numbers from Mersenne primes



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- Generate even perfect numbers via the Euclid–Euler theorem.
- Connect Mersenne prime exponents p to perfect numbers $N = 2^{p-1}(2^p - 1)$.
- Visualize growth and verify the defining property $\sigma(N) = 2N$ for sampled cases.

Goal

Make the Mersenne perfect-number connection computationally explicit:

If $M_p = 2^p - 1$ is prime, then:

$$N_p = 2^{p-1}(2^p - 1)$$

is an **even perfect number**.

Background (quick refresher)

- *Mersenne numbers and primes refresher*
- *Perfect numbers refresher*

Research question

Across the Mersenne prime exponents discovered within your scan bounds:

- how fast do the corresponding even perfect numbers grow?
- can we verify perfectness ($\sigma(N) = 2N$) efficiently for these cases?

Why this qualifies as a mathematical experiment

- **Finite procedure:** find a set of Mersenne primes within a finite bound and generate perfect numbers.
- **Observable(s):** size metrics (digits), and validation checks of $\sigma(N) = 2N$.
- **Parameter space:** vary the exponent bound and validation depth.
- **Outcome:** concrete examples + growth plots that support intuition.
- **Reproducibility:** exponents tested and successes recorded in artifacts.

Experiment design

Computation

- Obtain a list of Mersenne prime exponents p from a scan (or a fixed list for small p).
- For each p , compute $N_p = 2^{p-1}(2^p - 1)$.
- Verify perfectness for these cases:

$$\sigma(N_p) = 2N_p.$$

For modest p , exact computation is feasible; for larger p , report size metrics and skip expensive checks.

Outputs

- table: p , M_p size, N_p size, and validation status
- plot: p vs digits of N_p
- optional: prime-factor structure display for small cases

How to run

```
make run EXP=e010
```

or:

```
uv run python -m mathxlab.experiments.e010
```

Notes / pitfalls

- Don't attempt divisor-sum sieves for huge N_p ; validation must be bounded and explicit.
- Clearly separate “constructed from theorem” (conditional on M_p being prime) from “validated by computation”.

Extensions

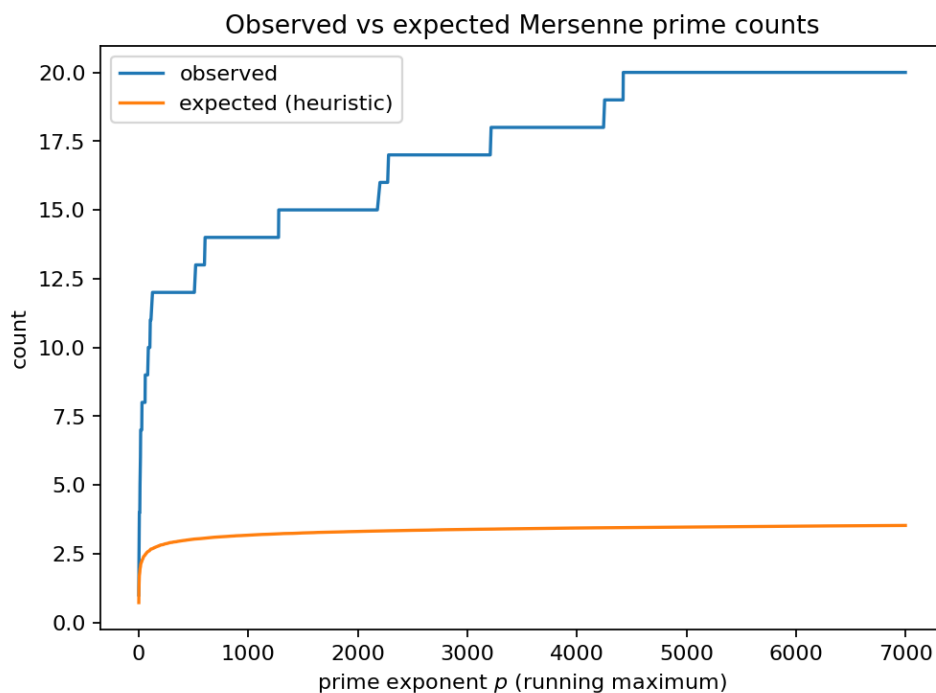
- Cross-link to E002 outputs for perfect numbers and compare growth on the same axes.
- Explore which parts of the perfectness check can be done symbolically using known factorization.

References

See *References*.

[Cald.a, con25b, Inc25a]

3.3.11 E011: Heuristic rarity of Mersenne primes



Tags: number-theory, quantitative-exploration, visualization (see *Valid Tags*)

Highlights

- Compare observed Mersenne-prime counts to a simple heuristic expectation curve.
- Visualize cumulative “found vs expected” as exponent bounds increase.
- Keep the discussion explicitly empirical (evidence, not proof).

Goal

Mersenne primes are extremely rare. This experiment compares:

- the observed count of Mersenne primes among prime exponents $p \leq P$
- a lightweight heuristic curve that grows slowly with P

The aim is intuition-building and trend visualization, not a rigorous model.

Background (quick refresher)

- *Mersenne numbers and primes refresher*

Research question

For increasing prime exponent bounds P :

- how does the cumulative count of “passes” from your scan grow?
- how does it compare to a simple expectation curve of the form:

$$E(P) \approx \sum_{p \leq P, p \text{ prime}} \frac{1}{p \ln 2}$$

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a finite scan and compute a finite expected sum.
- **Observable(s):** cumulative count of passes vs. cumulative expected value.
- **Parameter space:** vary P (and optionally cap runtime per test).
- **Outcome:** plots that show agreement/divergence and suggest questions for deeper study.
- **Failure modes:** small ranges are noisy; the experiment should clearly label bounds.

Experiment design

Computation

- From E008 (or a fixed list), get the set of exponents tested and which passed.
- For each bound P , compute:
 - $\text{Found}(P)$ = number of passing exponents $p \leq P$
 - $E(P)$ = cumulative heuristic sum
- Plot both curves against P .

Outputs

- plot: P vs $\text{Found}(P)$ and $E(P)$
- table: selected checkpoints (P , Found , E)

How to run

```
make run EXP=e011
```

or:

```
uv run python -m mathxlab.experiments.e011
```

Notes / pitfalls

- The heuristic curve is a **comparison tool**, not a theorem.
- Results depend heavily on which exponents were actually tested; record the tested set in the report.
- A single large Mersenne prime can dominate impressions—keep axes and annotations clear.

Extensions

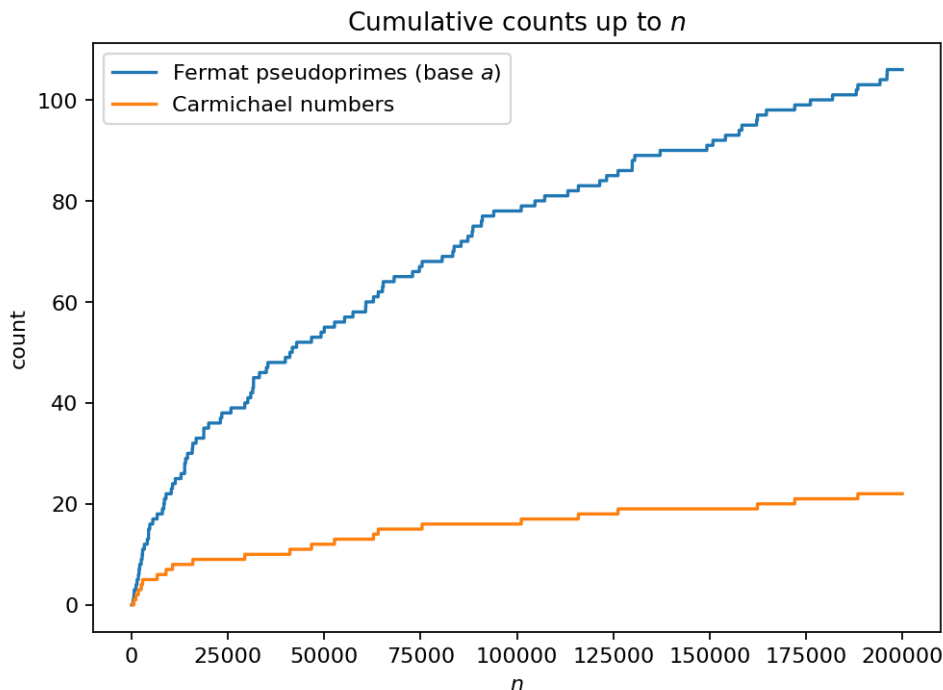
- Compare multiple heuristics (still empirical) and see which best matches the observed range.
- Extend the scan bound and see whether the deviation grows or stabilizes.

References

See *References*.

[Cal21, MR25]

3.3.12 E012: Fermat pseudoprimes and Carmichael numbers (counterexamples)



Tags: number-theory, counterexample-search, visualization (see *Valid Tags*)

Highlights

- Counterexamples to naive Fermat primality testing: base-a pseudoprimes and Carmichael numbers.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

Counterexamples to naive Fermat primality testing: base-a pseudoprimes and Carmichael numbers.

Background (quick refresher)

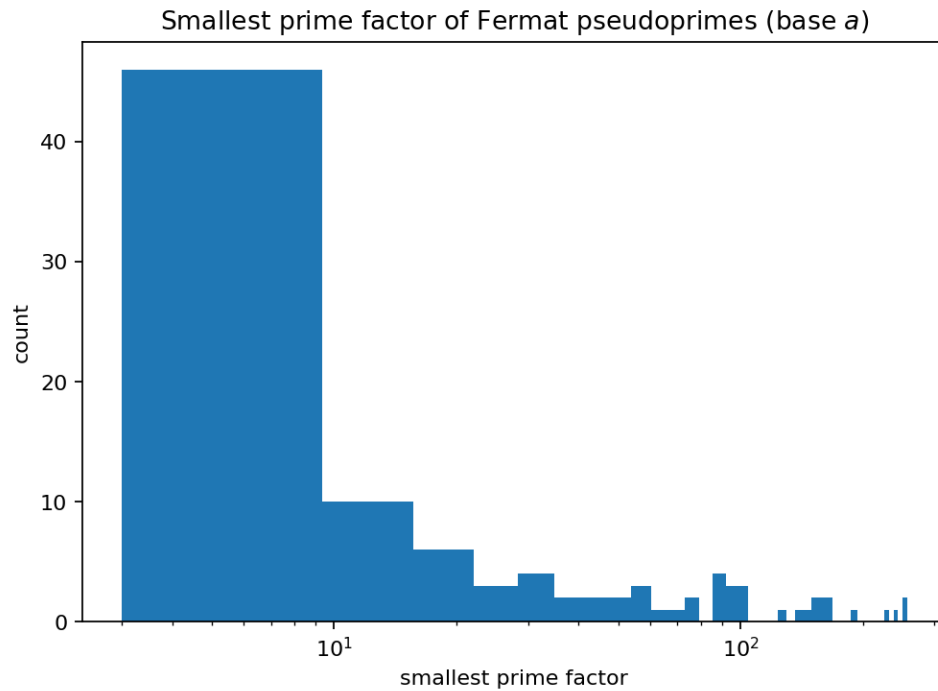
- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).



- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e012/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_01_cumulative_counts.png`
 - `figures/fig_02_smallest_factor_hist.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e012
```

or:

```
uv run python -m mathxlab.experiments.e012
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

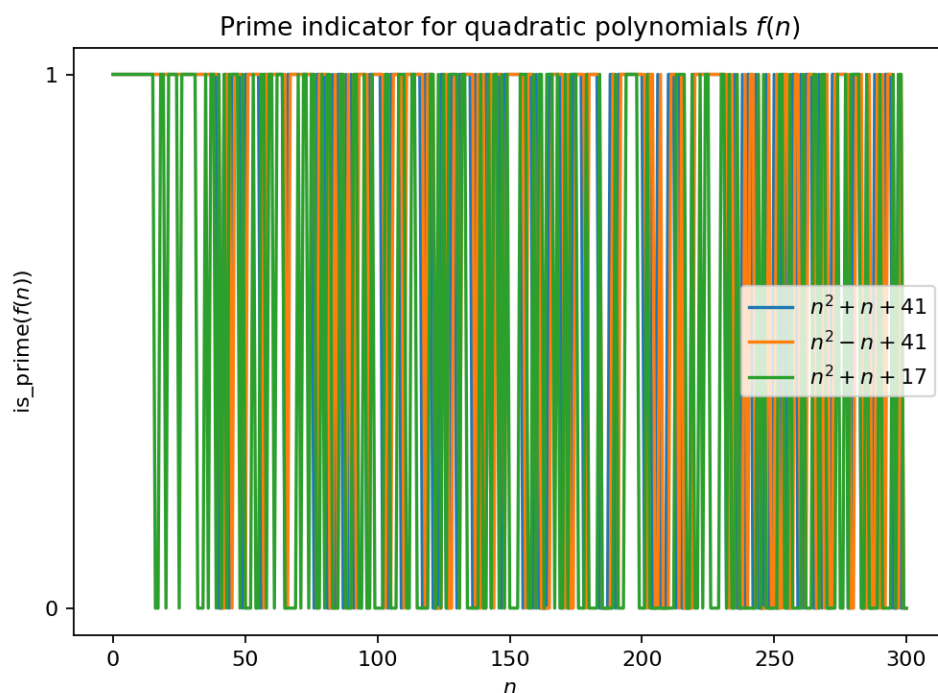
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.13 E013: Prime-polynomial counterexamples (Euler's $n^2 + n + 41$)



Tags: number-theory, counterexample-search, visualization (see [Valid Tags](#))

Highlights

- Turn “prime-generating polynomials” folklore into a crisp counterexample (Euler’s n^2+n+41).
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

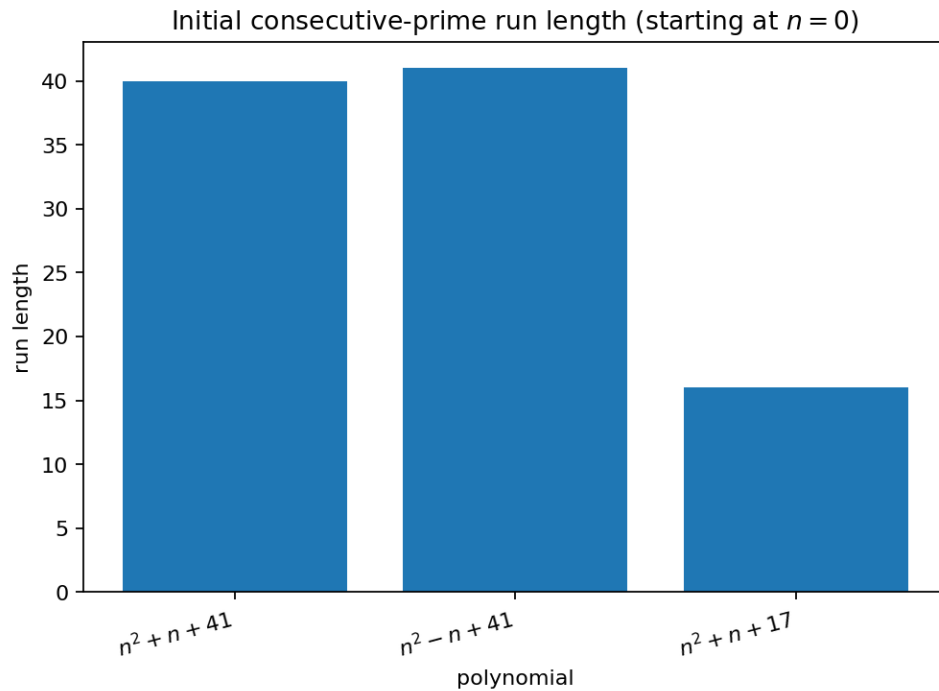
Turn “prime-generating polynomials” folklore into a crisp counterexample (Euler’s n^2+n+41).

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?



Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e013/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_01_prime_indicator.png`
 - `figures/fig_02_initial_prime_run_lengths.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e013
```

or:

```
uv run python -m mathxlab.experiments.e013
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

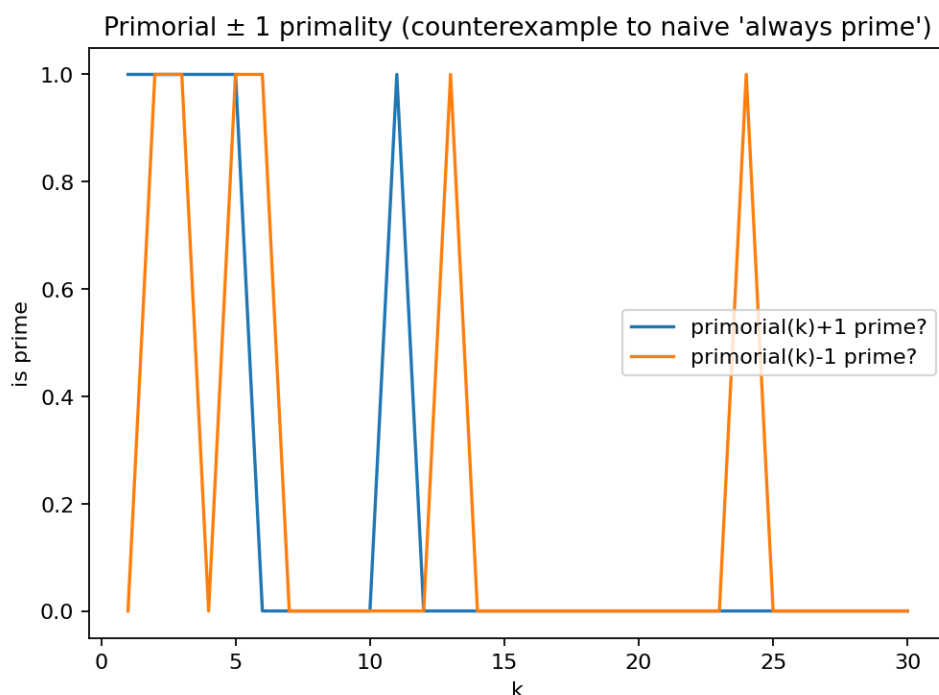
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.14 E014: Primorial ± 1 counterexamples



Tags: number-theory, counterexample-search, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e014/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e014
```

or:

```
uv run python -m mathxlab.experiments.e014
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

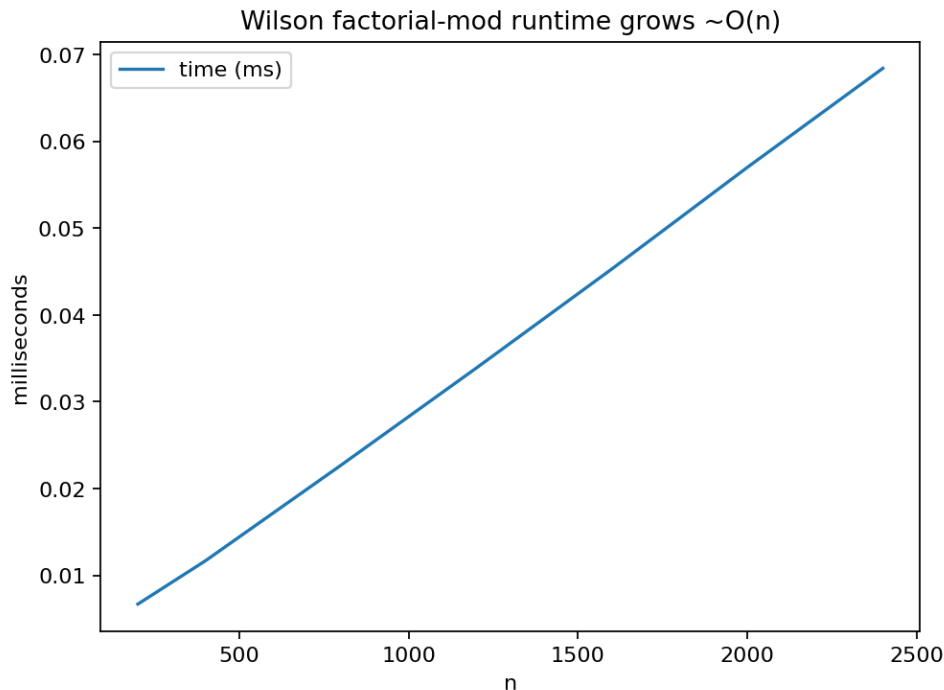
- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.15 E015: Wilson test infeasibility

Tags: number-theory, quantitative-exploration, visualization (see *Valid Tags*)



Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e015/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e015
```

or:

```
uv run python -m mathxlab.experiments.e015
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.16 E016: Trial division vs Miller–Rabin scaling

Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

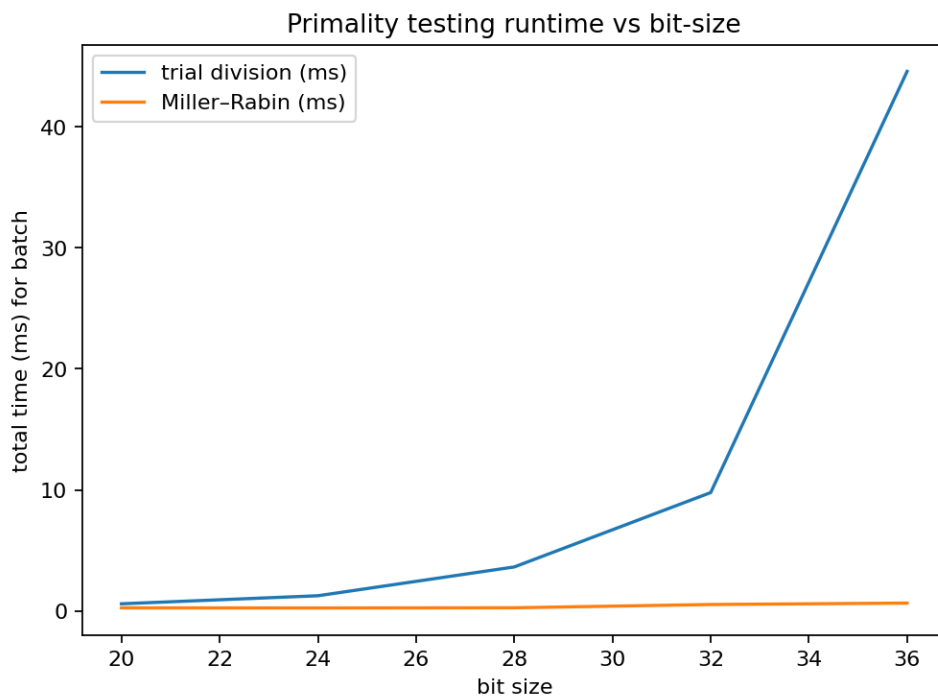
- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*



Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e016/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e016
```

or:

```
uv run python -m mathxlab.experiments.e016
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

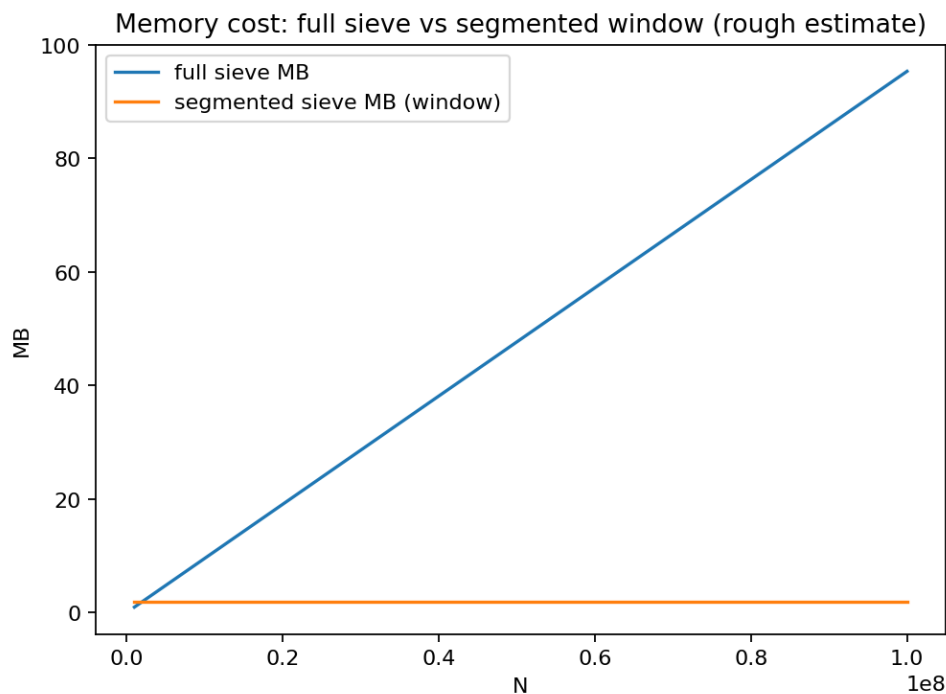
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.17 E017: Sieve memory blow-up vs segmented sieve



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e017/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e017
```

or:

```
uv run python -m mathxlab.experiments.e017
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

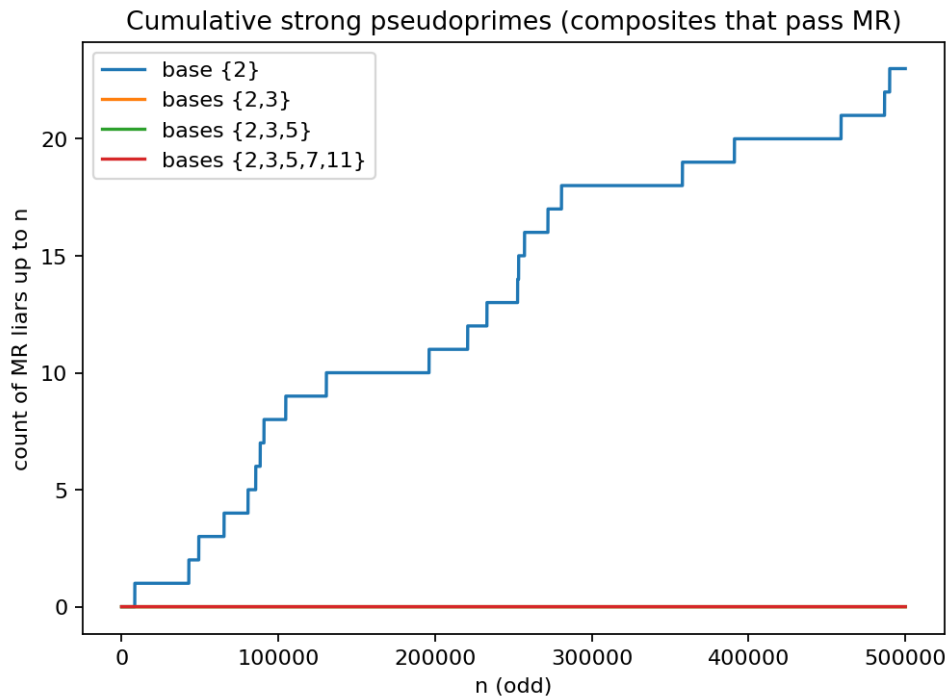
- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.18 E018: Miller–Rabin base choice counterexamples

Tags: number-theory, counterexample-search, visualization (see *Valid Tags*)



Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e018/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e018
```

or:

```
uv run python -m mathxlab.experiments.e018
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.19 E019: Prime density and PNT visualization

Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

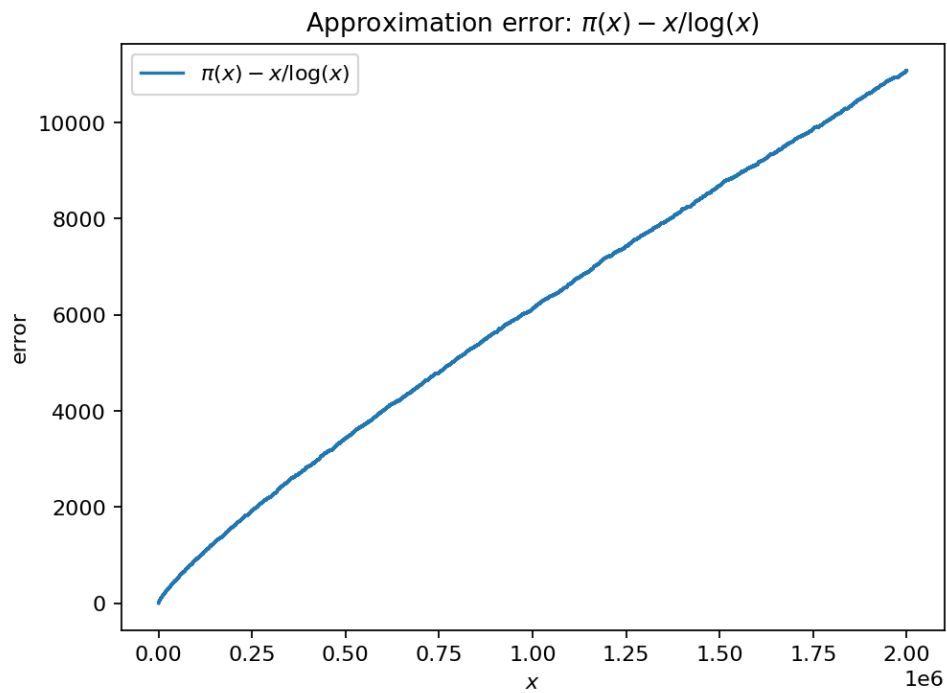
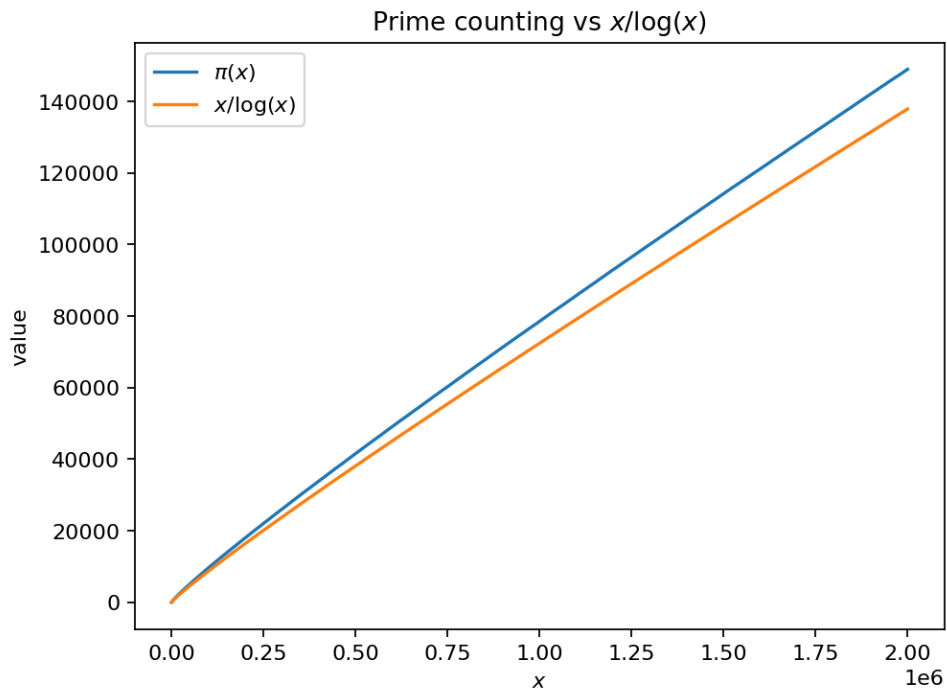
- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- [Prime numbers refresher](#)



Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e019/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e019
```

or:

```
uv run python -m mathxlab.experiments.e019
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

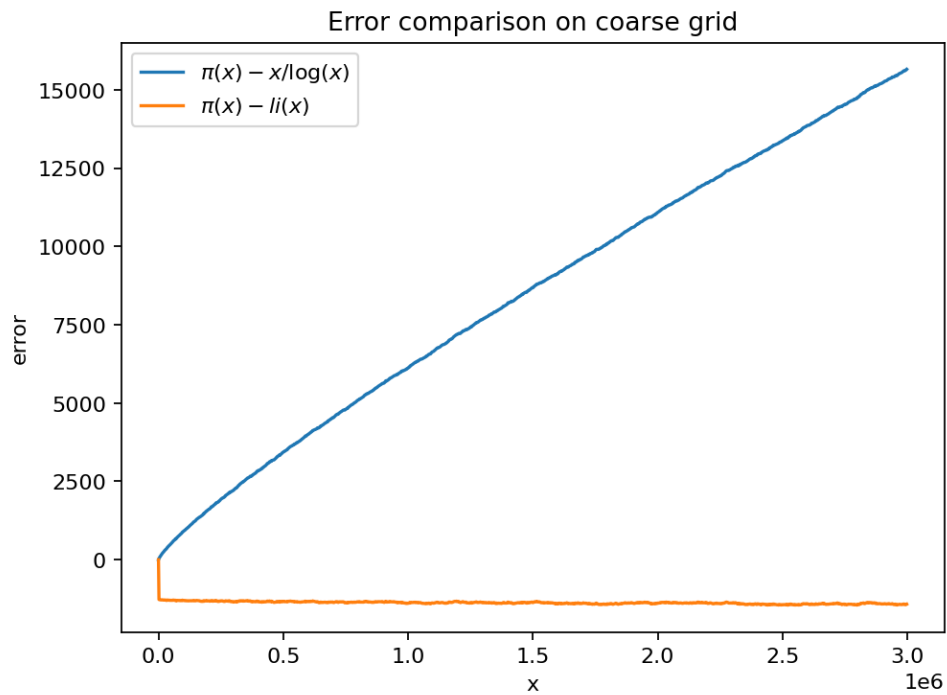
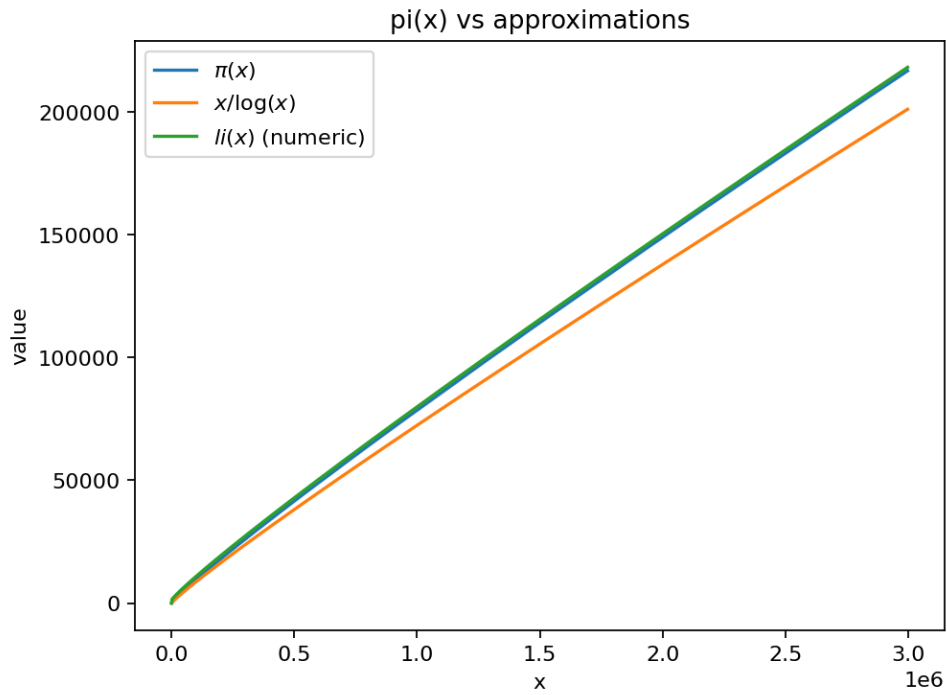
- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.20 E020: Compare $\pi(x)$ to $\text{li}(x)$ numerically

Tags: number-theory, quantitative-exploration, visualization (see *Valid Tags*)



Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e020/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e020
```

or:

```
uv run python -m mathxlab.experiments.e020
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

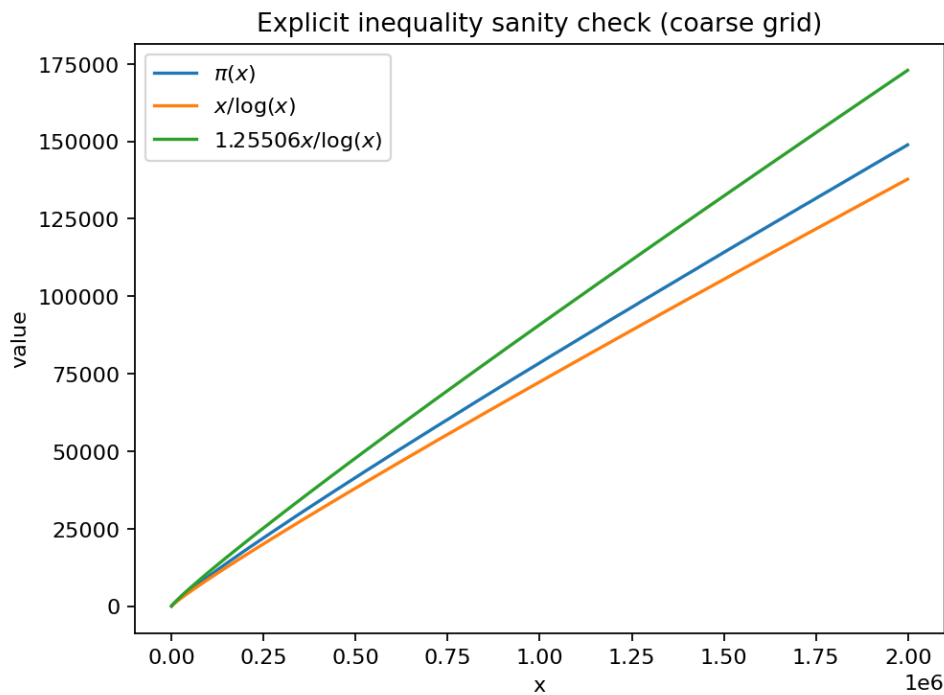
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.21 E021: Explicit bounds sanity checks



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e021/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e021
```

or:

```
uv run python -m mathxlab.experiments.e021
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

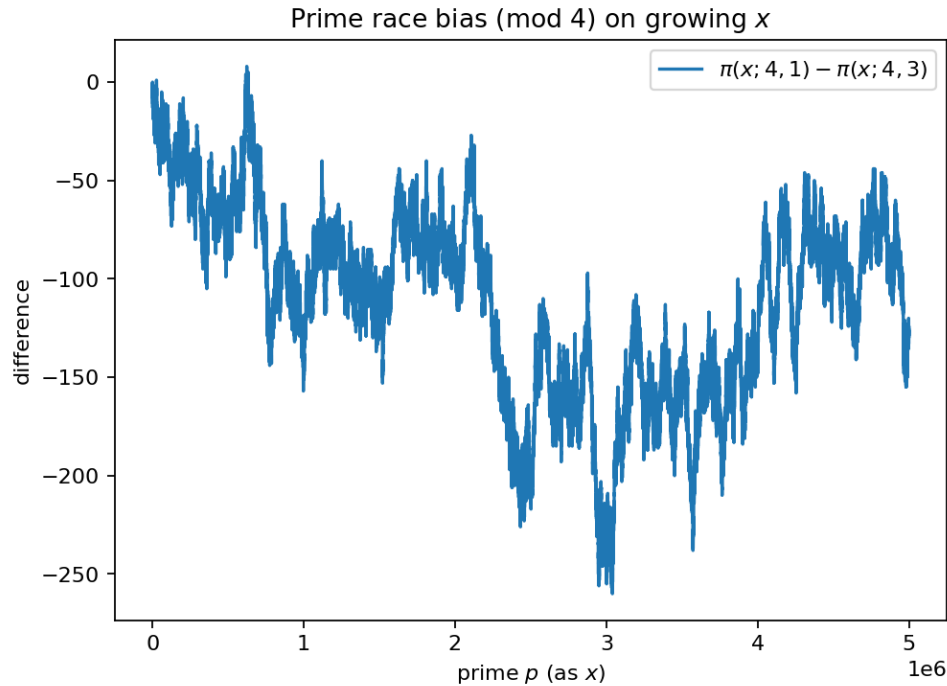
- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.22 E022: Prime race modulo 4

Tags: number-theory, quantitative-exploration, visualization (see *Valid Tags*)



Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e022/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e022
```

or:

```
uv run python -m mathxlab.experiments.e022
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.23 E023: Residue class distribution mod q

Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

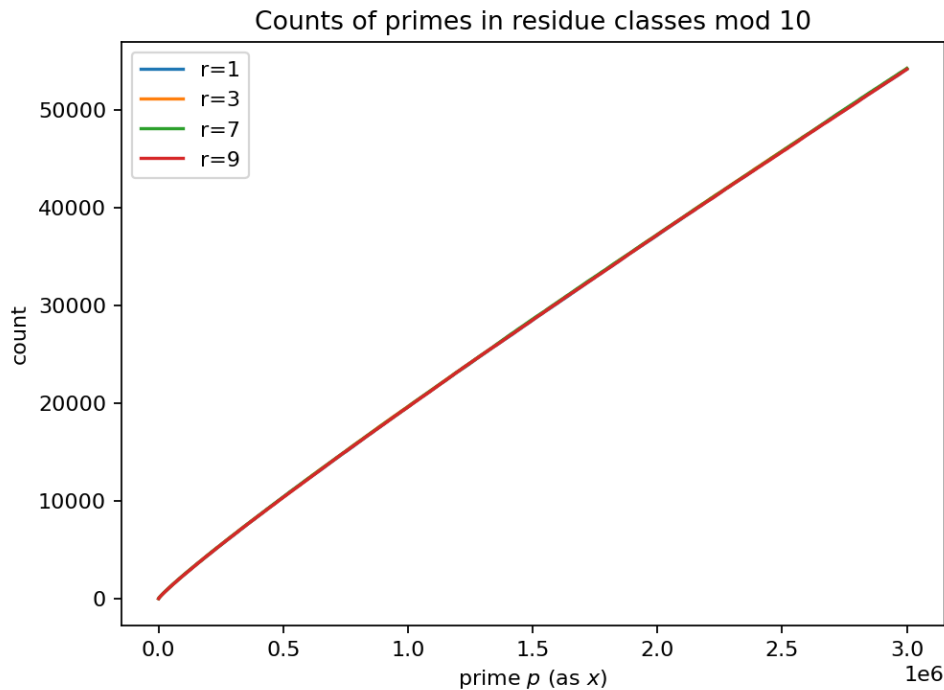
- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*



Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e023/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e023
```

or:

```
uv run python -m mathxlab.experiments.e023
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

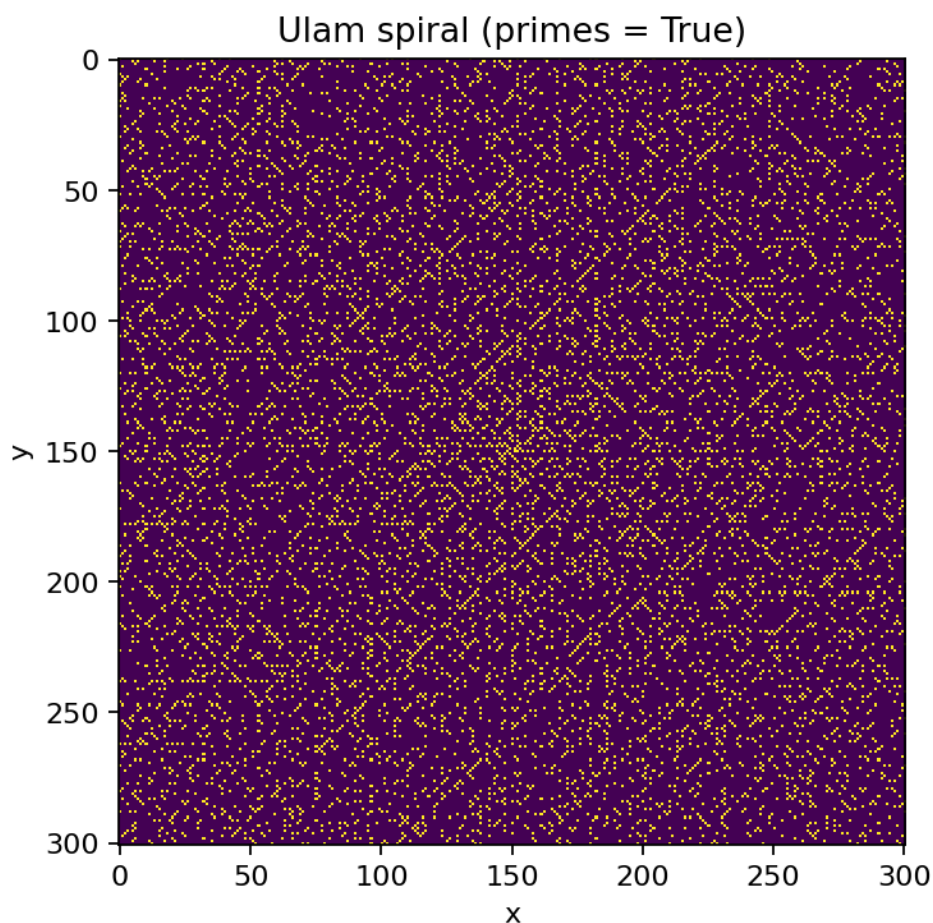
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.24 E024: Ulam spiral structure



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e024/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e024
```

or:

```
uv run python -m mathxlab.experiments.e024
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

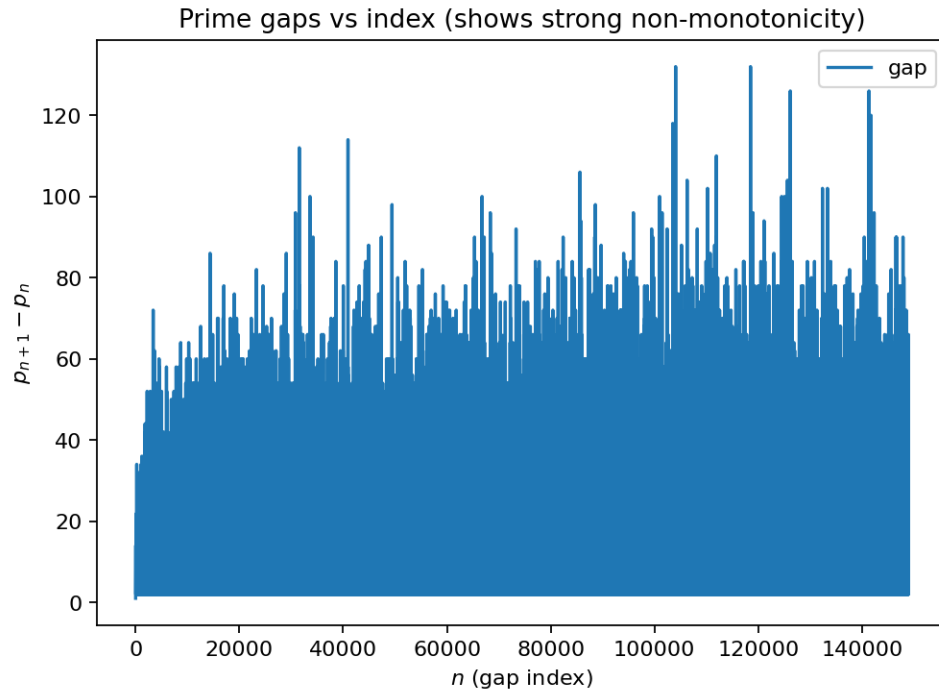
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.25 E025: Prime gaps are not monotone



Tags: number-theory, quantitative-exploration, visualization (see *Valid Tags*)

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e025/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e025
```

or:

```
uv run python -m mathxlab.experiments.e025
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.26 E026: Normalized prime gaps

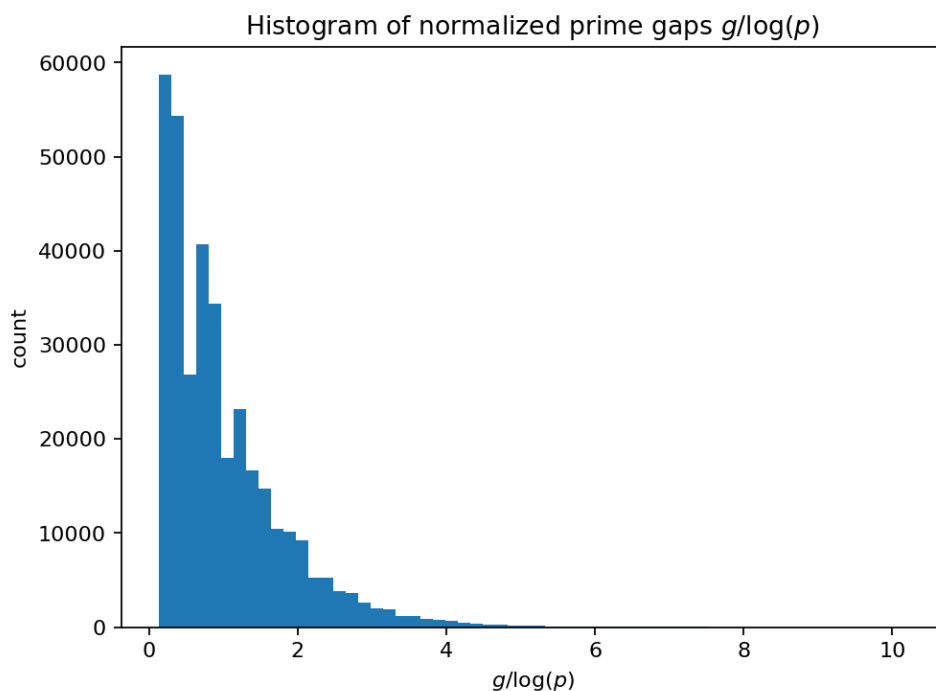
Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e026/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e026
```

or:

```
uv run python -m mathxlab.experiments.e026
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

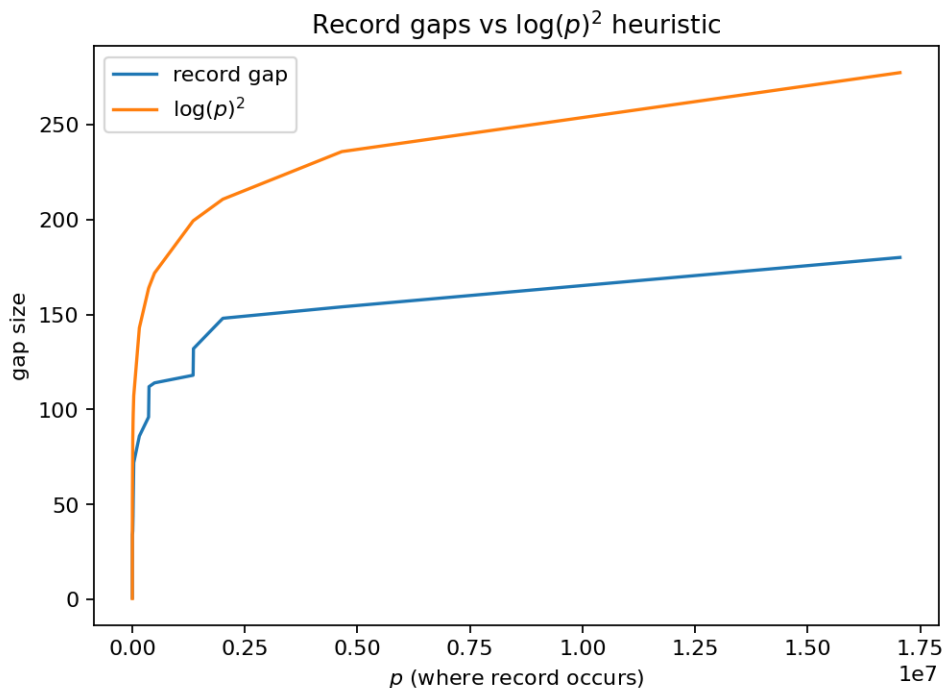
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.27 E027: Record prime gaps vs $\log^2$ heuristic



Tags: number-theory, conjecture-generation, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e027/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e027
```

or:

```
uv run python -m mathxlab.experiments.e027
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

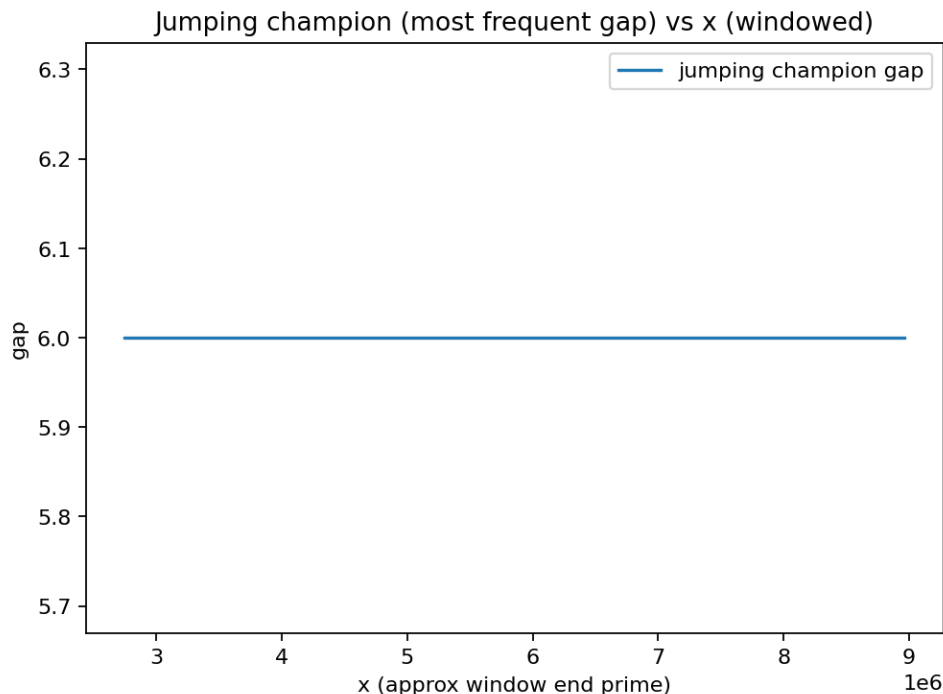
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.28 E028: Jumping champions (most frequent gaps)



Tags: number-theory, quantitative-exploration, visualization (see *Valid Tags*)

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e028/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e028
```

or:

```
uv run python -m mathxlab.experiments.e028
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.29 E029: Twin primes: observed vs heuristic

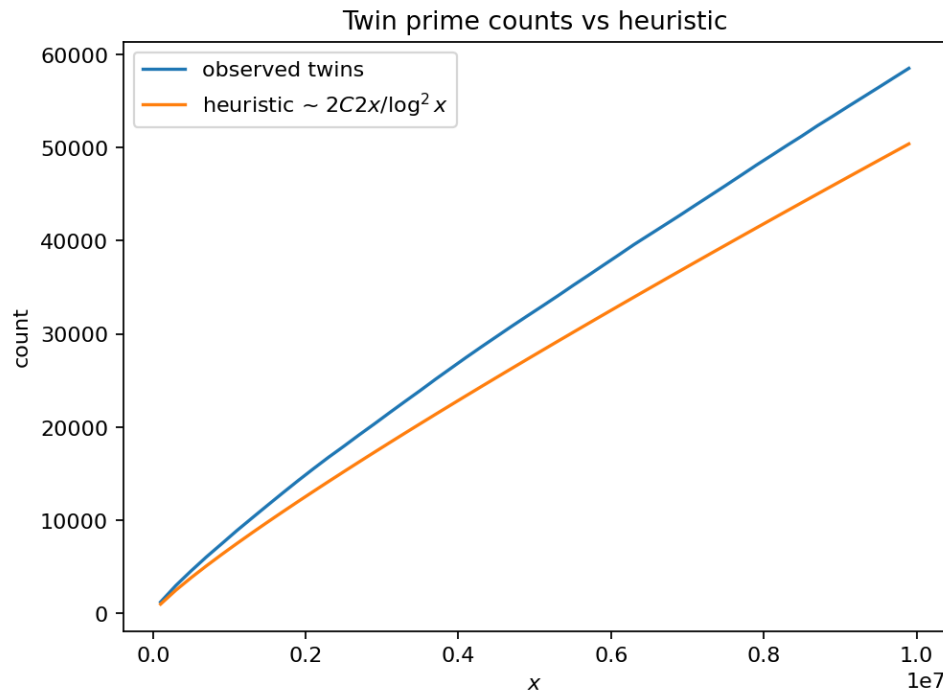
Tags: number-theory, conjecture-generation, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e029/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e029
```

or:

```
uv run python -m mathxlab.experiments.e029
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

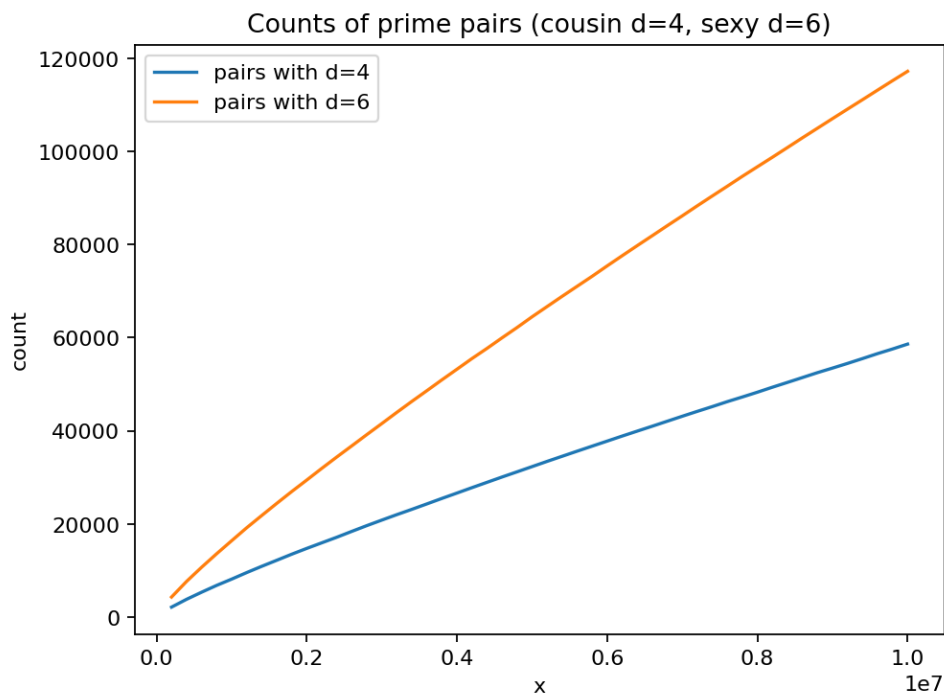
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.30 E030: Cousin and sexy prime pairs



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e030/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e030
```

or:

```
uv run python -m mathxlab.experiments.e030
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

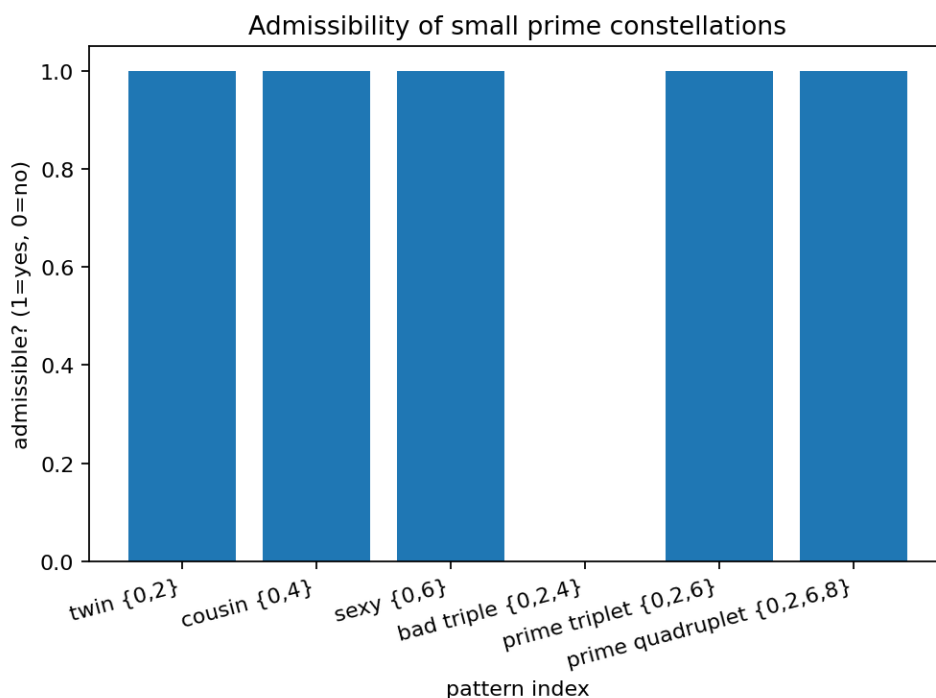
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.31 E031: Admissibility and modular obstructions



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- [Prime numbers refresher](#)

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e031/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e031
```

or:

```
uv run python -m mathxlab.experiments.e031
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.32 E032: Prime triplets and quadruplets

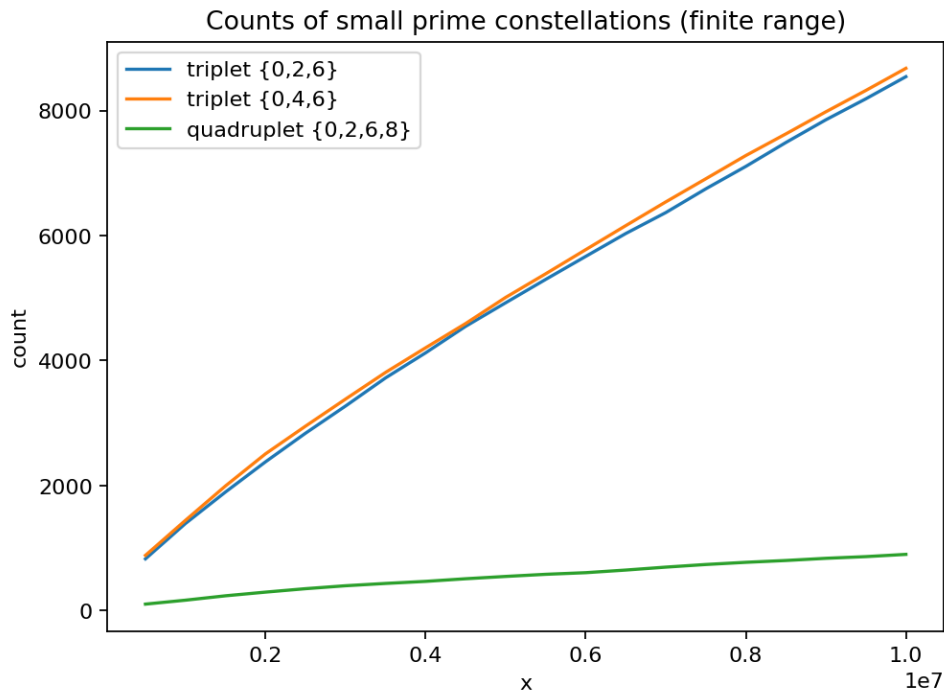
Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e032/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e032
```

or:

```
uv run python -m mathxlab.experiments.e032
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

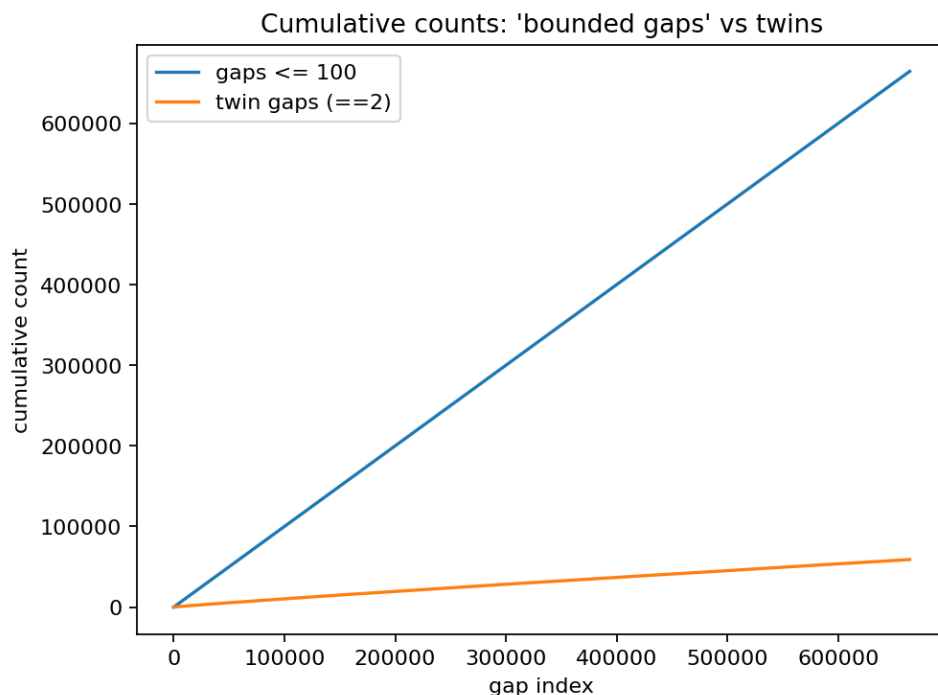
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.33 E033: Bounded gaps vs twin primes (not the same)



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e033/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e033
```

or:

```
uv run python -m mathxlab.experiments.e033
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

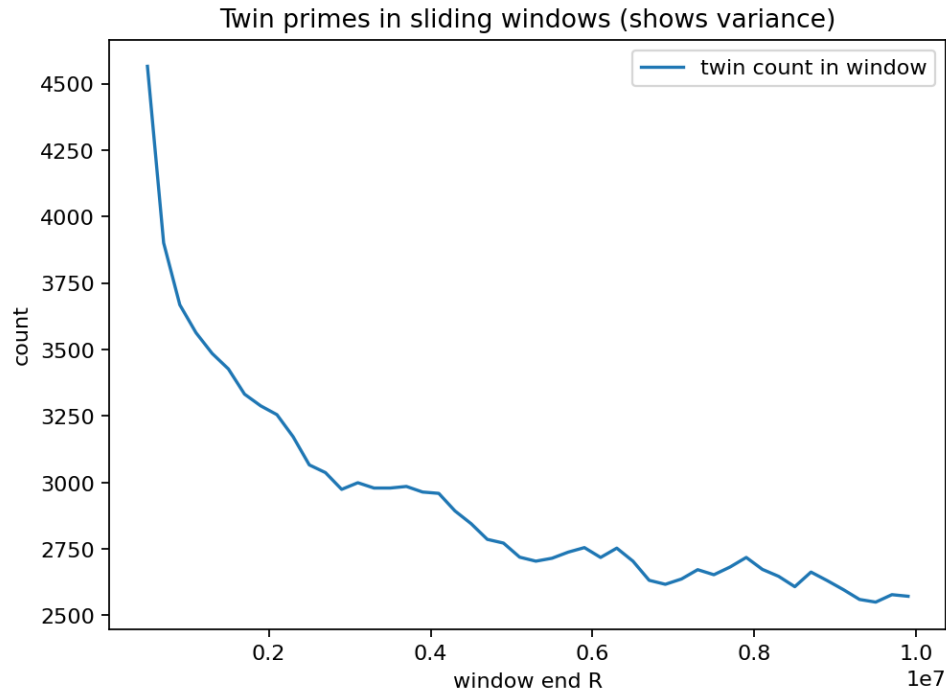
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.34 E034: Twin primes in sliding windows



Tags: number-theory, quantitative-exploration, visualization (see *Valid Tags*)

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e034/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e034
```

or:

```
uv run python -m mathxlab.experiments.e034
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.35 E035: Primes in arithmetic progressions mod q

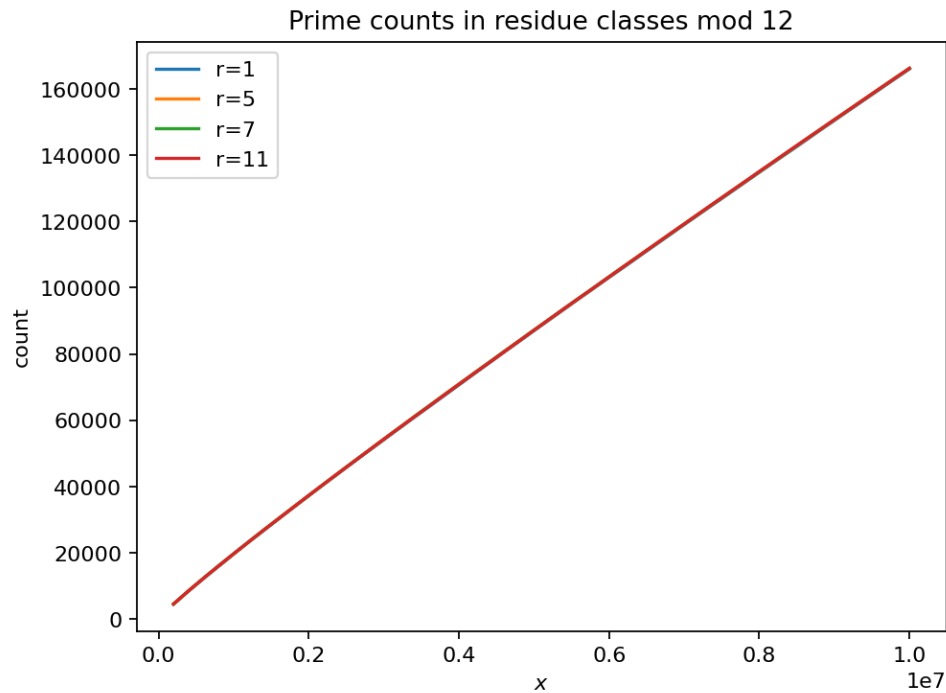
Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e035/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e035
```

or:

```
uv run python -m mathxlab.experiments.e035
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

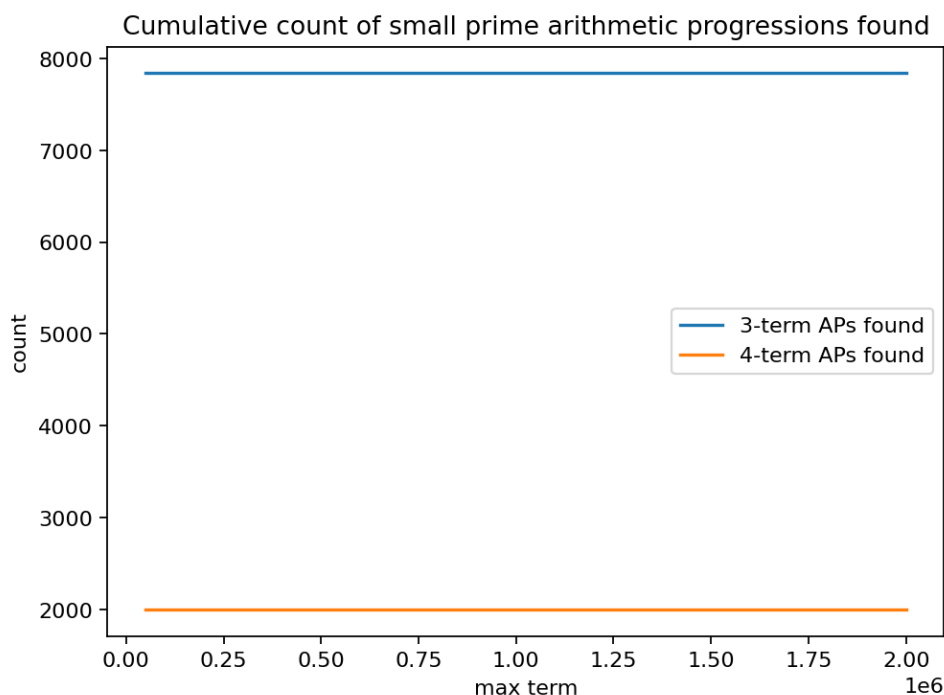
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.36 E036: Prime arithmetic progressions (small search)



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e036/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e036
```

or:

```
uv run python -m mathxlab.experiments.e036
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

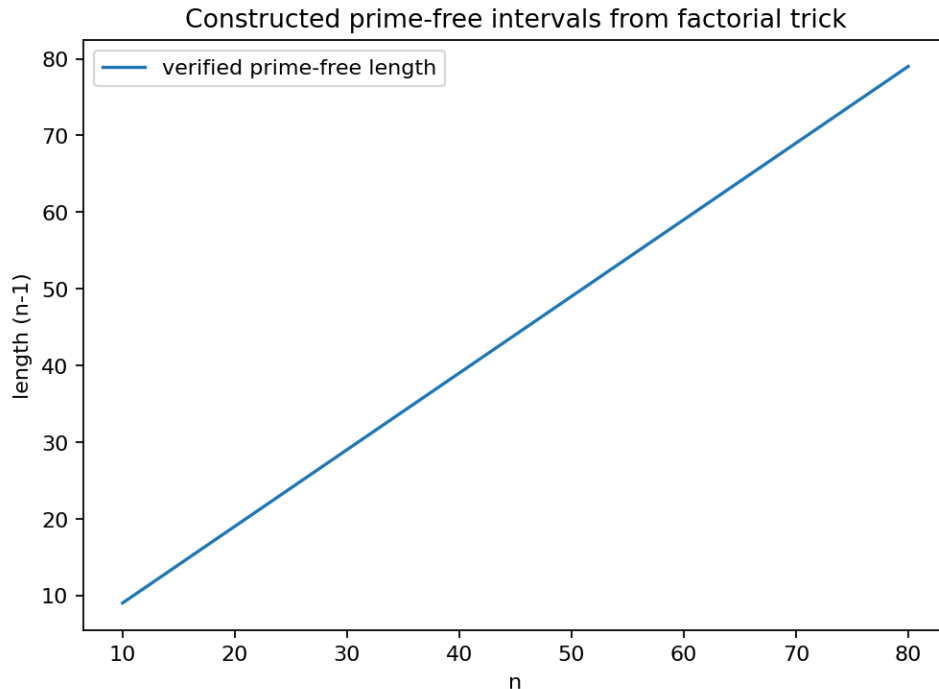
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See *References*.

3.3.37 E037: Prime-free intervals via factorial construction



Tags: number-theory, quantitative-exploration, visualization (see *Valid Tags*)

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e037/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e037
```

or:

```
uv run python -m mathxlab.experiments.e037
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.38 E038: Bertrand’s postulate (computational verification)

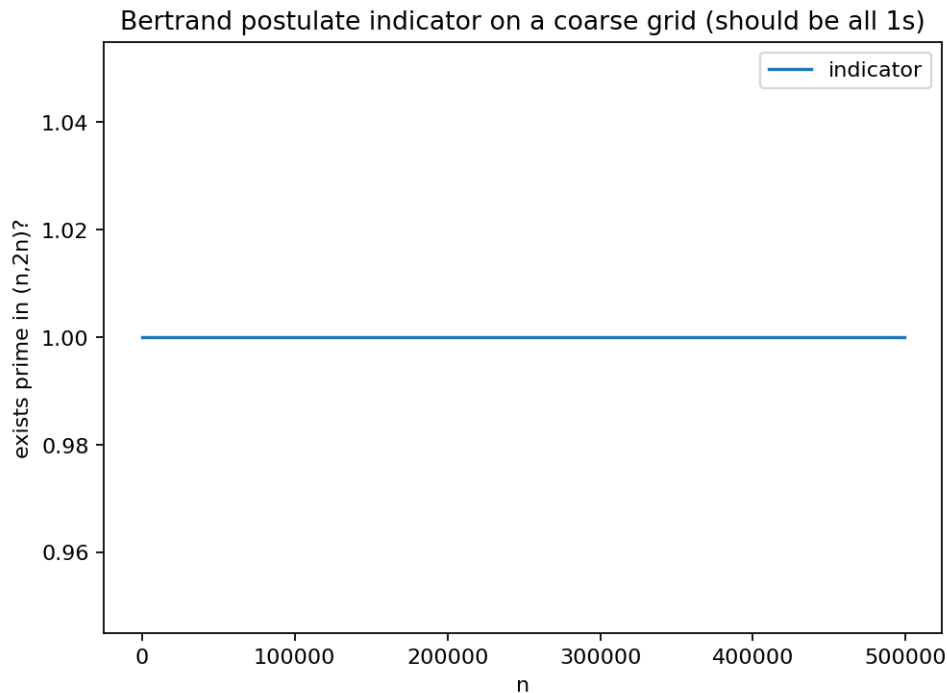
Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e038/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e038
```

or:

```
uv run python -m mathxlab.experiments.e038
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

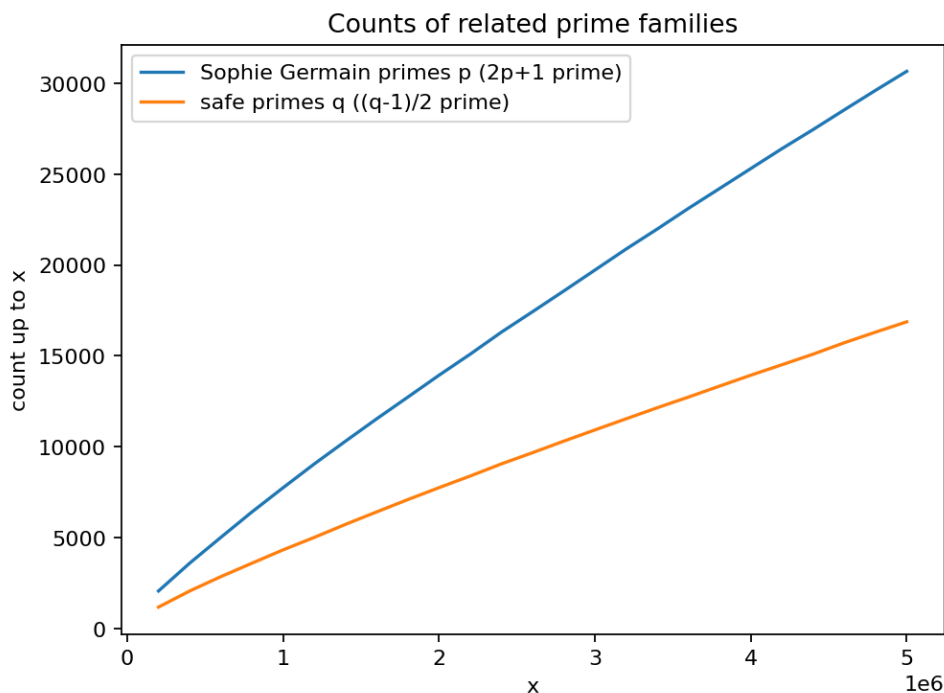
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.39 E039: Sophie Germain and safe primes



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e039/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e039
```

or:

```
uv run python -m mathxlab.experiments.e039
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

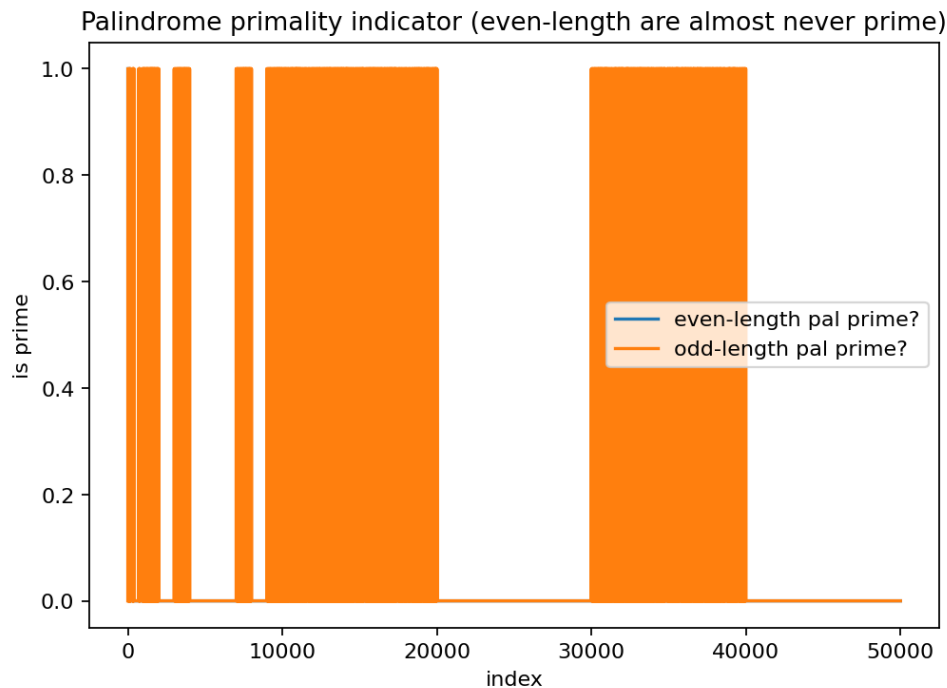
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.40 E040: Palindromic primes and the ‘11 trap’



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- [Prime numbers refresher](#)

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e040/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e040
```

or:

```
uv run python -m mathxlab.experiments.e040
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.41 E041: Fermat numbers: not all prime

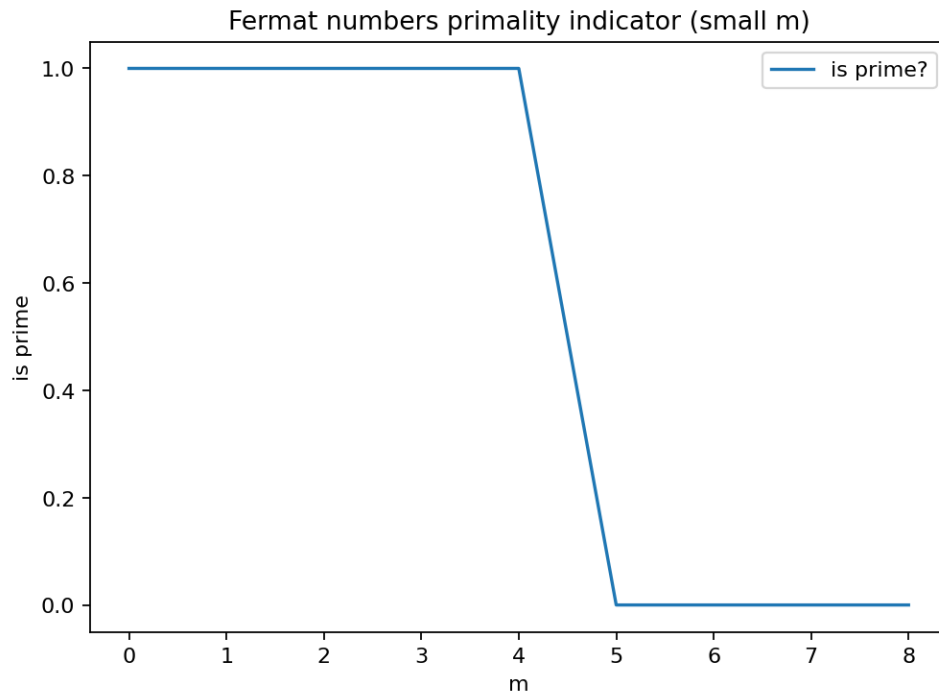
Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e041/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e041
```

or:

```
uv run python -m mathxlab.experiments.e041
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

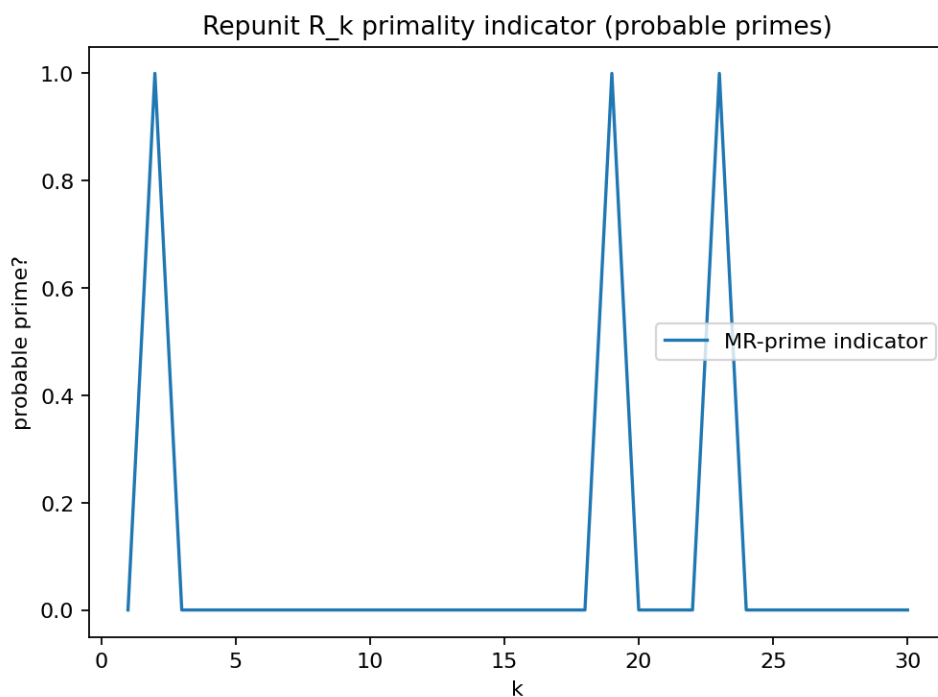
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.42 E042: Repunit primes (small k scan)



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e042/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e042
```

or:

```
uv run python -m mathxlab.experiments.e042
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

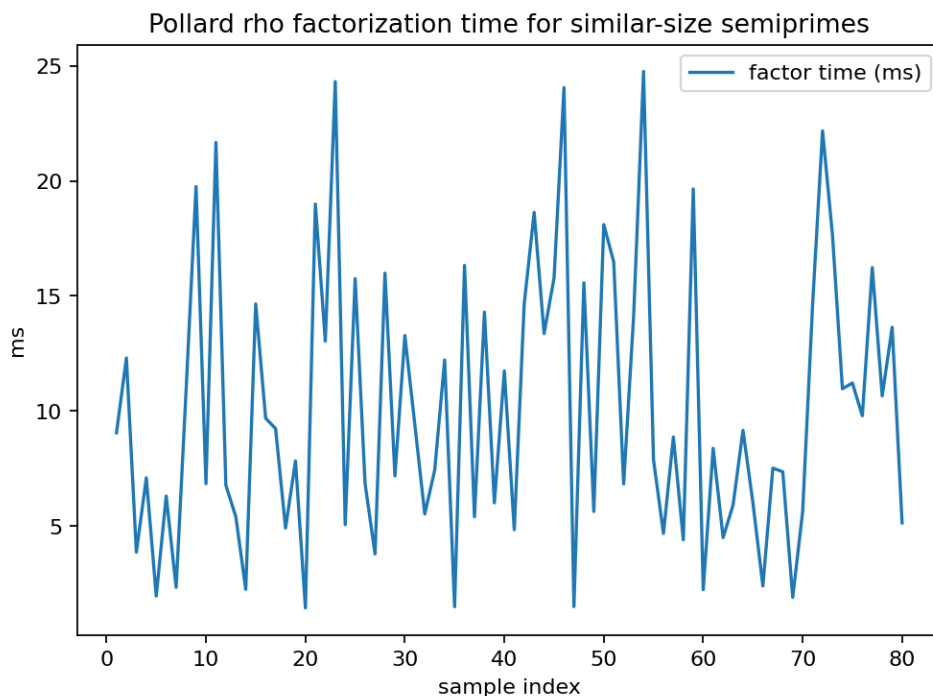
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.43 E043: Pollard rho runtime variability



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e043/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e043
```

or:

```
uv run python -m mathxlab.experiments.e043
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.44 E044: Solovay–Strassen vs Miller–Rabin (liars)

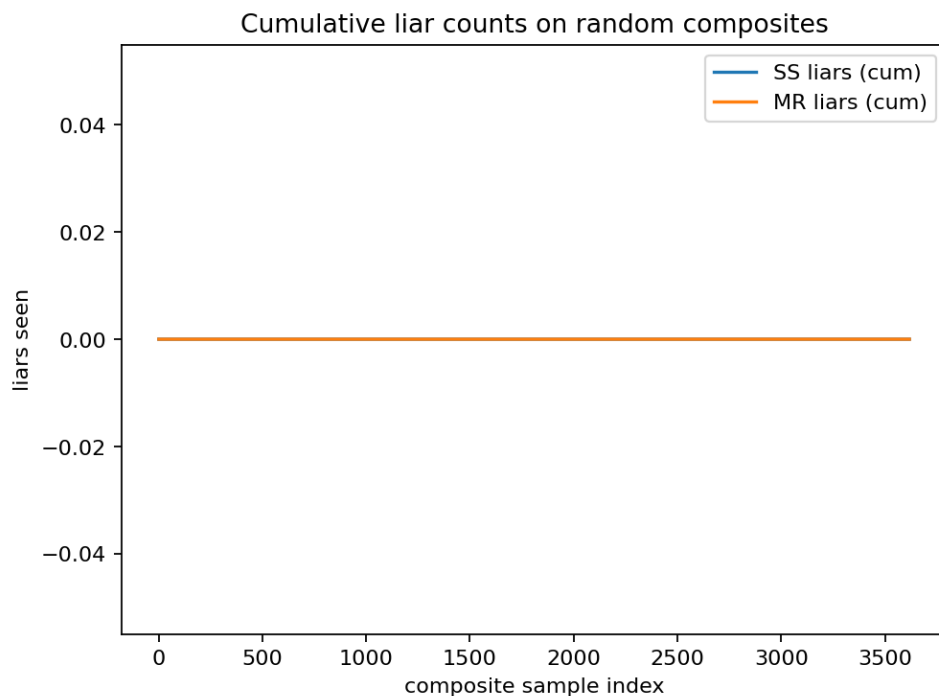
Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.



Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e044/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e044
```

or:

```
uv run python -m mathxlab.experiments.e044
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

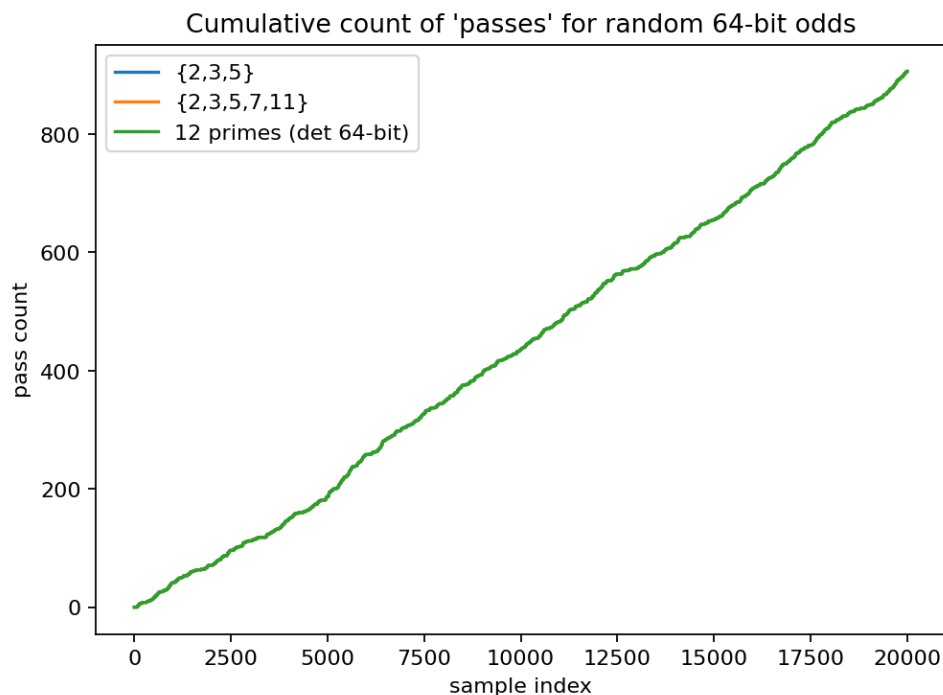
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.45 E045: Deterministic 64-bit MR base sets



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- *Prime numbers refresher*

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).
- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e045/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e045
```

or:

```
uv run python -m mathxlab.experiments.e045
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

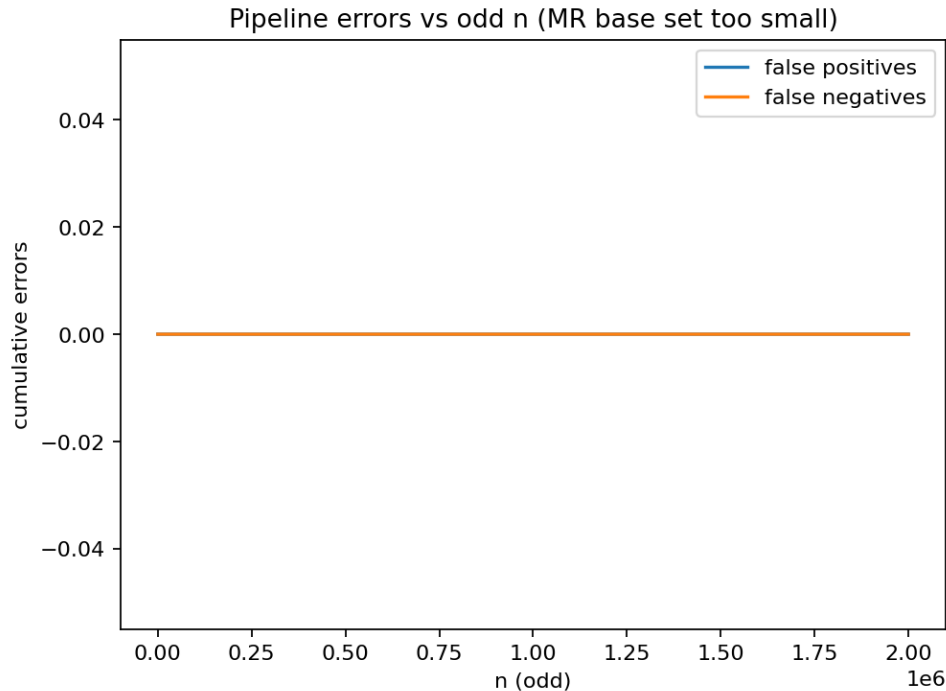
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.46 E046: Prime-testing pipeline and tuning pitfalls



Tags: number-theory, quantitative-exploration, visualization (see [Valid Tags](#))

Highlights

- This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.
- Writes reproducible artifacts (`params.json`, `report.md`, and figures).
- Designed to surface patterns *and* “looks-true-until-it-breaks” behavior.

Goal

This is a thin wrapper that follows the standard experiment template and delegates the actual computation to `:mod:mathxlab.experiments.prime_suite`.

Background (quick refresher)

- [Prime numbers refresher](#)

Research question

Which **prime-related claim, heuristic, or algorithm** breaks first under a clean, controlled computational sweep, and what does the smallest or clearest counterexample (or deviation) look like?

Why this qualifies as a mathematical experiment

- **Finite procedure:** run a bounded search / sweep with recorded parameters.
- **Observable(s):** counts, gaps, residues, runtime scaling, or first counterexample witnesses.
- **Parameter space:** vary bounds (and sometimes algorithmic choices).

- **Outcome:** plots/tables + “witness objects” for failures.
- **Reproducibility:** outputs saved to `out/e046/` with a parameter snapshot.

Experiment design

- **Computation:** bounded enumeration / sampling with explicit limits.
- **Outputs:** figures and a short `report.md` summarizing what was found.
- **Artifacts written:**
 - `figures/fig_*.png`
 - `params.json`
 - `report.md`

How to run

```
make run EXP=e046
```

or:

```
uv run python -m mathxlab.experiments.e046
```

Notes / pitfalls

- “No counterexample found” only means “none found within the configured bounds”.
- For probabilistic tests (when used), treat outcomes as *evidence*, not proof.

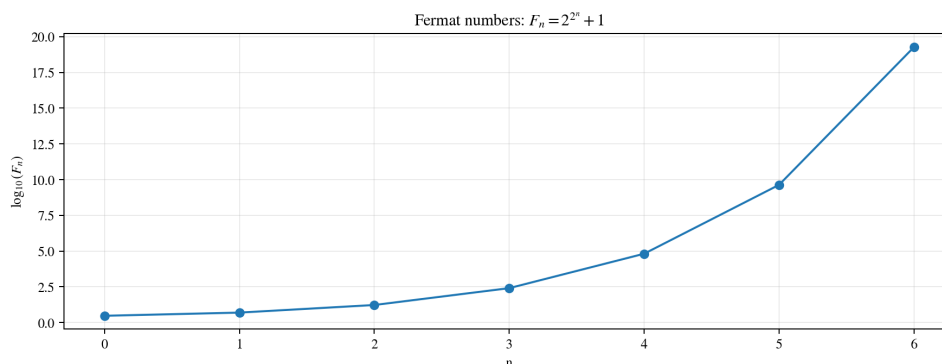
Extensions

- Increase bounds and rerun (recording runtime and memory).
- Compare alternative heuristics or algorithms on the same parameter grid.
- Turn found deviations into new, tighter conjectures.

References

See [References](#).

3.3.47 E047: Fermat numbers: Pépin test + factor witnesses



Tags: number-theory, counterexample-search, visualization, fermat (see [Valid Tags](#))

Highlights

- Runs Pépin’s test for $F_n = 2^{2^n} + 1$ (definitive for Fermat numbers).
- Finds small, explicit factor witnesses by bounded trial division.
- Visualizes how explosively F_n grows and where the first counterexample appears.

Goal

Demonstrate (computationally) why the statement “all Fermat numbers are prime” fails, and produce concrete witnesses.

Background (quick refresher)

- *Fermat numbers*

Research question

How quickly do Fermat numbers become composite in practice, and how easy is it to exhibit a *witness* factor for the first failures?

Experiment design

- Compute F_n for $n \leq n_{max}$.
- Apply Pépin’s test (for $n \geq 1$) to classify prime vs composite.
- If composite, attempt to find a small factor by trial division up to a configurable bound.
- Plot $\log_{10}(F_n)$ and the smallest discovered factor (if any).

Reproducibility

- `params.json` records the run settings.
- `report.md` summarizes the key findings.
- `figures/*.png` contains the plots for the run.

Interpreting the results

- If Pépin says “composite”, the number is definitively composite (not a probabilistic claim).
- The factor search is bounded: “no factor found” does not mean “prime”.
- The classic factor for F_5 (and often for F_6) appears quickly under the default bounds.

References

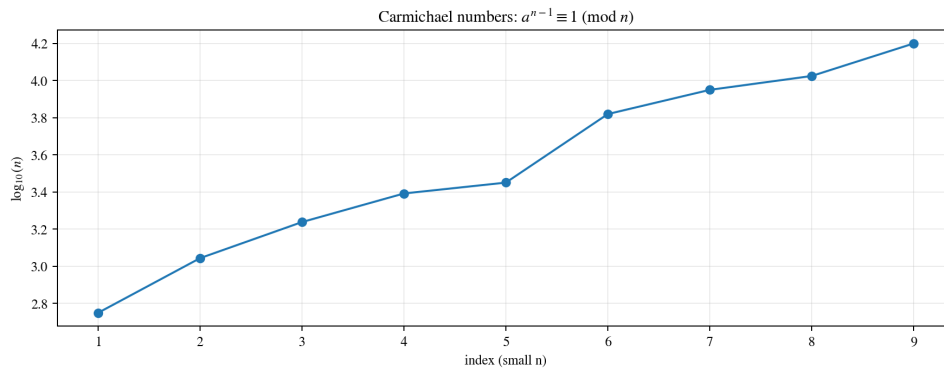
See Křiváček *et al.* [KřiváčekLvSomer13], Wikipedia contributors [Wikipediacontributors25b], The OEIS Foundation Inc. [TheOEIS25a].

3.3.48 E048: Carmichael numbers: Korselt scan + Fermat counterexamples

Tags: number-theory, counterexample-search, quantitative-exploration, visualization, carmichael, pseudoprime (see *Valid Tags*)

Highlights

- Enumerates many small Carmichael numbers by scanning squarefree products of three primes.
- Verifies that these composites pass Fermat’s test for several common bases.
- Visualizes the distribution of discovered Carmichael numbers on a log scale.



Goal

Show that Fermat’s primality test can fail spectacularly, and produce a dataset of explicit counterexamples.

Background (quick refresher)

- *Carmichael numbers*
- *Prime numbers refresher*

Research question

Within a moderate bound N , how many Carmichael numbers can we find by a simple Korselt scan, and how many Fermat bases do they fool?

Experiment design

- Enumerate candidates $n = pqr \leq N$ with distinct primes $p < q < r$.
- Apply Korselt’s criterion: $(p-1) \mid (n-1)$ for each prime factor $p \mid n$.
- For each Carmichael n , test Fermat congruence for a small base set (skipping non-coprime bases).
- Plot index vs n and “bases passed” vs n .

Reproducibility

- `params.json` records the run settings.
- `report.md` summarizes the key findings.
- `figures/*.png` contains the plots for the run.

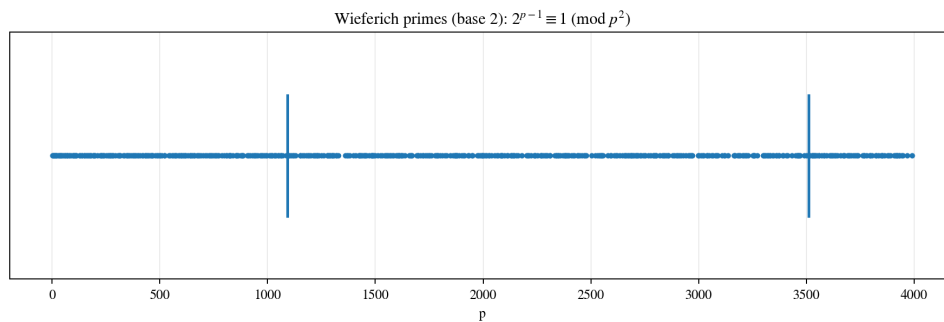
Interpreting the results

- All numbers found are composite by construction, yet they pass Fermat congruences for *all* coprime bases (in theory).
- The scan is restricted to 3-prime-factor Carmichael numbers; larger-factor Carmichaels exist too.
- Seeing “all bases passed” in the plot is a strong reminder: Fermat alone is not a reliable primality test.

References

See Alford *et al.* [AGP94], Carmichael [Car12], The OEIS Foundation Inc. [TheOFInc25e], Wikipedia contributors [Wikipediacontributors25a].

3.3.49 E049: Wieferich primes (base 2): scan and quotient visualization



Tags: number-theory, counterexample-search, visualization, wieferich (see [Valid Tags](#))

Highlights

- Scans primes up to a bound for the Wieferich condition $2^{p-1} \equiv 1 \pmod{p^2}$.
- Recovers the known small hits (1093 and 3511) under default settings.
- Visualizes a quotient-like value that is exactly zero for Wieferich primes.

Goal

Explore a rare strengthening of Fermat’s congruence and visualize how exceptional Wieferich primes are.

Background (quick refresher)

- [Wieferich primes](#)
- [Prime numbers refresher](#)

Research question

Up to a bound B , which primes satisfy the Wieferich condition (base 2), and how does the quotient-like statistic vary across primes?

Experiment design

- Generate all primes $p \leq B$ (excluding $p = 2$ for base 2).
- Compute $r = 2^{p-1} \bmod p^2$ and derive $q = (r - 1)/p \pmod{p}$.
- Mark primes with $q = 0$ as Wieferich hits.
- Plot hit positions and a scatter of q values.

Reproducibility

- `params.json` records the run settings.
- `report.md` summarizes the key findings.
- `figures/*.png` contains the plots for the run.

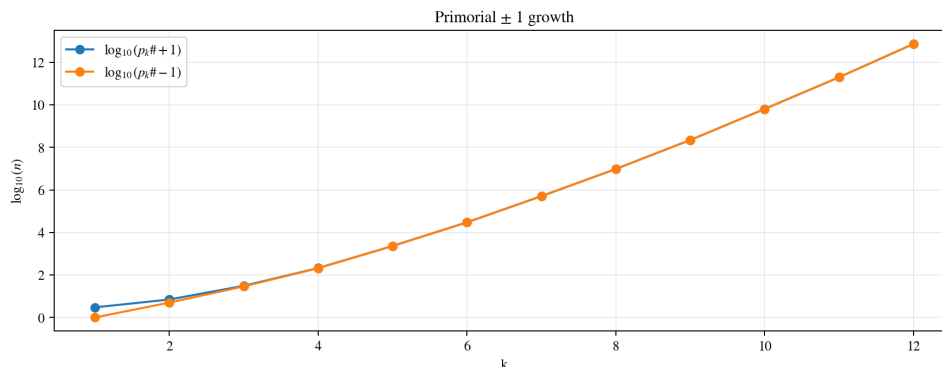
Interpreting the results

- Hits at 1093 and 3511 confirm the implementation.
- The quotient scatter is purely exploratory; it’s a compact way to see “how far” a prime is from the Wieferich condition.
- Increasing the bound quickly becomes compute-heavy (but remains feasible for moderate bounds in pure Python).

References

See Wieferich [Wie09], The OEIS Foundation Inc. [TheOFInc25b], Wikipedia contributors [Wikipediacontributors25h].

3.3.50 E050: Primorials and Euclid numbers: $p_k\# \pm 1$ are usually composite



Tags: number-theory, counterexample-search, quantitative-exploration, visualization, primorial (see *Valid Tags*)

Highlights

- Computes primorials $p_k\# = \prod_{i \leq k} p_i$ and Euclid-style numbers $p_k\# \pm 1$.
- Uses a probable-prime test to show “often composite” behavior early.
- Finds small factor witnesses via bounded trial division for many early k .

Goal

Demonstrate that Euclid-style numbers are coprime to small primes but are not reliably prime, and produce explicit factor witnesses.

Background (quick refresher)

- *Primorials*
- *Prime numbers refresher*

Research question

For small k , how often are $p_k\# \pm 1$ prime (or probable prime), and how easy is it to find a small factor when they are composite?

Experiment design

- Compute primorials for $k \leq k_{max}$.
- Test $p_k\# \pm 1$ with Miller–Rabin (probable prime).
- If composite under MR, try to find a small trial-division factor as a concrete witness.
- Plot $\log_{10}(p_k\# \pm 1)$ and smallest factor witnesses (log scale).

Reproducibility

- `params.json` records the run settings.
- `report.md` summarizes the key findings.
- `figures/*.png` contains the plots for the run.

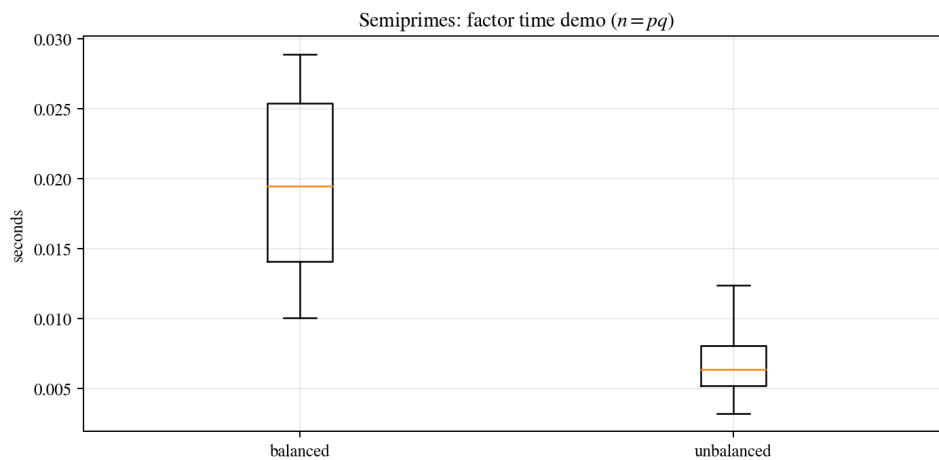
Interpreting the results

- “Probable prime” is not a proof; it’s a fast filter for this exploratory experiment.
- Factor witnesses come from bounded trial division: not finding a factor is expected for some k .
- You should see early composite examples such as $p_6\# + 1 = 30031$ (with a small factor) under default bounds.

References

See Hardy and Wright [HW08a], The OEIS Foundation Inc. [TheOFInc25d], Prime Pages (UTM) [PrimePUTM25b], Wikipedia contributors [Wikipediacontributors25f].

3.3.51 E051: Semiprimes: balanced vs unbalanced factorization timing



Tags: number-theory, quantitative-exploration, visualization, factorization, semiprime (see [Valid Tags](#))

Highlights

- Generates small semiprimes $n = pq$ in balanced and unbalanced regimes.
- Factors them using trial division + Pollard’s rho and records timings.
- Visualizes time distributions to make “balanced is harder” tangible.

Goal

Compare how factorization difficulty changes when one prime factor is much smaller than the other.

Background (quick refresher)

- *Semiprimes*
- *Prime numbers refresher*

Research question

For semiprimes of similar overall size, how does Pollard-rho timing differ between balanced ($p \approx q$) and unbalanced ($p \ll q$) cases?

Experiment design

- Generate balanced semiprimes with roughly equal factor bit lengths.
- Generate unbalanced semiprimes with a fixed small factor size.
- Factor each using trial division pre-pass + Pollard’s rho, measuring wall-clock time.

- Plot boxplots and a size-vs-time scatter.

Reproducibility

- `params.json` records the run settings.
- `report.md` summarizes the key findings.
- `figures/*.png` contains the plots for the run.

Interpreting the results

- Timings are noisy: random instance structure matters a lot at this size range.
- The trend should still show unbalanced instances often factoring faster (small factor is easier to find).
- This is an educational-scale experiment, not a cryptographic benchmark.

References

See Rivest *et al.* [RSA78], The OEIS Foundation Inc. [TheOFInc25c].

3.4 Background

This section provides mathematical foundations for the experiments.

3.4.1 Carmichael numbers

A Carmichael number is a composite n such that

$$[a^{n-1} \equiv 1 \pmod{n}]$$

for every integer a coprime to n . They are **absolute Fermat pseudoprimes**, so they defeat the naive Fermat primality test. [PrimePUTM25a, Wikipediacontributors25a]

Key facts

- **Korselt’s criterion (1899):** n is Carmichael iff (i) n is squarefree and (ii) for every prime $p \mid n$, we have $(p-1) \mid (n-1)$. [Con04, Wikipediacontributors25a]
- **At least 3 prime factors:** No Carmichael number is the product of only two primes. [Wikipediacontributors25a]
- **Infinitely many:** Alford–Granville–Pomerance proved there are infinitely many Carmichael numbers. [AGP94]
- **Historical origin:** Carmichael studied these numbers in the early 20th century. [Car12]

What to experiment with

- **Counterexample search:** Find Carmichael numbers by generating squarefree products and testing Korselt’s criterion.
- **“Fermat test is not enough”:** Show that all bases a with $\gcd(a, n) = 1$ pass Fermat for a Carmichael n , while a Miller–Rabin test quickly finds witnesses.
- **Distribution experiments:** Count Carmichael numbers up to X and compare growth to simple heuristics.
- **Construction families:** Implement Chernick-style constructions and see when factors are prime.

References

See Alford *et al.* [AGP94], Conrad [Con04], The OEIS Foundation Inc. [TheOFInc25e], Wikipedia contributors [Wikipediacontributors25a].

3.4.2 Fermat numbers

Fermat numbers are the integers

$$F_n = 2^{2^n} + 1 \quad (n \geq 0).$$

They grow extremely quickly and are central to a classic “counterexample” story: Fermat conjectured that all F_n are prime, but F_5 is composite (Euler). [KvrvizekLvSomer13, Wikipediacontributors25b]

Key facts

- **Coprime property:** $\gcd(F_m, F_n) = 1$ for $m \neq n$. In fact, $F_0 F_1 \cdots F_{n-1} = F_n - 2$. [KvrvizekLvSomer13]
- **Fermat primes:** If F_n is prime, it is called a Fermat prime (known: $n = 0..4$). [TheOFInc25a]
- **Pépin test (primality):** For $n \geq 1$, F_n is prime iff $3^{(F_n-1)/2} \equiv -1 \pmod{F_n}$. [KvrvizekLvSomer13]

What to experiment with

- **Factor hunting:** Search for small prime factors of F_n (using modular arithmetic and wheel/primorial filters).
- **Pépin vs. trial division:** Compare performance and failure modes as n increases.
- **Structure of known factors:** Many known prime factors have the form $k \cdot 2^{n+2} + 1$; explore how often such candidates appear.
- **Euclid-style numbers:** Explore $E_n = 1 + \prod_{i=0}^n p_i$ and compare “Euclid primes” vs. Fermat primes (they behave very differently).

References

See Křivánek *et al.* [KvrvizekLvSomer13], The OEIS Foundation Inc. [TheOFInc25a], Wikipedia contributors [Wikipediacontributors25b].

3.4.3 Mersenne numbers and primes refresher

This page is a *beginner-friendly* refresher for experiments about **Mersenne numbers** and **Mersenne primes**.

Core definitions

The **Mersenne numbers** are the integers

$$M_n = 2^n - 1 \quad (n \in \mathbb{N}).$$

A **Mersenne prime** is a Mersenne number that is prime:

$$M_p \text{ is a Mersenne prime} \iff 2^p - 1 \text{ is prime.}$$

A key (easy) fact is that if $2^n - 1$ is prime, then n must be prime.

So, in experiments, you typically only test M_p for *prime* exponents p . [con25b]

Key theorem / test: Lucas–Lehmer (why Mersenne primes are “computable”)

For an odd prime exponent p , define $M_p = 2^p - 1$ and a sequence $\{s_k\}$ by

$$s_0 = 4, \quad s_{k+1} = s_k^2 - 2.$$

Compute this sequence *modulo* M_p at each step, and then:

$$M_p \text{ is prime} \iff s_{p-2} \equiv 0 \pmod{M_p}.$$

This is the **Lucas–Lehmer test**. It’s the workhorse behind most practical Mersenne-prime checks (and historically central to GIMPS). [con25a, MR24]

What experiments typically visualize

- **Growth with the exponent:** number of bits / digits of M_p as p grows.
- **Prime vs. composite behavior:** the Lucas–Lehmer residue $s_{p-2} \bmod M_p$ across many prime exponents.
- **Factor patterns for composites:** quickly finding small factors of M_p (to avoid running LLT when a trivial factor exists).
- **Connections to perfect numbers:** if M_p is prime, then $2^{p-1}(2^p - 1)$ is an even perfect number (Euclid–Euler). [Cald.a]

For curated sequences (lists of exponents / known values), OEIS is a convenient “ground truth” reference. [Inc25a, Inc25b]

Practical numerical caveats

- **Always reduce modulo M_p in Lucas–Lehmer.**
If you don’t, the intermediate values explode in size (each squaring roughly doubles the bit-length).
- **Test the exponent first.**
If p is composite, M_p is automatically composite, so there’s no point running LLT.
- **Huge integers are fine, but you must be intentional.**
Python’s big integers won’t overflow, but performance depends on bit-length and on the efficiency of modular squaring.
- **Separate “demo scale” from “real scale.”**
For educational experiments, keep p modest (e.g., $p \leq 10^5$ is already huge for pure Python LLT). For large exponents, you’d rely on specialized implementations and careful FFT-based multiplication.

References

See *References*.

[Cal21, con25a, con25b, Inc25a, Inc25b]

3.4.4 Perfect numbers refresher

This page is a *beginner-friendly* refresher for experiments about **perfect numbers**. You only need basic number theory facts (divisors, primes) to follow it.

Core definitions

For a positive integer n , let

- $d \mid n$ mean “ d divides n ”,
- $\sigma(n) = \sum_{d \mid n} d$ be the **sum-of-divisors function**,
- $s(n) = \sum_{d \mid n, d < n} d = \sigma(n) - n$ be the sum of **proper** divisors.

A number n is **perfect** iff its proper divisors sum to itself:

$$n \text{ is perfect} \iff s(n) = n \iff \sigma(n) = 2n.$$

Examples:

- 6 is perfect because $1 + 2 + 3 = 6$.
- 28 is perfect because $1 + 2 + 4 + 7 + 14 = 28$.

Key theorem: all even perfect numbers (Euclid–Euler)

A classic result completely characterizes **even** perfect numbers:

Euclid–Euler theorem.

An integer n is an even perfect number **iff**

$$n = 2^{p-1}(2^p - 1)$$

where $2^p - 1$ is prime (a **Mersenne prime**).

So every known perfect number is generated from a Mersenne prime exponent p . This is the main “generator” you’ll use in experiments. [Cald.a, Voi98]

Why σ matters (multiplicativity)

If n factors as

$$n = \prod_{i=1}^k p_i^{a_i},$$

then

$$\sigma(n) = \prod_{i=1}^k \sigma(p_i^{a_i}) = \prod_{i=1}^k \frac{p_i^{a_i+1} - 1}{p_i - 1}.$$

This formula turns “sum all divisors” into a fast computation once you know the prime factorization.

The big open question: odd perfect numbers

It is **unknown** whether any **odd** perfect numbers exist. A large literature proves *constraints* (congruences, size bounds, number of prime factors, etc.). Experiments can explore these constraints (and why they make brute-force search unrealistic). [Guy04, OR14, Sto24]

What experiments typically visualize

Typical “lab” questions you can turn into plots and tables:

- **Verification by computation:** compute $\sigma(n)$ (via factorization) and check $\sigma(n) = 2n$ for candidates.
- **Generator experiment:** produce even perfect numbers from known Mersenne exponents p and confirm perfection.
- **Growth:** number of digits / bit length of $2^{p-1}(2^p - 1)$ as a function of p .
- **Divisor-function behavior:** compare $\sigma(n)/n$ (abundancy index) for random n vs. perfect numbers.
- **Odd constraints (toy models):** test necessary conditions on odd n and see how restrictive they are.

For data (lists of perfect numbers and exponents), OEIS is a convenient reference. [OEISFInc25a]

Practical numerical caveats

Even with correct math, computation has a few traps:

- **Factorization dominates.** Computing $\sigma(n)$ via divisors is slow unless you factor n . For large n , factorization becomes infeasible; prefer the Euclid–Euler generator for even perfect numbers.
- **Big integers are fine, but expensive.** Python integers won’t overflow, but operations on huge numbers scale with the number of bits (so be mindful in loops and plotting).
- **Prime testing vs. proof.** Testing that $2^p - 1$ is prime is nontrivial for large p . For experiments, use a curated list of known Mersenne prime exponents (e.g., from OEIS / GIMPS) rather than trying to discover new ones from scratch. [[MersenneResearchIncGIMPS24](#), [MersenneResearchIncGIMPS25](#)]
- **Be explicit about definitions.** Some sources define “perfect” via $\sigma(n) = 2n$; others via proper divisors. Use one convention consistently in code and docs.

References

See *References*.

[Cald.a, Sto24, Voi98, OEISFInc25a]

3.4.5 Prime numbers refresher

This page is a *beginner-friendly* refresher for experiments about **prime numbers**. It is intentionally broad: it covers the prime-related ideas most commonly explored with computation (distribution, sieves, primality testing, gaps, and prime patterns).

Core definitions

Primes and composites

A positive integer $p \geq 2$ is **prime** if its only positive divisors are 1 and p . If $n \geq 2$ is not prime, it is **composite**.

Equivalently,

$$p \text{ is prime} \iff \forall a, b \in \mathbb{N} : p = ab \Rightarrow (a = 1 \text{ or } b = 1).$$

Greatest common divisor and coprimality

The **greatest common divisor** of integers a, b , written $\gcd(a, b)$, is the largest positive integer dividing both. We say a and b are **coprime** if $\gcd(a, b) = 1$.

Coprimality appears constantly in modular arithmetic and in many prime-related proofs.

Congruences (modular arithmetic)

For integers a, b and $m \geq 2$,

$$a \equiv b \pmod{m}$$

means m divides $a - b$. Many prime experiments (residue classes, sieves, primality tests) are naturally expressed modulo m .

Prime factorization (the “atomic” viewpoint)

The **Fundamental Theorem of Arithmetic** says every integer $n \geq 2$ factors uniquely (up to ordering) as

$$n = \prod_{i=1}^k p_i^{\alpha_i},$$

where the p_i are distinct primes and the α_i are positive integers. This makes primes the “building blocks” of the integers. [HW08b]

Key quantitative language

Infinitely many primes

Euclid’s classic argument shows there are infinitely many primes. This matters experimentally because it justifies questions like “how often do primes occur as numbers grow?” [HW08b]

The prime counting function and the Prime Number Theorem

Let $\pi(x)$ be the number of primes $\leq x$. The **Prime Number Theorem** (PNT) states

$$\pi(x) \sim \frac{x}{\log x} \quad (x \rightarrow \infty).$$

Informally: primes thin out, and the “typical” gap size near x is about $\log x$. [Apo76]

For experiments, two very common comparisons are:

- $\pi(x)$ vs. $x/\log x$,
- $\pi(x)$ vs. $\text{Li}(x)$ (the logarithmic integral), which is often a better approximation in practice. [Apo76, RS62]

Explicit bounds (useful sanity checks)

Many sources provide explicit inequalities for $\pi(x)$, the n th prime p_n , and related functions. These bounds are useful to validate computations and pick experiment ranges safely. [Axl19, Dus18, RS62]

Computational building blocks

Generating primes: sieves

If you need *all* primes up to a bound N , a sieve is usually best.

Sieve of Eratosthenes (idea). Start with a boolean array “possibly prime,” then repeatedly cross out multiples of each found prime. Time is roughly $O(N \log \log N)$, memory is $O(N)$ booleans.

Segmented sieve. For large N , store only a window $[L, R]$ at a time; this keeps memory bounded while preserving speed. Segmented sieves are often the right choice once N grows beyond typical RAM-friendly sizes. [CP05]

Testing primality (single numbers)

If you need to test primality of individual (possibly large) integers, common strategies are:

- **Trial division** up to $\sqrt{n}$ (great for small n , too slow for large),
- **Probabilistic tests** such as **Miller–Rabin**, fast and extremely reliable with good parameters, [Rab80]
- **Deterministic polynomial-time** primality testing (AKS), important theoretically but rarely used for practical large-number work. [AKS04]

In an experiment repo, it is common to use Miller–Rabin for speed and then either: (1) accept “probable prime” status (with a stated error bound), or (2) confirm with stronger checks for the sizes you care about. [CP05]

Factorization (often the bottleneck)

Many arithmetic functions become easy once a number is factored, but factoring large integers is hard. Even “toy” factorization methods (e.g., Pollard’s rho) make excellent experiments because performance varies wildly with input structure. [CP05]

Prime gaps and prime patterns

Prime gaps

Let p_n be the n th prime. The **prime gap** sequence is

$$g_n = p_{n+1} - p_n.$$

Heuristically, “typical” gaps near p are size $\log p$, but gaps vary a lot. Two natural experimental viewpoints are:

- **local statistics**: histogram of g_n in a range,
- **record gaps**: the largest gap seen up to x (maximal gaps / first occurrences). [Cald.b, Cramer36, Nic99]

A common normalization is $g_n / \log p_n$, which helps compare different scales.

Twin, cousin, and sexy primes (prime pairs)

A **prime pair** is a pair of primes $(p, p + d)$ with fixed even difference d :

- **twin primes**: $d = 2$,
- **cousin primes**: $d = 4$,
- **sexy primes**: $d = 6$.

These are all instances of “prime constellations.”

Prime k -tuples (constellations)

A **prime k -tuple** is a set of offsets $\{h_1, \dots, h_k\}$ such that $p + h_i$ are simultaneously prime. Hardy–Littlewood heuristics predict asymptotic counts for many such patterns (including twins). [HL23]

Modern sieve breakthroughs show **bounded prime gaps** exist (there are infinitely many g_n below some constant), but we still do not know whether the twin prime conjecture is true. [May15, Zha14, DHJPolymath14]

Primes in structure (residue classes and progressions)

Residue classes

Primes (except 2) are odd, so they lie in residue classes modulo 2. More generally, you can study how primes distribute among residue classes modulo q . Experiments here include “prime races” and small biases in counts.

Arithmetic progressions of primes

A classical result (Dirichlet’s theorem) says that if $\gcd(a, q) = 1$, then the arithmetic progression

$$a, a + q, a + 2q, \dots$$

contains infinitely many primes. A landmark modern result (Green–Tao) shows primes contain arbitrarily long arithmetic progressions. [GT08]

For experiments, you can stay at an elementary level (searching for 3-term or 4-term prime APs in ranges) while still getting meaningful patterns.

Other prime families you may include (optional)

These are often nice “side quests” that still fit under “Prime Numbers”:

- **Sophie Germain primes**: p prime and $2p + 1$ prime,
- **safe primes**: q prime with $(q - 1)/2$ prime,

- **Mersenne primes:** $2^p - 1$ prime (you already have a dedicated page), [con25b]
- **palindromic / repunit primes:** “pattern-based” primes useful for search experiments.

What experiments typically visualize

Typical “prime numbers lab” plots and tables include:

- **Prime density:** $\pi(x)$ vs. $x/\log x$, and error curves $\pi(x) - x/\log x$. [Apo76]
- **Bounds and approximations:** compare $\pi(x)$ with explicit inequalities (sanity checks). [Dus18, RS62]
- **Sieve scaling:** runtime/memory vs. N for classic vs. segmented sieves. [CP05]
- **Gaps:** histograms of g_n , normalized gaps $g_n/\log p$, record gaps vs. $\log^2 x$. [Cramer36, Nic99]
- **Prime pairs:** counts of twin/cousin/sexy primes in $[1, x]$, compared to Hardy–Littlewood style heuristics. [HL23]
- **Bounded gaps story:** “what bound was known when?” (Zhang $\rightarrow$ Maynard $\rightarrow$ Polymath) plus empirical gap data. [May15, Zha14, DHJPolymath14]
- **Primality tests:** speed vs. integer size (bits), and false-positive rates for weak tests vs. Miller–Rabin. [AKS04, Rab80]
- **Progressions:** frequency of 3-term prime APs in ranges; maximal AP length in a window. [GT08]

For “ground truth” sequences and curated lists, OEIS and PrimePages are practical reference points. [Cald.b, OEISFInc25b, OEISFInc25c]

Practical numerical caveats

- **Pick the right primitive:** if you need *many* primes up to N , use a sieve; if you need primality of a few large numbers, use primality testing.
- **Memory matters for sieves:** a naive boolean list of length N can be huge; segmented sieves avoid this.
- **State what “prime” means in code:** if you use Miller–Rabin, your results are “probable primes” unless you add a deterministic guarantee for your size range. [Rab80]
- **Avoid floating-point traps:** use natural logs, guard against $\log(0)$, and remember that $\log x$ changes slowly.
- **Be precise with pair-counting:** decide whether you count pairs by their smaller prime p (common), and avoid double-counting.
- **Separate discovery from verification:** for pattern-hunting (e.g., long APs), do fast screening first, then verify candidates carefully.

References

See *References*.

[AKS04, Apo76, Cramer36, CP05, Dus18, GT08, HL23, HW08b, May15, Rab80, RS62, Zha14, DHJPolymath14]

3.4.6 Primorials

The primorial of a prime p_k is

$$[p_k\# = \prod_{i=1}^k p_i]$$

(the product of the first k primes). Primorials are a natural “smoothing knob” in experimental number theory: they amplify small prime structure and appear in constructions like Euclid numbers $p_k\# \pm 1$. [PrimePUTM25b, TheOFInc25d]

Key facts

- **Growth:** $\log(p_k\#) = \sum_{i \leq k} \log p_i$; by the prime number theorem this is asymptotic to p_k (in a coarse sense). [HW08a]
- **Euclid numbers:** $p_k\# + 1$ is coprime to all primes $\leq p_k$, but is usually composite (so: “Euclid’s proof does *not* generate primes”). [PrimePUTM25b]

What to experiment with

- **Euclid-number factorization:** For $E_k = p_k\# \pm 1$, try to find small factors and compare $+1$ vs -1 .
- **Primorial wheels:** Use primorials to build “wheel sieves” and benchmark against a simple sieve.
- **Primorial primes / plus/minus primes:** Scan for primes of the form $p_k\# \pm 1$ and visualize rarity.

References

See Prime Pages (UTM) [PrimePUTM25b], The OEIS Foundation Inc. [TheOFInc25d], Wikipedia contributors [Wikipediacontributors25f].

3.4.7 Semiprimes

A semiprime is an integer that is the product of two primes (not necessarily distinct), i.e.

[$n = pq \text{ \texttt{\textbackslash quad} } (p, q \text{ \texttt{\textbackslash text{ } prime} })$.]

Semiprimes sit at the center of practical factorization hardness and are the basic objects behind RSA-style moduli. [RSA78, TheOFInc25c]

Key facts

- **Sequence:** The semiprimes form OEIS sequence A001358. [TheOFInc25c]
- **RSA context:** RSA uses $N = pq$ and the difficulty of factoring N (when p, q are large and random-looking). [RSA78]
- **Special cases:** $p = q$ yields squares of primes; $p \neq q$ yields “RSA-like” semiprimes.

What to experiment with

- **Distribution:** Count semiprimes up to X and compare empirical growth with simple heuristic approximations.
- **Factorization methods:** Benchmark trial division, Pollard’s rho, and (for larger sizes) ECM bindings if you later allow optional deps.
- **RSA-style generation:** Generate balanced semiprimes (p, q same bitlength) and compare factorization difficulty vs unbalanced ones.
- **Smoothness:** Explore how often $p - 1$ or $q - 1$ is smooth (motivates $p - 1$ factoring methods).

References

See Rivest *et al.* [RSA78], The OEIS Foundation Inc. [TheOFInc25c].

3.4.8 Taylor series refresher

This page is a *beginner-friendly* refresher for experiments that use Taylor polynomials. You only need basic calculus (derivatives) to follow it.

Taylor polynomial

Assume f has enough derivatives near x_0 (this is true for $\sin(x)$, $\cos(x)$, polynomials, exponentials, etc.). The Taylor polynomial of degree n around x_0 is

$$T_n(x; x_0) = \sum_{k=0}^n \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k.$$

Intuition: $T_n(x; x_0)$ is the polynomial that matches $f(x_0)$ and the first n derivatives at x_0 . It is usually accurate when x is close to x_0 .

For $f(x) = \sin(x)$, the derivatives cycle, and around $x_0 = 0$ this becomes

$$\sin(x) \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \dots$$

Truncation error and the remainder

The approximation error is the remainder

$$R_n(x; x_0) = f(x) - T_n(x; x_0).$$

Under standard conditions, Taylor’s theorem gives a remainder representation. One common form is the (Lagrange) remainder:

$$R_n(x; x_0) = \frac{f^{(n+1)}(\xi)}{(n+1)!} (x - x_0)^{n+1} \quad \text{for some } \xi \text{ between } x \text{ and } x_0.$$

This is the key qualitative message for experiments like E001:

- the factor $(x - x_0)^{n+1}$ makes the method **local** (good near x_0 , potentially bad far away),
- increasing n helps most where $|x - x_0|$ is small.

What experiments typically visualize

In a numerical experiment, you often look at

- absolute error: $|R_n(x; x_0)|$
- relative error: $|R_n(x; x_0)|/|f(x)|$ (careful near zeros of f)

and plot them across a domain to see where the approximation is reliable.

Practical numerical caveats

Even when the mathematics are correct, computation can mislead:

- large $|x - x_0|$ and high n can produce huge intermediate terms,
- subtractive cancellation can reduce accuracy,
- floating-point rounding can dominate before the theoretical truncation error does.

A common “extension” experiment is to repeat the same plots using higher precision arithmetic to separate *truncation error* from *rounding error*.

Introductory reading

If you want a longer, *beginner-friendly* treatment (beyond this refresher), these are good starting points:

- A quick overview / definitions and examples: [Wikipediacontributors25g].
- A rigorous calculus textbook with a clean presentation of Taylor’s theorem and remainders: [Apo91].
- A proof-oriented classic (slower, deeper): [Spi08].
- For the numerical viewpoint (truncation vs. rounding error): [BFB15].

References

See *References*.

[Apo91, BFB15, Spi08, Wikipediacontributors25g]

3.4.9 Wieferich primes

A Wieferich prime (base 2) is a prime p such that

$$[2^{p-1} \equiv 1 \pmod{p^2}.]$$

Only two are currently known: 1093 and 3511. [TheOFInc25b, Wikipediacontributors25h]

Key facts

- **Origin (Fermat’s Last Theorem, first case):** Wieferich proved in 1909 that if the first case of FLT fails for an odd prime exponent p , then p must be a Wieferich prime (base 2). [Wie09]
- **Rarity:** Heuristics suggest Wieferich primes are very sparse; whether infinitely many exist is open. [Wikipediacontributors25h]
- **Generalization:** One can define Wieferich primes for any base a via $a^{p-1} \equiv 1 \pmod{p^2}$. [PrimePUTM25c]

What to experiment with

- **Fast search:** Implement modular exponentiation (pow with mod) to test primes up to a bound and recover 1093, 3511.
- **Near-Wieferich primes:** Measure the p -adic valuation $v_p(2^{p-1} - 1)$ and look for unusually large values.
- **Connections:** Explore overlaps with other “rare prime” phenomena (e.g., Wall–Sun–Sun, Wilson) as *tags* rather than hard coupling.

References

See Wieferich [Wie09], The OEIS Foundation Inc. [TheOFInc25b], Wikipedia contributors [Wikipediacontributors25h].

3.5 Getting started

This project uses an **uv-only** workflow and a small Makefile wrapper to run everything consistently.

3.5.1 Prerequisites

You need:

- **Python 3.14**
- **uv** on your PATH
- **GNU Make**

Windows notes

- Install GNU Make (e.g., via Chocolatey).
- You can run everything from `cmd.exe`, PowerShell, or from WSL.

3.5.2 Verify tools

From the repository root:

```
make uv-check
make python-check
make python-info
```

3.5.3 First-time setup

Create a virtual environment and install dependencies:

```
make venv
make install-dev
make install-docs
```

3.5.4 Run the full development chain

```
make final
```

`make final` runs:

- formatting (Ruff)
- linting (Ruff)
- type checking (mypy)
- tests (pytest)
- documentation (sphinx)

Documentation can be built separately via:

```
make docs
```

3.5.5 Run an experiment

Example (E001):

```
make run EXP=e001 ARGS="--seed 1"
```

See the experiment overview here: [Experiments Gallery](#).

3.5.6 Build documentation locally

```
make docs
```

Output:

- docs/_build/html

3.5.7 Troubleshooting

error: Failed to spawn: sphinx-build

Sphinx is not installed in the uv environment.

Fix:

```
make install-docs
make docs
```

```
make python-check fails
```

The repo enforces Python **3.14**. Install Python 3.14, then recreate the environment:

```
make clean-venv
make venv
make install-dev
make install-docs
```

3.6 Development

This page describes the development workflow and the conventions used in this repository.

3.6.1 Workflow overview

Use the Makefile targets for everything. They wrap `uv run ...` so you do not need to activate a virtual environment manually.

Typical day-to-day:

```
make final
```

Documentation-only build:

```
make docs
```

Clean caches and build artifacts:

```
make clean
```

Remove the virtual environment (full reset):

```
make clean-venv
```

3.6.2 Makefile workflow

The Makefile is the **single entry point** for development tasks (env setup, quality checks, docs builds, and running experiments).

- Prefer `make final` before pushing.
- Prefer `make docs` to validate documentation changes.
- Use `make run EXP=<id>` to execute an experiment and write artifacts to `out/<id>/`.

Dependency groups

This summary is included from the Makefile documentation:

Dependencies are organized via `pyproject.toml` extras:

- **default**: runtime dependencies needed to run the package
- **dev** (`--extra dev`): developer tooling (ruff, mypy, pytest, etc.)
- **docs** (`--extra docs`): documentation tooling (sphinx, furo, myst-parser, sphinx-design, bibtex, ...)

What the Makefile does

- Most dev targets run via:

```
uv run --extra dev ...
```

- Documentation targets run via:

```
uv run --extra docs ...
```

Why this matters

CI and local development can install only what they need:

- fast “dev chain” (no docs):

```
uv sync --extra dev
```

- docs build:

```
uv sync --extra docs
```

Run logs

Experiments live under `mathxlab/experiments/` and can be run either directly with Python or through Make targets (if available in your Makefile).

Run an experiment module directly

```
uv run python -m mathxlab.experiments.e001
```

Typical output locations (convention)

Depending on the experiment runner implementation, outputs are usually placed in:

- `out/e###/` (generated artifacts, figures, manifests)
- `docs/gallery/`, `docs/reports/`, `docs/manifests/` (published snapshots)

If you add new experiments, keep the numbering stable (e001, e002, ...) so the gallery and documentation can remain consistent.

Common workflows

Task	Command
Complete quality check	<code>make final</code>
Build HTML docs	<code>make docs-html</code>
Run tests	<code>make test</code>
Clean all artifacts	<code>make clean</code>
Reset environment	<code>make clean && make venv</code>

Full reference

For the complete target-by-target reference and troubleshooting, see `makefile`.

3.6.3 Formatting, linting, typing, tests

- Formatting: **Ruff** formatter
- Linting: **Ruff**
- Typing: **mypy**
- Tests: **pytest**

CI formatting behavior

In CI, formatting runs in check mode (`ruff format --check`). Locally it formats in place.

3.6.4 Experiment authoring guidelines

When adding a new experiment:

1. Add a new module under `mathxlab/experiments/`, e.g. `e002_...py`.
2. Prefer deterministic outputs:
 - `--seed` argument if randomness is involved
 - write results to a single `--out` directory
3. Keep the experiment runnable as a module:
 - `python -m mathxlab.experiments.e002`
4. Update the docs:
 - add a short entry to *Experiments Gallery*
 - optionally add a dedicated page under `docs/experiments/` later

3.6.5 Documentation

Docs are built with Sphinx + MyST.

Build locally:

```
make install-docs
make docs
```

Deployed website:

- GitHub Pages from the `docs` workflow

3.6.6 Contributing (high-level)

- Create a feature branch.
- Open a PR against `main`.
- CI must pass before merge.
- Keep PRs small and well-scoped.

3.7 References

3.8 PDF download

A PDF build of this documentation is generated by GitHub Actions and published alongside the HTML site.

- [Download PDF](#)

If the link is missing, it usually means you are viewing an older deployment or the PDF build step failed.

BIBLIOGRAPHY

- [Expa] Experimental mathematics (project euclid archive). URL: <https://projecteuclid.org/journals/experimental-mathematics> (visited on 2025-12-22).
- [Expb] Experimental mathematics lab (university of luxembourg). URL: <https://math.uni.lu/eml/> (visited on 2025-12-22).
- [Expc] Experimental mathematics website. URL: <https://www.experimentalmath.info> (visited on 2025-12-22).
- [Exp92] Experimental mathematics (journal). 1992. URL: <https://www.tandfonline.com/journals/uexm20>.
- [AKS04] Manindra Agrawal, Neeraj Kayal, and Nitin Saxena. PRIMES is in P. *Annals of Mathematics*, 160(2):781–793, 2004. URL: <https://annals.math.princeton.edu/2004/160-2/p12> (visited on 2025-12-27), doi:10.4007/annals.2004.160.781.
- [AGP94] W. R. Alford, Andrew Granville, and Carl Pomerance. There are infinitely many carmichael numbers. *Annals of Mathematics*, 139(3):703–722, 1994. URL: <https://annals.math.princeton.edu/1994/139-3/p10>.
- [Apo76] Tom M. Apostol. *Introduction to Analytic Number Theory*. Springer, 1976. URL: <https://link.springer.com/book/10.1007/978-1-4757-5579-4> (visited on 2025-12-27), doi:10.1007/978-1-4757-5579-4.
- [Apo91] Tom M. Apostol. *Calculus, Volume 1*. John Wiley & Sons, 2 edition, 1991. ISBN 9780471000051. URL: https://books.google.com/books/about/Calculus\TU\textbackslashslash{}_Volume\TU\textbackslashslash{}_1.html?id=o2D4DwAAQBAJ.
- [Arn15] Vladimir I. Arnold. *Experimental Mathematics*. MSRI Mathematical Circles Library. American Mathematical Society, 2015. ISBN 9780821894163. URL: <https://bookstore.ams.org/msri-13/> (visited on 2025-12-22).
- [Axl19] Christian Axler. New estimates for the n th prime number. *Journal of Integer Sequences*, 22(4):Article 19.4.2, 2019. URL: <https://cs.uwaterloo.ca/journals/JIS/VOL22/Axler/axler17.pdf> (visited on 2025-12-27).
- [Axl24] Christian Axler. Effective estimates for some functions defined over primes. *Integers*, 24:A34, 2024. URL: <https://math.colgate.edu/~integers/y34/y34.pdf> (visited on 2025-12-27).
- [BB05] David H. Bailey and Jonathan M. Borwein. Experimental mathematics: examples, methods and implications. *Notices of the American Mathematical Society*, 52(5):502–514, 2005. URL: <https://www.ams.org/notices/200505/fea-borwein.pdf>.
- [BBC04] David H. Bailey, Jonathan M. Borwein, and Richard E. Crandall. Ten problems in experimental mathematics. *Experimental Mathematics*, 13(2):193–207, 2004.
- [BvdPSZ14] Jonathan Borwein, Alf van der Poorten, Jeffrey Shallit, and Wadim Zudilin. *Neverending Fractions: An Introduction to Continued Fractions*. Volume 23 of Australian Mathematical Society Lecture Series. Cambridge University Press, 2014. ISBN 9780521186490. URL: <https://>

- [//www.cambridge.org/core/books/neverending-fractions/A3900DAB483D65CE6CB960A6B71226EE](https://www.cambridge.org/core/books/neverending-fractions/A3900DAB483D65CE6CB960A6B71226EE) (visited on 2025-12-22).
- [Bor05] Jonathan M. Borwein. The experimental mathematician: the pleasure of discovery and the role of proof. *International Journal of Computers for Mathematical Learning*, 10:75–108, 2005. doi:10.1007/s10758-005-6244-3.
- [Bor09] Jonathan M. Borwein. *The Crucible: An Introduction to Experimental Mathematics*. A K Peters, Wellesley, MA, USA, 2009. ISBN 9781568813438.
- [BB08] Jonathan M. Borwein and David H. Bailey. *Mathematics by Experiment: Plausible Reasoning in the 21st Century*. A K Peters, Wellesley, MA, USA, 2 edition, 2008. ISBN 9781568814421.
- [BBG04] Jonathan M. Borwein, David H. Bailey, and Roland Girgensohn. *Experimentation in Mathematics: Computational Paths to Discovery*. A K Peters, Natick, MA, USA, 2004. ISBN 9781568811369. doi:10.1201/9781439864197.
- [BBG+07] Jonathan M. Borwein, David H. Bailey, Roland Girgensohn, David Luke, and Victor H. Moll. *Experimental Mathematics in Action*. A K Peters, Wellesley, MA, USA, 2007. ISBN 9781568812717. URL: <https://carmamaths.org/resources/jon/Preprints/Books/EMA/ema.pdf> (visited on 2025-12-22).
- [BB04] Jonathan M. Borwein and Richard P. Brent. *Inquiries into Experimental Mathematics*. A K Peters, Wellesley, MA, USA, 2004. ISBN 9781568812113.
- [BFB15] Richard L. Burden, J. Douglas Faires, and Annette M. Burden. *Numerical Analysis*. Cengage Learning, 10 edition, 2015. ISBN 9781305253667. URL: <https://www.cengage.com/c/numerical-analysis-10e-faires/9781305253667/>.
- [Cald.a] Chris Caldwell. Characterizing all even perfect numbers. n.d. PrimePages (The Prime Database), proof note on the Euclid–Euler characterization. URL: <https://t5k.org/notes/proofs/EvenPerfect.html> (visited on 2025-12-25).
- [Cal21] Chris K. Caldwell. Mersenne primes: history, theorems and lists. Online, 2021. PrimePages reference page collecting history, key theorems, and curated tables related to Mersenne primes and perfect numbers. URL: <https://primes.utm.edu/mersenne/> (visited on 2025-12-27).
- [Cald.b] Chris K. Caldwell. Prime gaps and related records. Online, n.d. PrimePages (The Prime Database), curated notes and links on prime gaps and record computations. URL: <https://t5k.org/notes/primegaps.html> (visited on 2025-12-27).
- [Car12] R. D. Carmichael. On composite numbers p which satisfy the fermat congruence $a^{p-1} \equiv 1 \pmod{p}$. *American Mathematical Monthly*, 19(2):22–27, 1912. URL: <https://www.jstor.org/stable/2974142>.
- [Con04] Keith Conrad. Korselt's criterion for carmichael numbers. <https://kconrad.math.uconn.edu/blurbs/ugradnumthy/carmichael.pdf>, 2004. Online lecture note; accessed 2025-12-28.
- [con25a] Wikipedia contributors. Lucas–lehmer primality test — wikipedia, the free encyclopedia. Online, 2025. Revision as of 01:16, 31 October 2025. Abstract: Description and correctness of the Lucas–Lehmer test used to prove primality of Mersenne numbers $M_p = 2^p - 1$. URL: https://en.wikipedia.org/w/index.php?oldid=1319643028&title=Lucas%E2%80%93Lehmer_primality_test (visited on 2025-12-27).
- [con25b] Wikipedia contributors. Mersenne prime — wikipedia, the free encyclopedia. Online, 2025. Revision as of 20:10, 11 December 2025. Abstract: Overview of Mersenne primes (numbers of the form $2^p - 1$), their connection to perfect numbers, and known results and records. URL: https://en.wikipedia.org/w/index.php?oldid=1326946346&title=Mersenne_prime (visited on 2025-12-27).
- [Cramer36] Harald Cramér. On the order of magnitude of the difference between consecutive prime numbers. *Acta Arithmetica*, 2(1):23–46, 1936. URL: <https://www.impan.pl/en/publishing-house/journals-and-series/acta-arithmetica/all/2/1/93277/on-the-order-of-magnitude-of-the-difference-between-consecutive-prime-numbers> (visited on 2025-12-27), doi:10.4064/aa-2-1-23-46.

- [CP05] Richard Crandall and Carl Pomerance. *Prime Numbers: A Computational Perspective*. Springer, 2005. URL: <https://link.springer.com/book/10.1007/978-1-4684-9316-0> (visited on 2025-12-27), doi:10.1007/978-1-4684-9316-0.
- [Dus18] Pierre Dusart. Explicit estimates of some functions over primes. *The Ramanujan Journal*, 45(1):227–251, 2018. URL: <https://link.springer.com/article/10.1007/s11139-016-9839-4> (visited on 2025-12-27), doi:10.1007/s11139-016-9839-4.
- [FI10] John Friedlander and Henryk Iwaniec. *Opera de Cribro*. Volume 57 of Colloquium Publications. American Mathematical Society, 2010. URL: <https://bookstore.ams.org/coll-57/> (visited on 2025-12-27), doi:10.1090/coll/057.
- [GT08] Ben Green and Terence Tao. The primes contain arbitrarily long arithmetic progressions. *Annals of Mathematics*, 167(2):481–547, 2008. URL: <https://annals.math.princeton.edu/2008/167-2/p01> (visited on 2025-12-27), doi:10.4007/annals.2008.167.481.
- [Guy04] Richard K. Guy. *Unsolved Problems in Number Theory*. Springer, 3 edition, 2004. ISBN 9780387208602. URL: <https://link.springer.com/book/10.1007/978-0-387-26677-0>, doi:10.1007/978-0-387-26677-0.
- [HR74] Heini Halberstam and Hans-Egon Richert. *Sieve Methods*. Academic Press, 1974. URL: <https://www.sciencedirect.com/book/9780123182501/sieve-methods> (visited on 2025-12-27).
- [HL23] G. H. Hardy and J. E. Littlewood. Some problems of ‘partitio numerorum’; III: on the expression of a number as a sum of primes. *Acta Mathematica*, 44:1–70, 1923. URL: <https://link.springer.com/article/10.1007/BF02403921> (visited on 2025-12-27), doi:10.1007/BF02403921.
- [HW08a] G. H. Hardy and E. M. Wright. *An Introduction to the Theory of Numbers*. Oxford University Press, 6 edition, 2008. ISBN 9780199219858.
- [HW08b] G. H. Hardy and E. M. Wright. *An Introduction to the Theory of Numbers*. Oxford University Press, 6 edition, 2008. ISBN 9780199219865. URL: <https://global.oup.com/ukhe/product/an-introduction-to-the-theory-of-numbers-9780199219865> (visited on 2025-12-27).
- [Inc25a] OEIS Foundation Inc. A000043 — mersenne exponents. Online, 2025. OEIS entry for primes p such that $2^p - 1$ is prime (exponents of Mersenne primes). URL: <https://oeis.org/A000043> (visited on 2025-12-27).
- [Inc25b] OEIS Foundation Inc. A001348 — mersenne numbers: $2^p - 1$ where p is prime. Online, 2025. OEIS entry for the Mersenne numbers sequence $2^p - 1$ with prime exponents p . URL: <https://oeis.org/A001348> (visited on 2025-12-27).
- [IK04] Henryk Iwaniec and Emmanuel Kowalski. *Analytic Number Theory*. Volume 53 of Colloquium Publications. American Mathematical Society, 2004. URL: <https://bookstore.ams.org/coll-53/> (visited on 2025-12-27), doi:10.1090/coll/053.
- [KvřivzekLvSomer13] Michal Křiváček, Florian Luca, and Ladislav Šomer. *17 Lectures on Fermat Numbers: From Number Theory to Geometry*. CMS Books in Mathematics. Springer, 2013. ISBN 9781461465250. doi:10.1007/978-1-4614-6526-7.
- [LCD+03] Shangzhi Li, Falai Chen, Jiansong Deng, Yaohua Wu, and Yunhua Zhang. *Mathematics Experiments*. World Scientific, 2003. ISBN 9789812380500. doi:10.1142/5008.
- [May15] James Maynard. Small gaps between primes. *Annals of Mathematics*, 181(1):383–413, 2015. URL: <https://annals.math.princeton.edu/2015/181-1/p07> (visited on 2025-12-27), doi:10.4007/annals.2015.181.1.7.
- [MR24] Inc. Mersenne Research. Gimps — the math — primenet. Online, 2024. Background on the mathematics and algorithms used in the GIMPS search strategy (trial factoring, P-1, PRP testing, Lucas–Lehmer, double-checking). URL: <https://www.mersenne.org/various/math.php> (visited on 2025-12-27).
- [MR25] Inc. Mersenne Research. List of known mersenne prime numbers. Online, 2025. PrimeNet/GIMPS list of known Mersenne primes including discovery metadata and verification method. URL: <https://www.mersenne.org/primes/> (visited on 2025-12-27).

- [Nic99] Thomas R. Nicely. New maximal prime gaps and first occurrences. *Mathematics of Computation*, 68(227):1311–1315, 1999. URL: <https://www.ams.org/mcom/1999-68-227/S0025-5718-99-01065-0/> (visited on 2025-12-27), doi:10.1090/S0025-5718-99-01065-0.
- [OR14] Pascal Ochem and Michaël Rao. Lower bounds on odd perfect numbers. 2014. Slides (Montpellier, 2014-07-02). URL: https://www.lirmm.fr/~ochem/opn/opn\TU\textbackslash_slide.pdf (visited on 2025-12-25).
- [PetkovsekWZ96] Marko Petkovšek, Herbert S. Wilf, and Doron Zeilberger. *A=B*. A K Peters, Wellesley, MA, USA, 1996. ISBN 9781568810635. URL: <https://sites.math.rutgers.edu/~zeilberg/expmath/> (visited on 2025-12-22).
- [Polya54a] George Pólya. *Mathematics and Plausible Reasoning, Volume I: Induction and Analogy in Mathematics*. Princeton University Press, Princeton, NJ, USA, 1954.
- [Polya54b] George Pólya. *Mathematics and Plausible Reasoning, Volume II: Patterns of Plausible Inference*. Princeton University Press, Princeton, NJ, USA, 1954.
- [Rab80] Michael O. Rabin. Probabilistic algorithm for testing primality. *Journal of Number Theory*, 12(1):128–138, 1980. URL: <https://www.sciencedirect.com/science/article/pii/0022314X80900840> (visited on 2025-12-27), doi:10.1016/0022-314X(80)90084-0.
- [Rib96] Paulo Ribenboim. *The New Book of Prime Number Records*. Springer, 1996. ISBN 9780387944579. URL: <https://link.springer.com/book/10.1007/978-1-4612-0759-7> (visited on 2025-12-27), doi:10.1007/978-1-4612-0759-7.
- [Rie94] Hans Riesel. *Prime Numbers and Computer Methods for Factorization*. Birkhäuser, 2 edition, 1994. URL: <https://link.springer.com/book/10.1007/978-0-8176-8298-9> (visited on 2025-12-27), doi:10.1007/978-0-8176-8298-9.
- [RSA78] Ronald L. Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978. doi:10.1145/359340.359342.
- [RS62] J. Barkley Rosser and Lowell Schoenfeld. Approximate formulas for some functions of prime numbers. *Illinois Journal of Mathematics*, 6(1):64–94, 1962. URL: <https://projecteuclid.org/journals/illinois-journal-of-mathematics/volume-6/issue-1/Approximate-Formulas-for-Some-Functions-of-Prime-Numbers/ijm/1255631807.full> (visited on 2025-12-27), doi:10.1215/ijm/1255631807.
- [SS77] Robert M. Solovay and Volker Strassen. A fast Monte-Carlo test for primality. *SIAM Journal on Computing*, 6(1):84–85, 1977. URL: <https://epubs.siam.org/doi/10.1137/0206006> (visited on 2025-12-27), doi:10.1137/0206006.
- [Spi08] Michael Spivak. *Calculus*. Publish or Perish, Inc., 4 edition, 2008. ISBN 9780914098911. URL: <https://www.amazon.com/Calculus-4th-Michael-Spivak/dp/0914098918>.
- [Sto24] Andrew Stone. Improved upper bounds for odd perfect numbers — part i. *Integers: Electronic Journal of Combinatorial Number Theory*, 2024. URL: <https://math.colgate.edu/~integers/y114/y114.pdf>.
- [Voi98] John Voight. Perfect numbers: an elementary introduction. 1998. Lecture notes / survey. Date: May 31, 1998; updated January 27, 2024. URL: <https://jvoight.github.io/notes/perf/elelem-051015.pdf> (visited on 2025-12-25).
- [Wei03] Eric W. Weisstein. Perfect number. 2003. MathWorld—A Wolfram Web Resource. URL: <https://mathworld.wolfram.com/PerfectNumber.html> (visited on 2025-12-25).
- [Wie09] Arthur Wieferich. Zum letzten fermatschen theorem. *Journal für die reine und angewandte Mathematik*, 136:293–302, 1909.
- [WK09] George Woltman and Scott Kurowski. On the discovery of the 45th and 46th known mersenne primes. *The Fibonacci Quarterly*, 46/47(3):194–197, 2009. Abstract PDF. Describes GIMPS methods and reports the discoveries of M37156667 and M43112609. URL: https://www.fq.math.ca/Abstracts/46_47-3/woltman.pdf (visited on 2025-12-27).

- [Zha14] Yitang Zhang. Bounded gaps between primes. *Annals of Mathematics*, 179(3):1121–1174, 2014. URL: <https://annals.math.princeton.edu/2014/179-3/p07> (visited on 2025-12-27), doi:10.4007/annals.2014.179.3.7.
- [DHJPolymath14] D. H. J. Polymath. Variants of the selberg sieve, and bounded intervals containing many primes. *Research in the Mathematical Sciences*, 1:12, 2014. URL: <https://link.springer.com/article/10.1186/s40687-014-0019-3> (visited on 2025-12-27), doi:10.1186/s40687-014-0019-3.
- [MersenneResearchIncGIMPS24] Mersenne Research, Inc. (GIMPS). Mersenne prime discovery: $2^{136279841}-1$ is prime! 2024. GIMPS press release page (52nd known Mersenne prime). URL: <https://www.mersenne.org/primes/?press=M136279841> (visited on 2025-12-25).
- [MersenneResearchIncGIMPS25] Mersenne Research, Inc. (GIMPS). Gimps milestones report. 2025. URL: https://www.mersenne.org/report/TU\textbackslash{}_milestones/ (visited on 2025-12-25).
- [OEISFInc25a] OEIS Foundation Inc. A000396: perfect numbers. 2025. The On-Line Encyclopedia of Integer Sequences (OEIS). URL: <https://oeis.org/A000396> (visited on 2025-12-25).
- [OEISFInc25b] OEIS Foundation Inc. A001097: twin primes. Online, 2025. The On-Line Encyclopedia of Integer Sequences (OEIS). URL: <https://oeis.org/A001097> (visited on 2025-12-27).
- [OEISFInc25c] OEIS Foundation Inc. A001223: prime gaps $p_{n+1}-p_n$. Online, 2025. The On-Line Encyclopedia of Integer Sequences (OEIS). URL: <https://oeis.org/A001223> (visited on 2025-12-27).
- [OEISFInc25d] OEIS Foundation Inc. A001359: lesser of twin primes. Online, 2025. The On-Line Encyclopedia of Integer Sequences (OEIS). URL: <https://oeis.org/A001359> (visited on 2025-12-27).
- [OEISFInc25e] OEIS Foundation Inc. A023201: primes p such that $p+6$ is also prime (sexy primes). Online, 2025. The On-Line Encyclopedia of Integer Sequences (OEIS). URL: <https://oeis.org/A023201> (visited on 2025-12-27).
- [OEISFInc25f] OEIS Foundation Inc. A046132: primes p such that $p-4$ is also prime (cousin prime pairs). Online, 2025. The On-Line Encyclopedia of Integer Sequences (OEIS). URL: <https://oeis.org/A046132> (visited on 2025-12-27).
- [PrimePUTM25a] Prime Pages (UTM). Carmichael number (prime glossary). <https://t5k.org/glossary/page.php?sort=CarmichaelNumber>, 2025. Online; accessed 2025-12-28.
- [PrimePUTM25b] Prime Pages (UTM). Primorial (prime glossary). <https://t5k.org/glossary/page.php?sort=Primorial>, 2025. Online; accessed 2025-12-28.
- [PrimePUTM25c] Prime Pages (UTM). Wieferich prime (prime glossary). <https://t5k.org/glossary/page.php?sort=WieferichPrime>, 2025. Online; accessed 2025-12-28.
- [TheOFInc25a] The OEIS Foundation Inc. A000215: Fermat numbers $F(n) = 2^{(2^n)} + 1$. <https://oeis.org/A000215>, 2025. Online; accessed 2025-12-28.
- [TheOFInc25b] The OEIS Foundation Inc. A001220: Wieferich primes (base 2). <https://oeis.org/A001220>, 2025. Online; accessed 2025-12-28.
- [TheOFInc25c] The OEIS Foundation Inc. A001358: Semiprimes. <https://oeis.org/A001358>, 2025. Online; accessed 2025-12-28.
- [TheOFInc25d] The OEIS Foundation Inc. A002110: Primorial numbers $n\#$. <https://oeis.org/A002110>, 2025. Online; accessed 2025-12-28.
- [TheOFInc25e] The OEIS Foundation Inc. A002997: Carmichael numbers. <https://oeis.org/A002997>, 2025. Online; accessed 2025-12-28.
- [Wikipediacontributors25a] Wikipedia contributors. Carmichael number — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?oldid=1328580842&title=Carmichael_number, 2025. Online; accessed 2025-12-28.

- [Wikipediacontributors25b] Wikipedia contributors. Fermat number — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?oldid=1329504472&title=Fermat_number, 2025. Online; accessed 2025-12-28.
- [Wikipediacontributors25c] Wikipedia contributors. Harmonic number. Wikipedia, 2025. Permanent revision as of 20:03, 12 December 2025 (UTC). URL: https://en.wikipedia.org/w/index.php?oldid=1327129204&title=Harmonic_number (visited on 2025-12-25).
- [Wikipediacontributors25d] Wikipedia contributors. Perfect number. Wikipedia, 2025. Permanent revision as of 13:46, 22 December 2025 (UTC). URL: https://en.wikipedia.org/w/index.php?oldid=1328905011&title=Perfect_number (visited on 2025-12-25).
- [Wikipediacontributors25e] Wikipedia contributors. Prime number. Wikipedia, 2025. Permanent revision as of 20:47, 2 December 2025 (UTC). URL: https://en.wikipedia.org/w/index.php?oldid=1325385945&title=Prime_number (visited on 2025-12-25).
- [Wikipediacontributors25f] Wikipedia contributors. Primorial — Wikipedia, the free encyclopedia. <https://en.wikipedia.org/w/index.php?oldid=1328293471&title=Primorial>, 2025. Online; accessed 2025-12-28.
- [Wikipediacontributors25g] Wikipedia contributors. Taylor series. Wikipedia, 2025. Permanent revision as of 04:10, 24 December 2025 (UTC). URL: https://en.wikipedia.org/w/index.php?oldid=1329166943&title=Taylor_series (visited on 2025-12-25).
- [Wikipediacontributors25h] Wikipedia contributors. Wieferich prime — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?oldid=1329186350&title=Wieferich_prime, 2025. Online; accessed 2025-12-28.